



AI VIET NAM

@aivietnam.edu.vn

LSTM and GRU

(Timeseries and Text data)

[Code & Data](#)

Vinh Dinh Nguyen - PhD in Computer Science

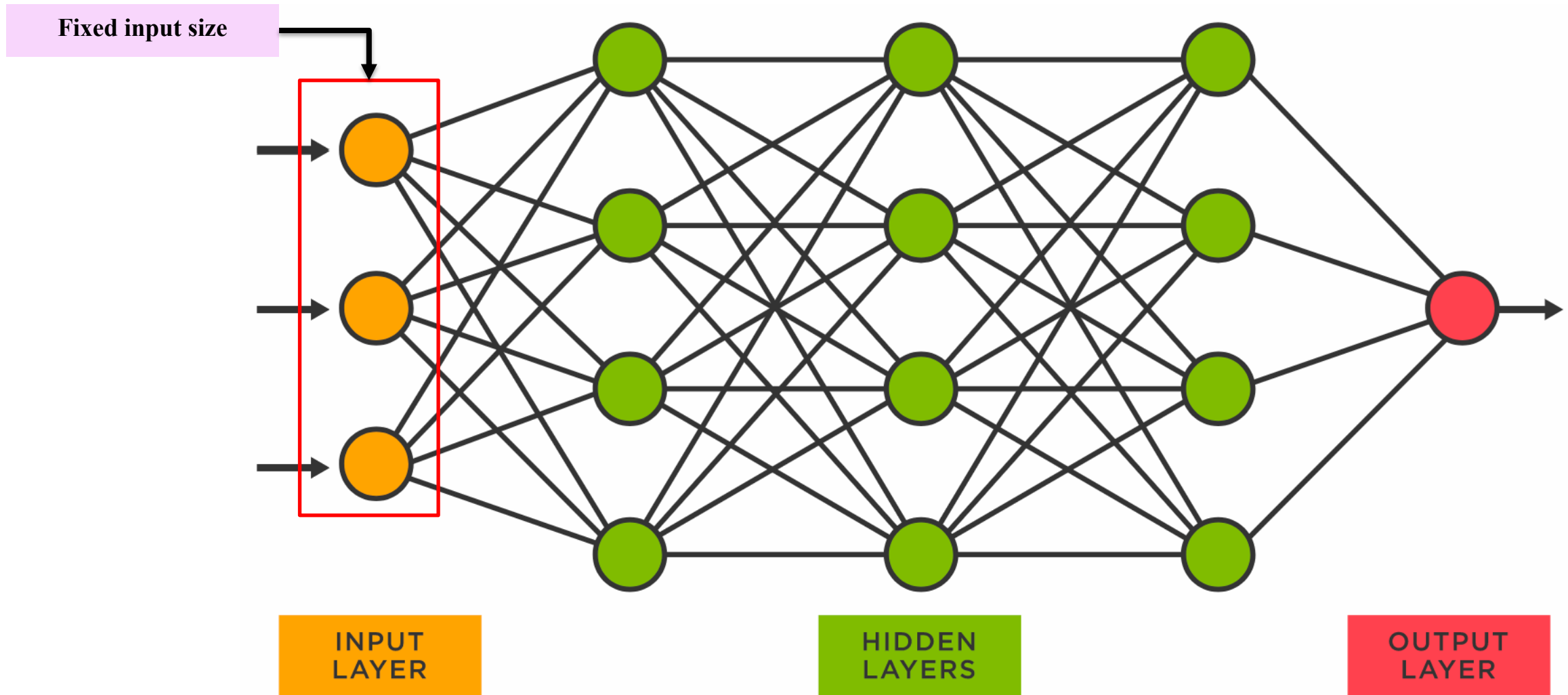
Khiem Nguyen & Thinh Nguyen - STA

Outline

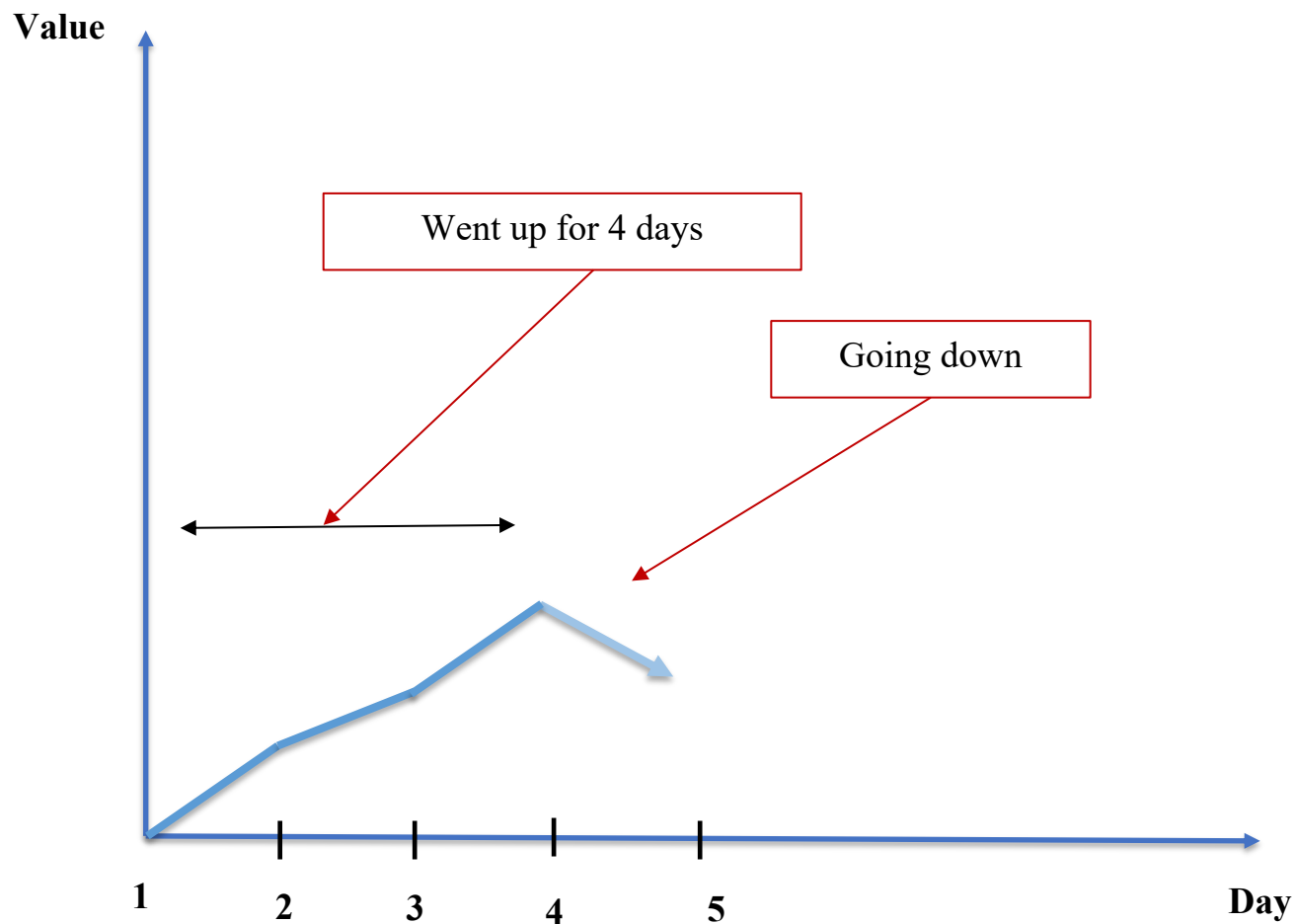


- **Review: Recurrent Neural Network (RNN)**
- **Long-Short Term Memory (LSTM)**
- **Gated Recurrent Unit (GRU)**
- **Time Series – Basic Knowledge**
- **LSTM, GRU for Time Series Data**
- **LSTM, GRU for Text Data**

Neural Network: Limitations



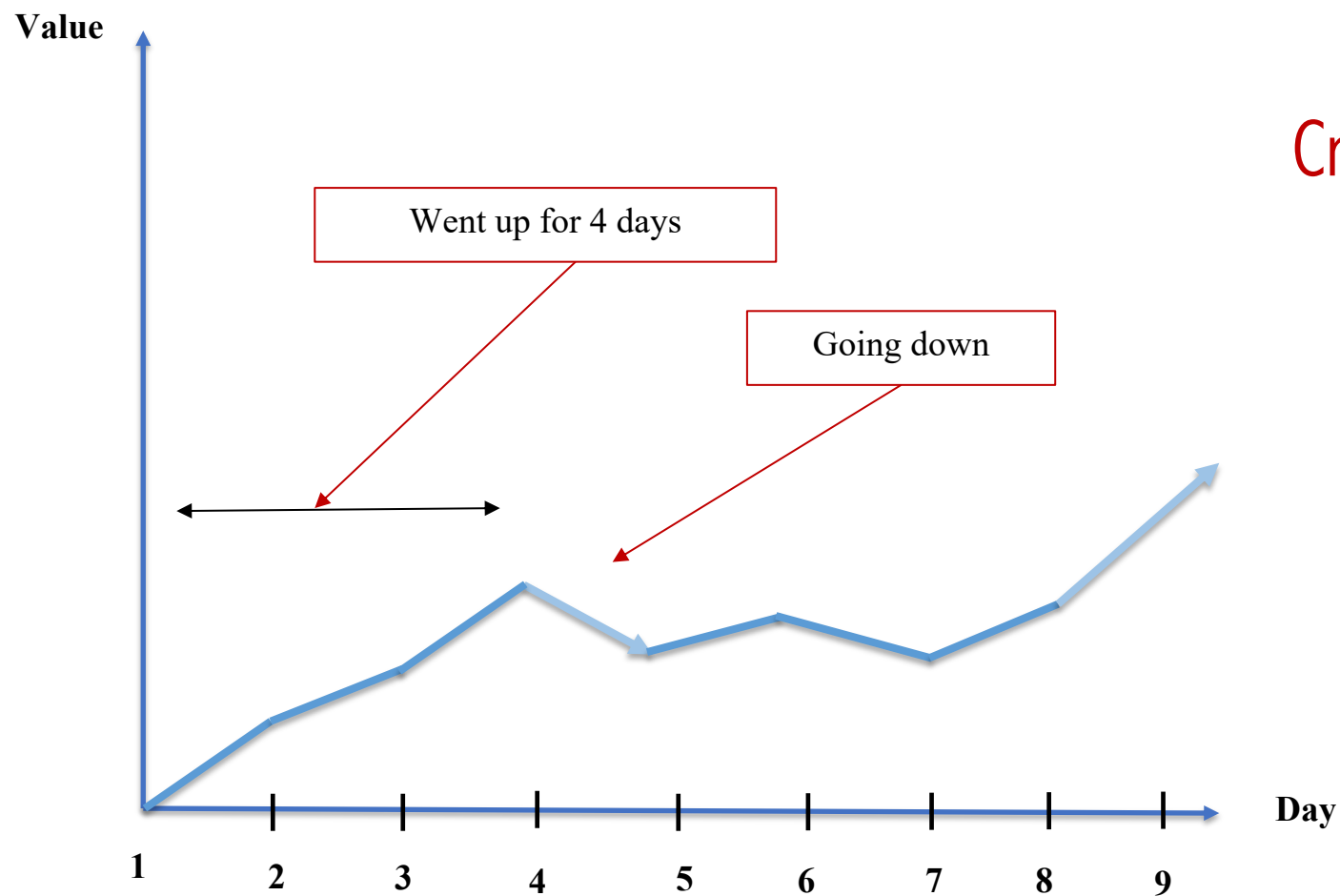
RNN: Review



Cryptocurrency Prediction

BTC-USD

RNN: Review



Cryptocurrency Prediction

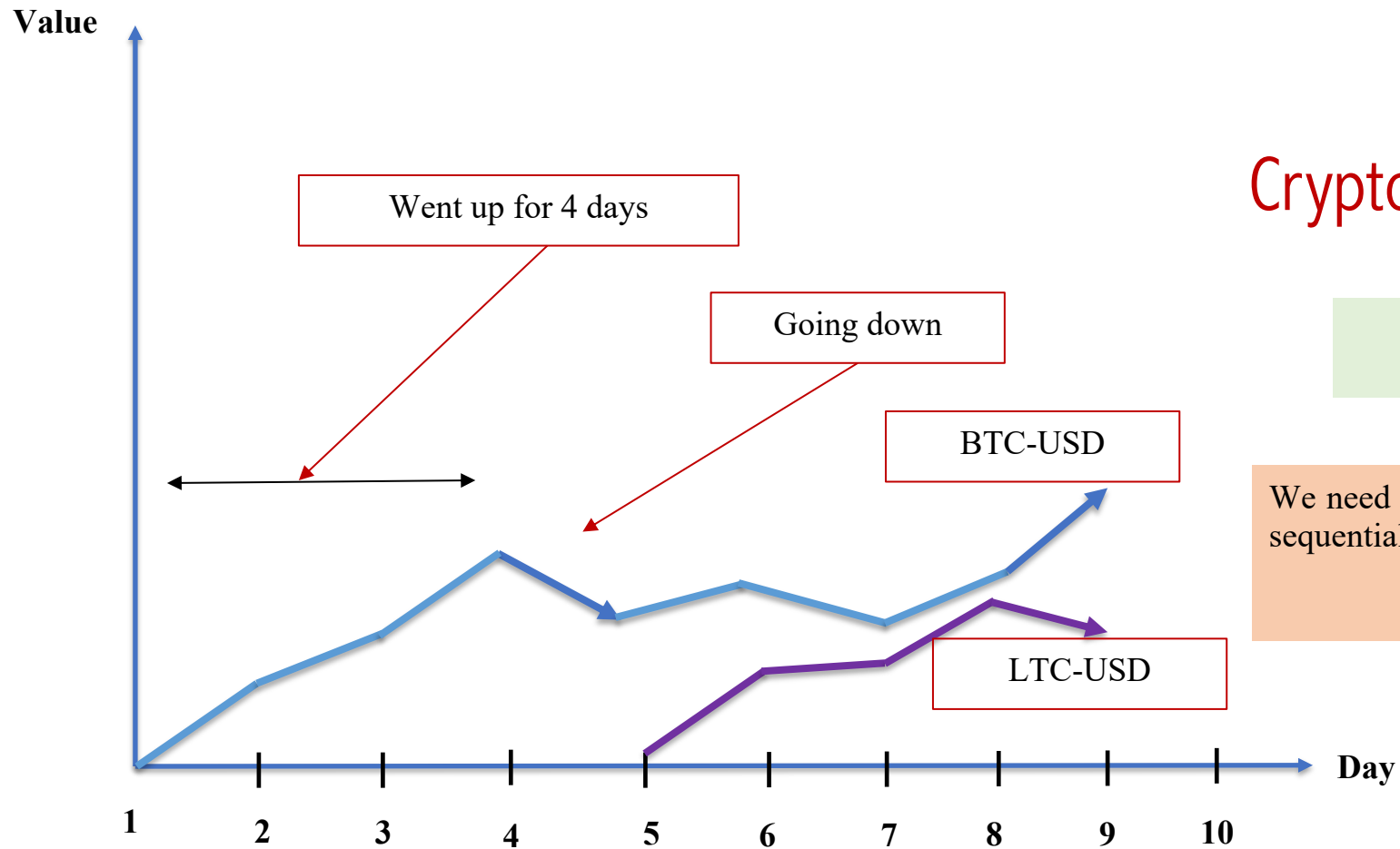
BTC-USD

RNN: Review

Cryptocurrency Prediction

BTC-USD

We need the NN to be more flexible in term of how much sequential data we use to make a prediction

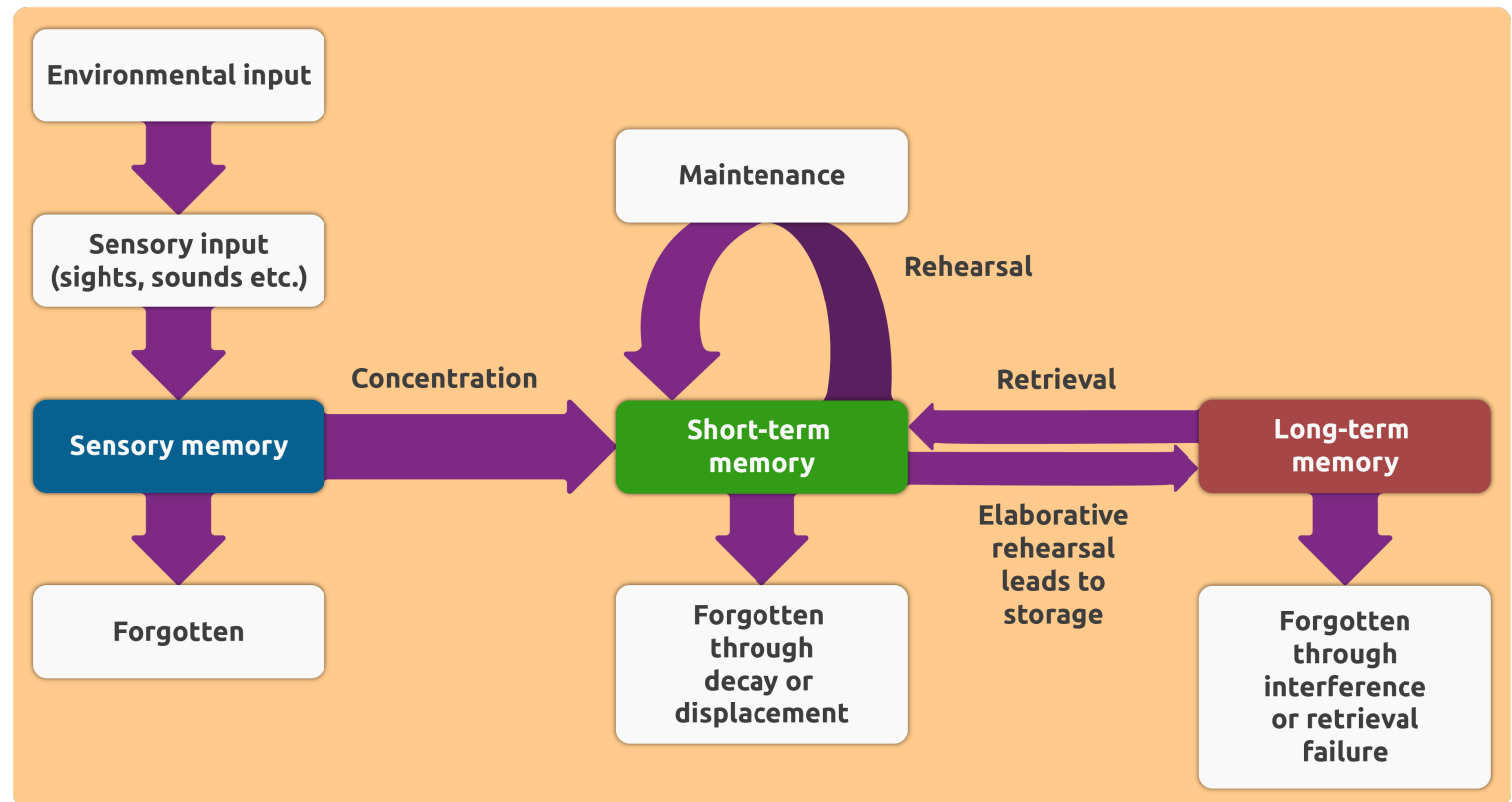


Predict LTC's price, we need data for 5 days (5,6,7,8,9)

Predict BTC's price, we need data for 9 days (1,2,3,4,5,6,7,8,9)

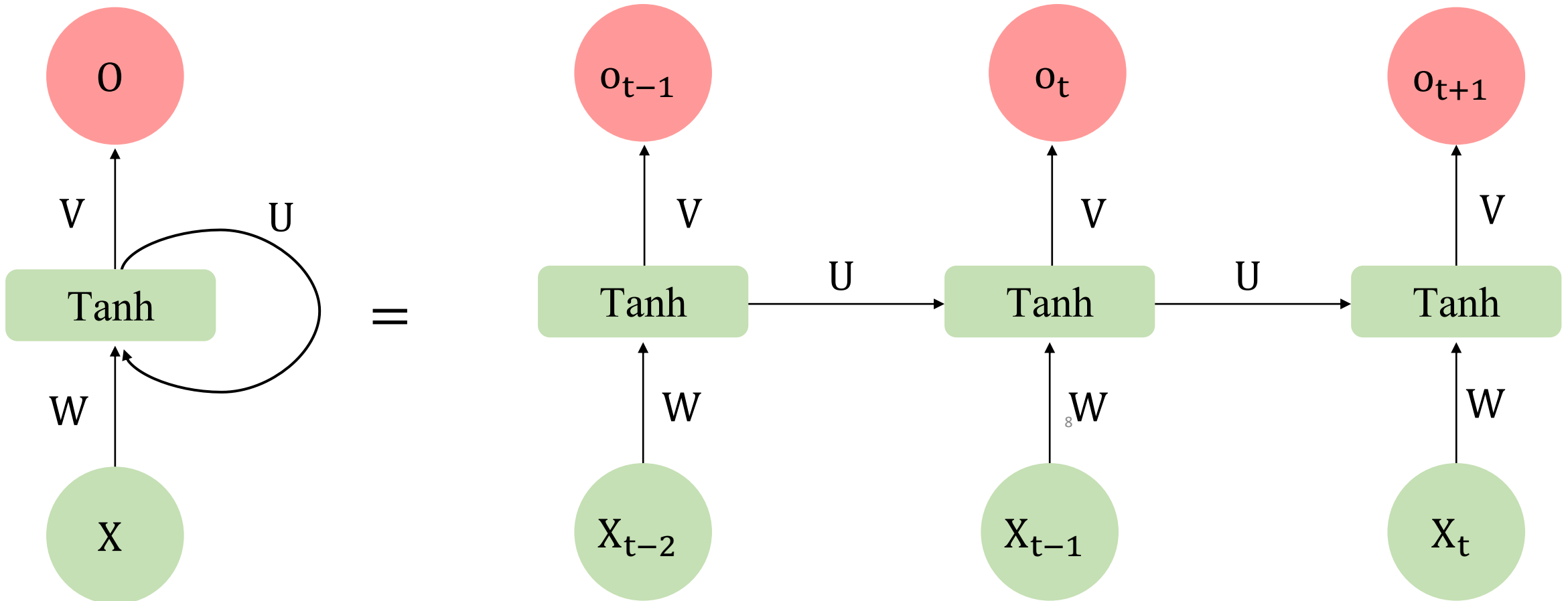
RNN: Review

Humans tend to retrieve information from **memory, short or long**, use **current information** with it and derive logic to take next action (or impulse/habit, again based on previous experiences).



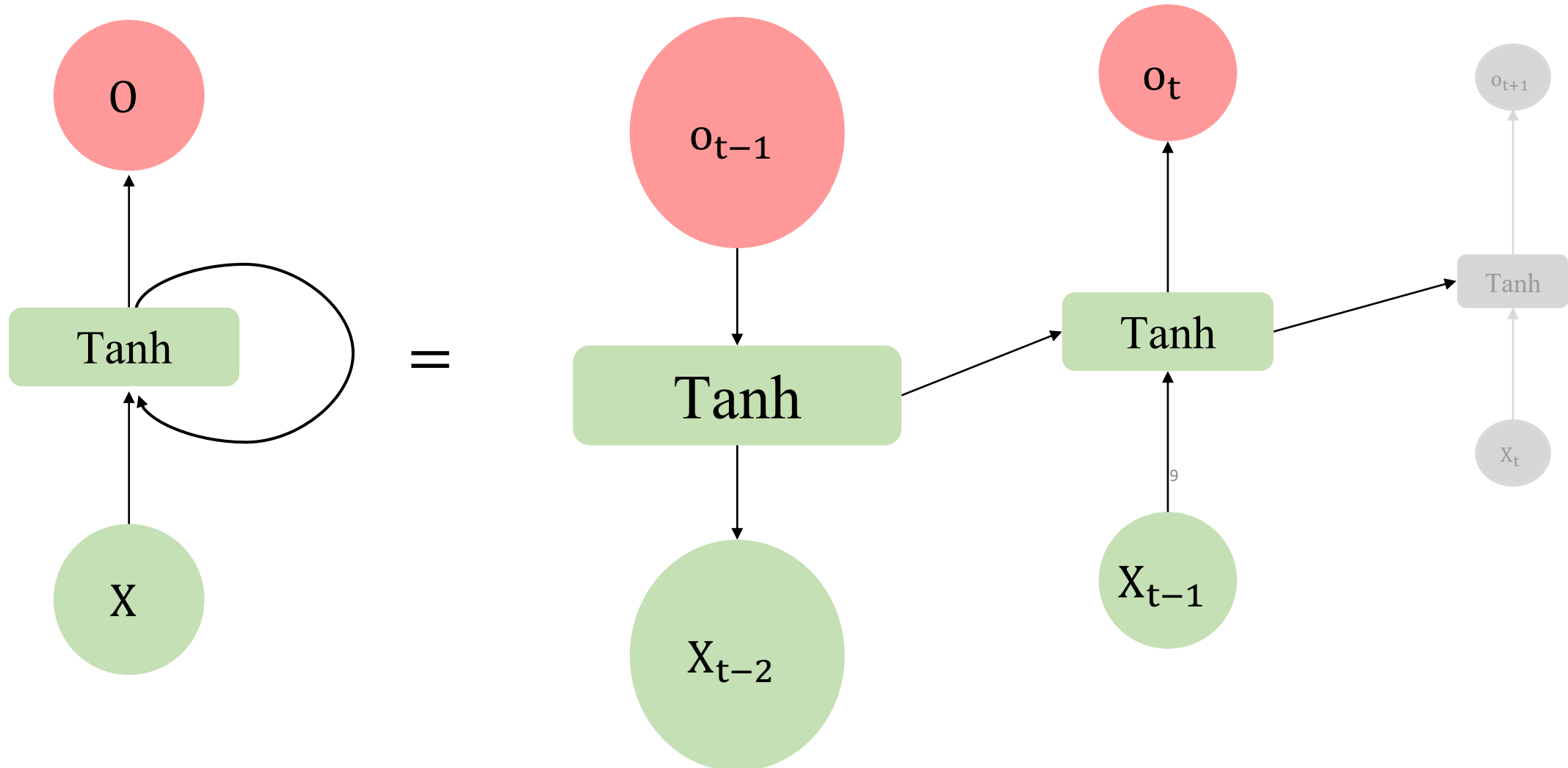
RNN: Review

Recurrent Neural Networks (RNNs)



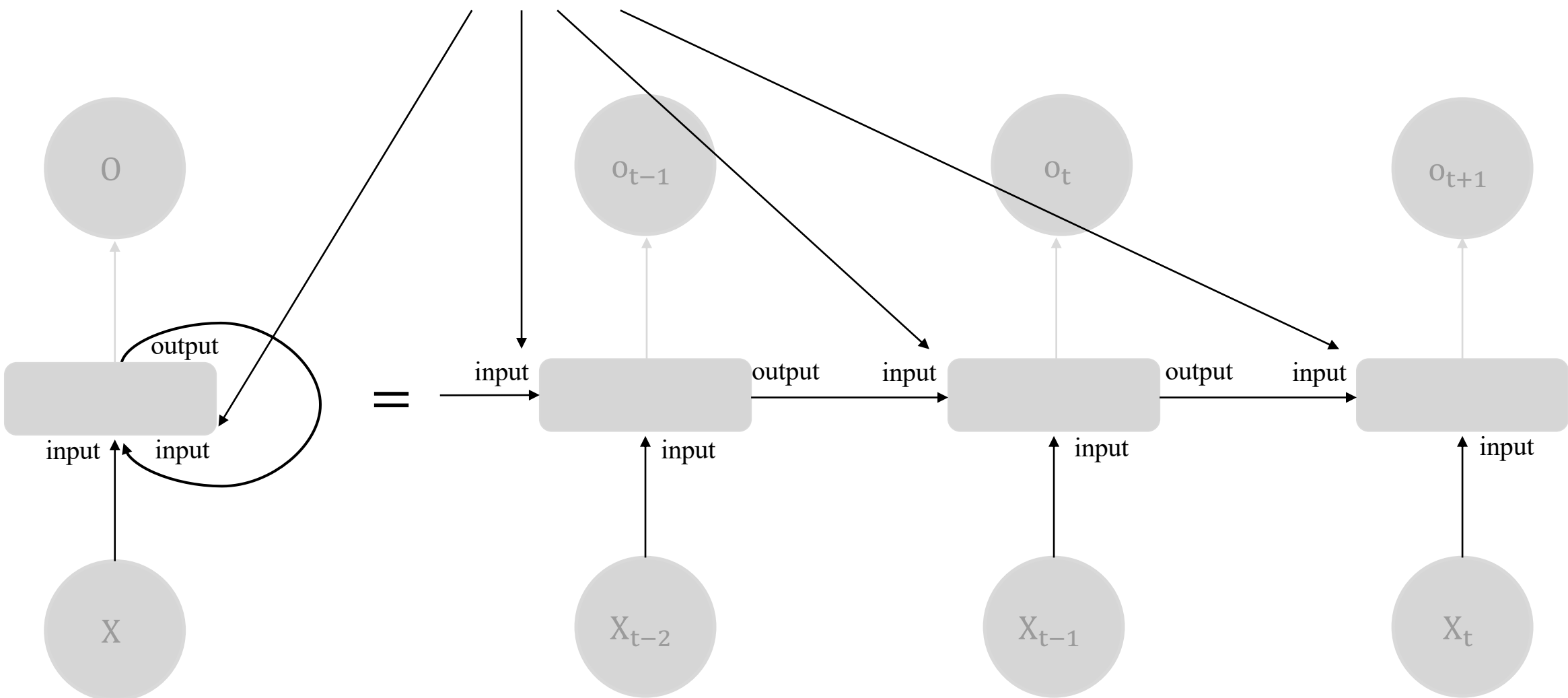
RNN: Review

Recurrent means Sequence to sequence



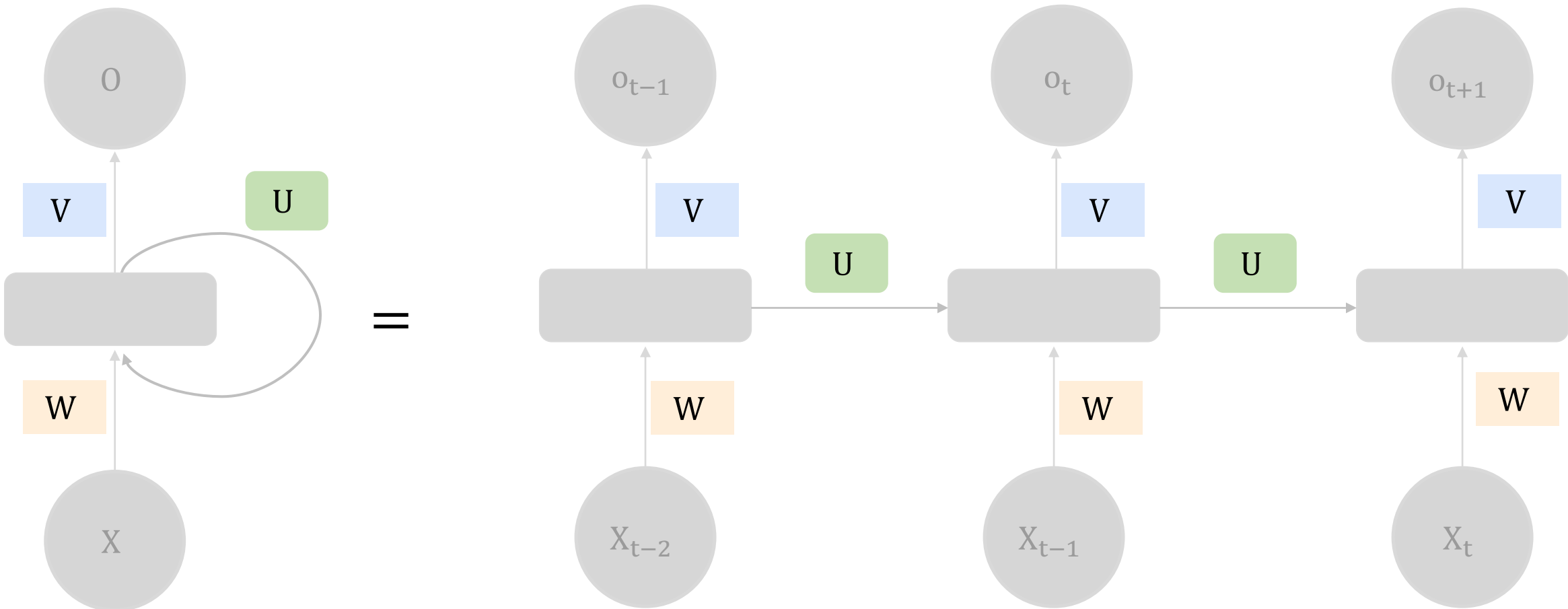
RNN: Review

Recurrent means Feedback loops

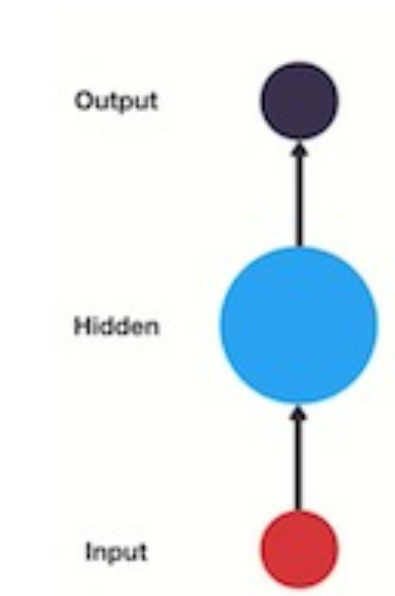


RNN: Review

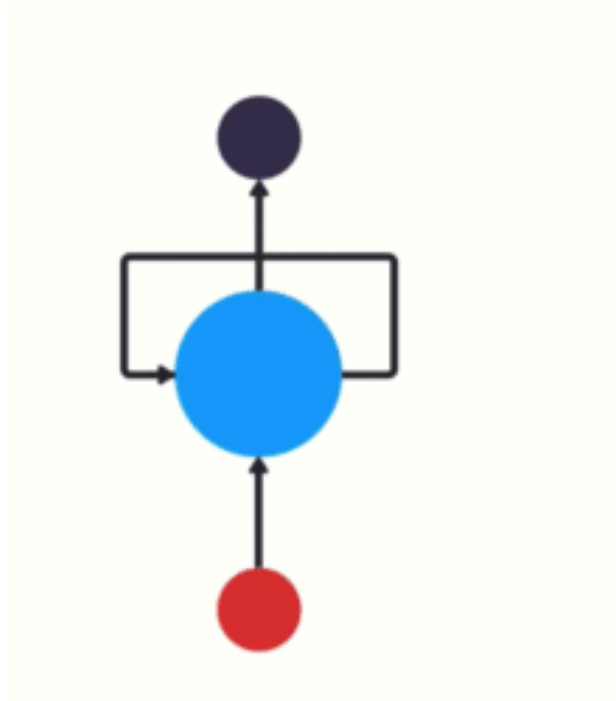
Recurrent means Sharing Weights



RNN: Review



NN

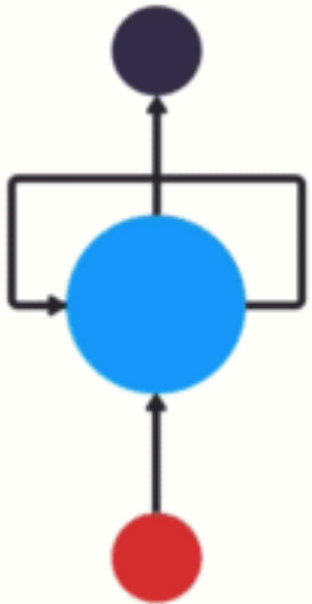


RNN



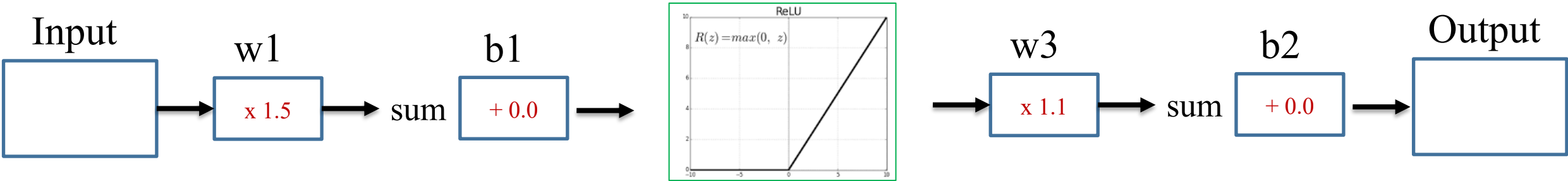
RNN

RNN: Review

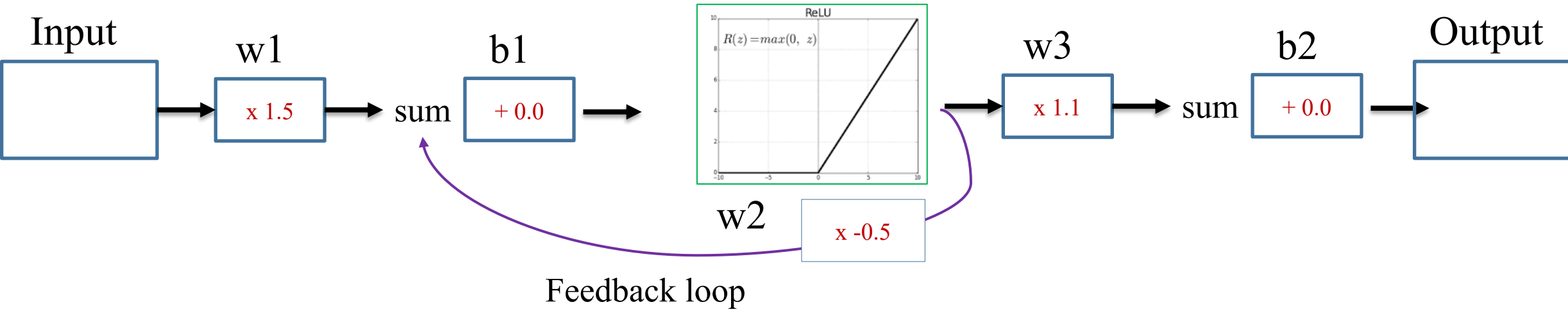


RNN: Review

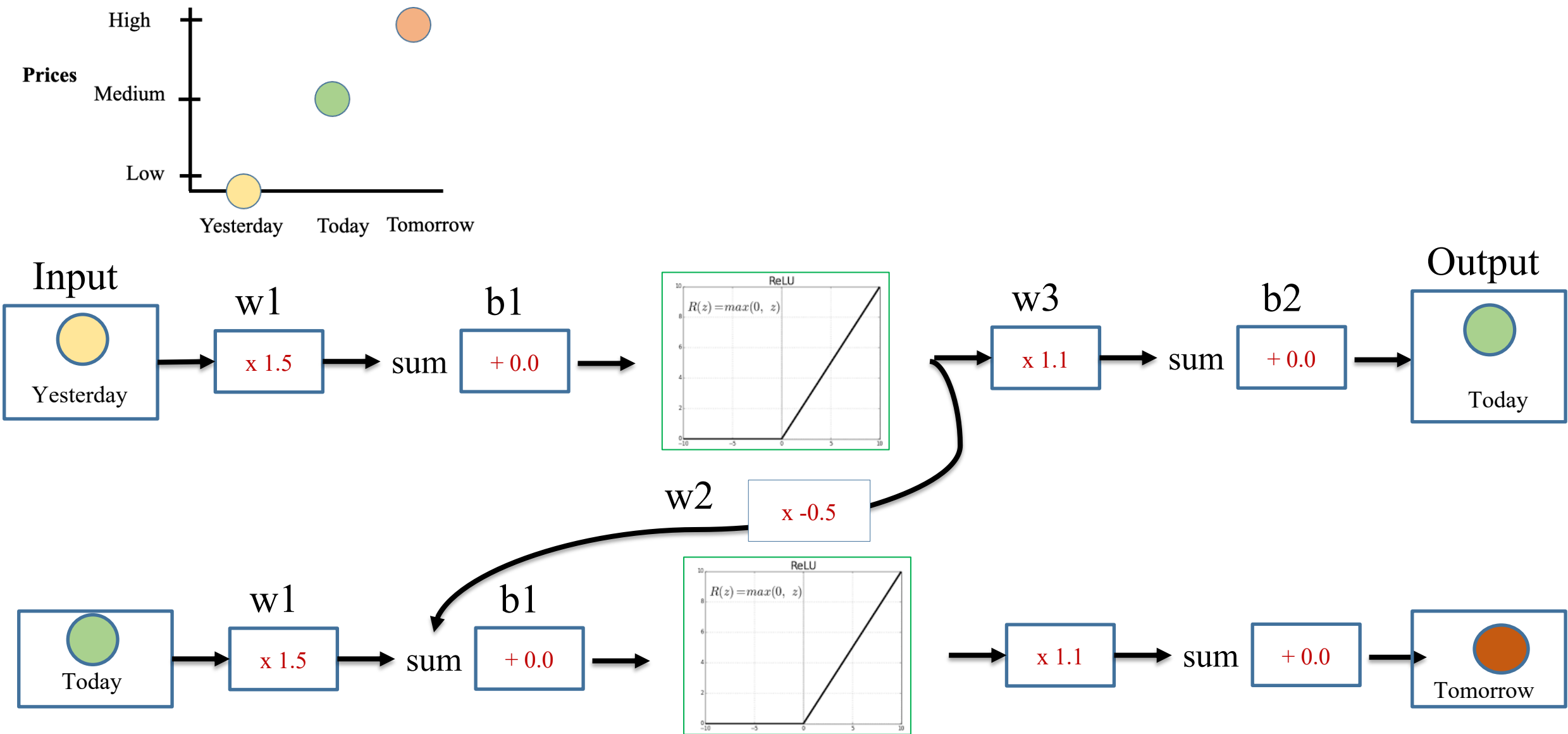
Traditional Neural Network



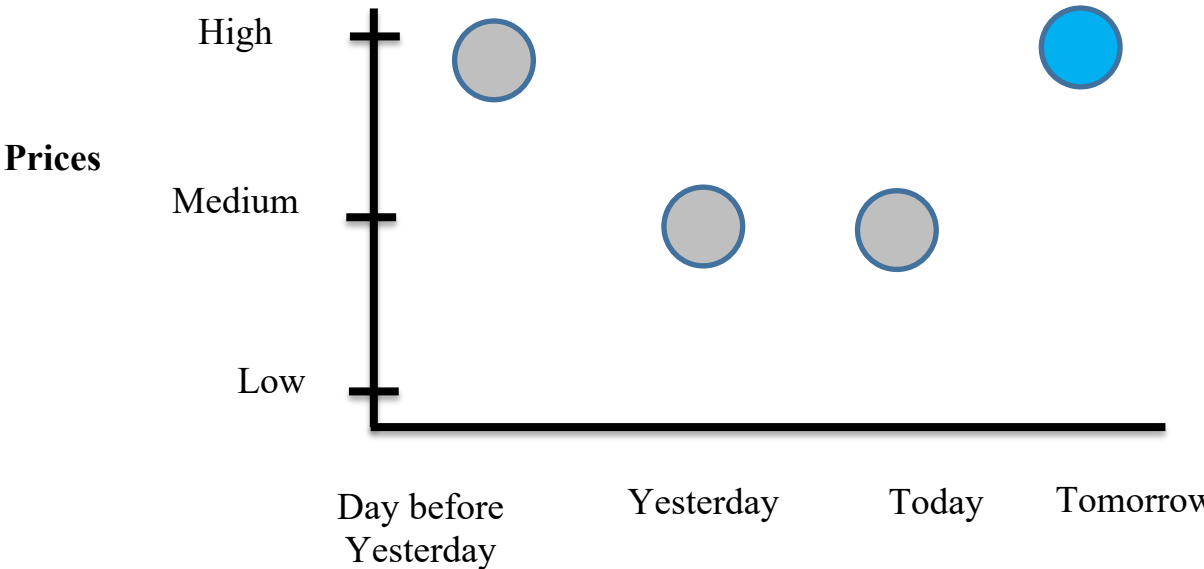
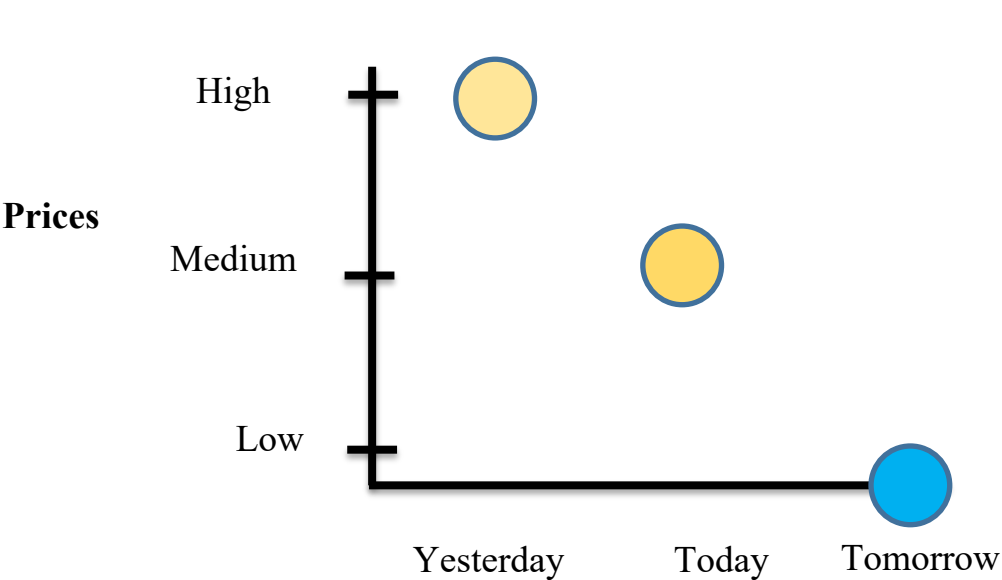
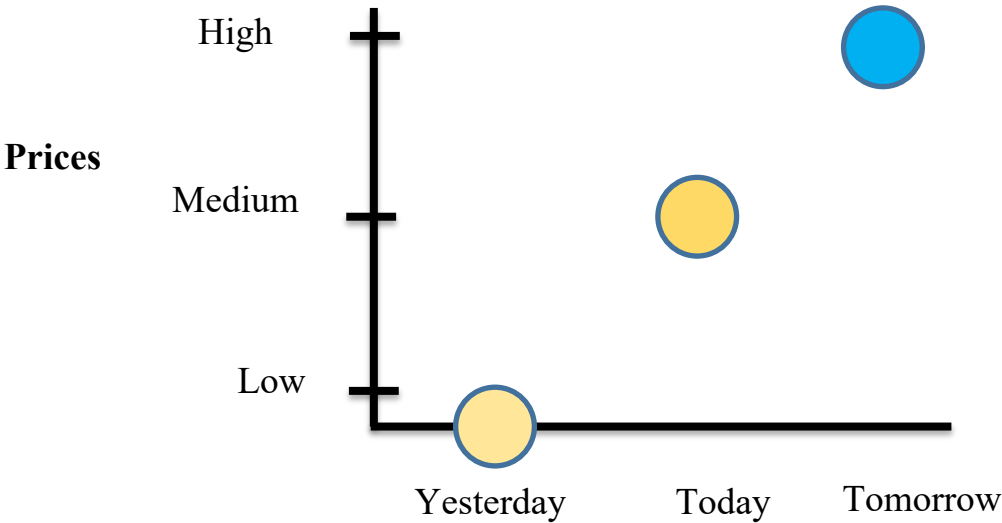
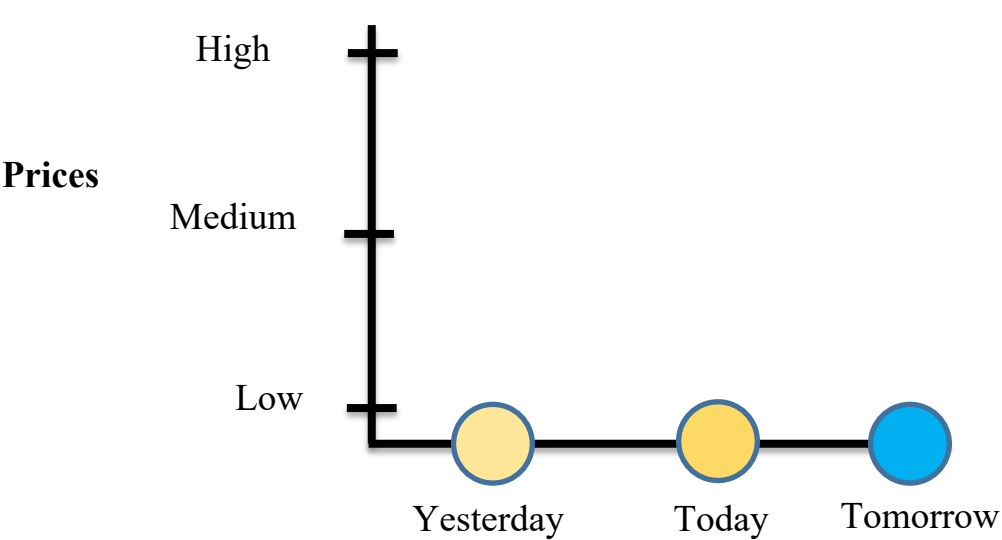
Recurrent Neural Network for sequential input values



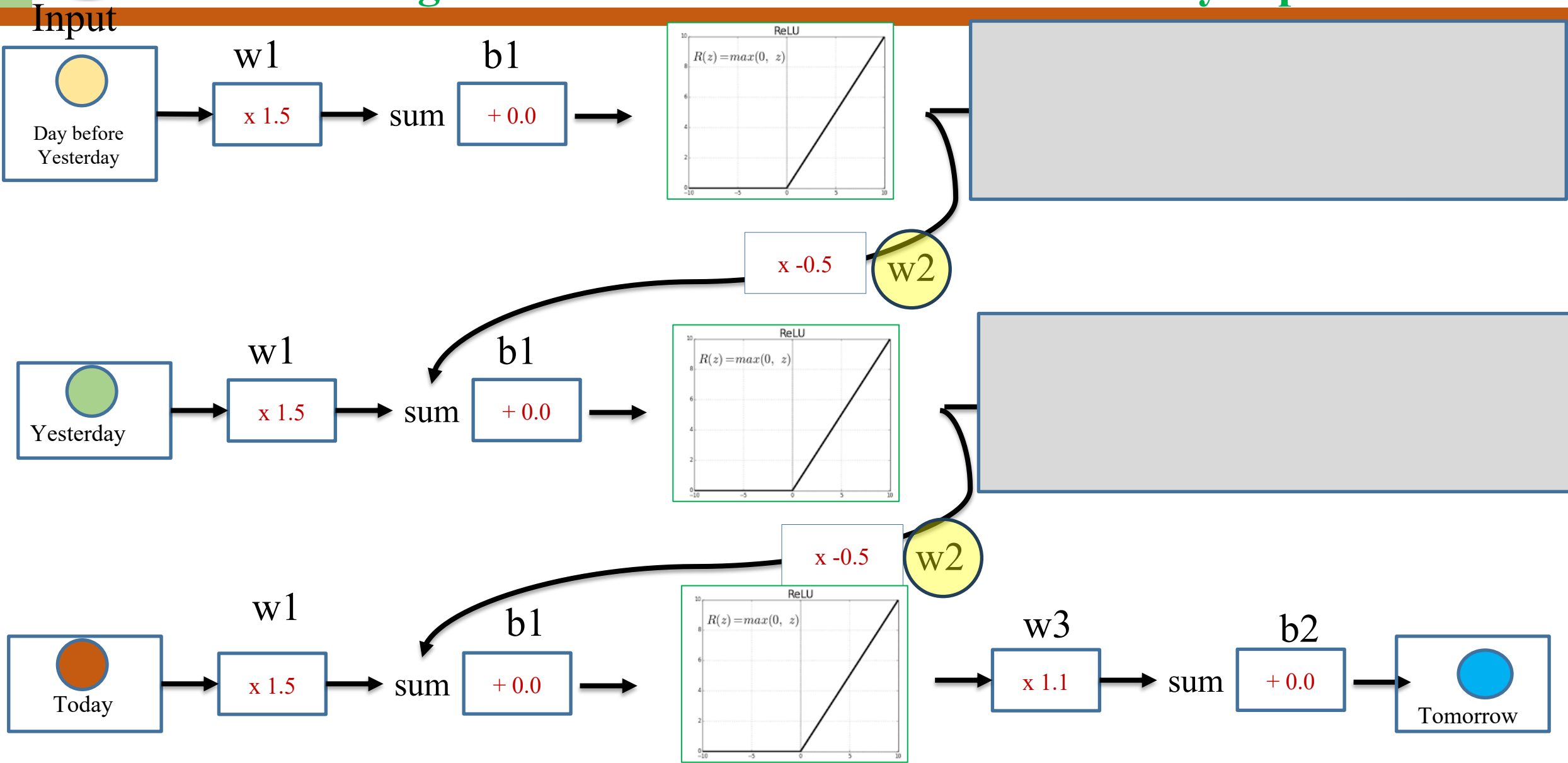
RNN: Review



RNN: Review



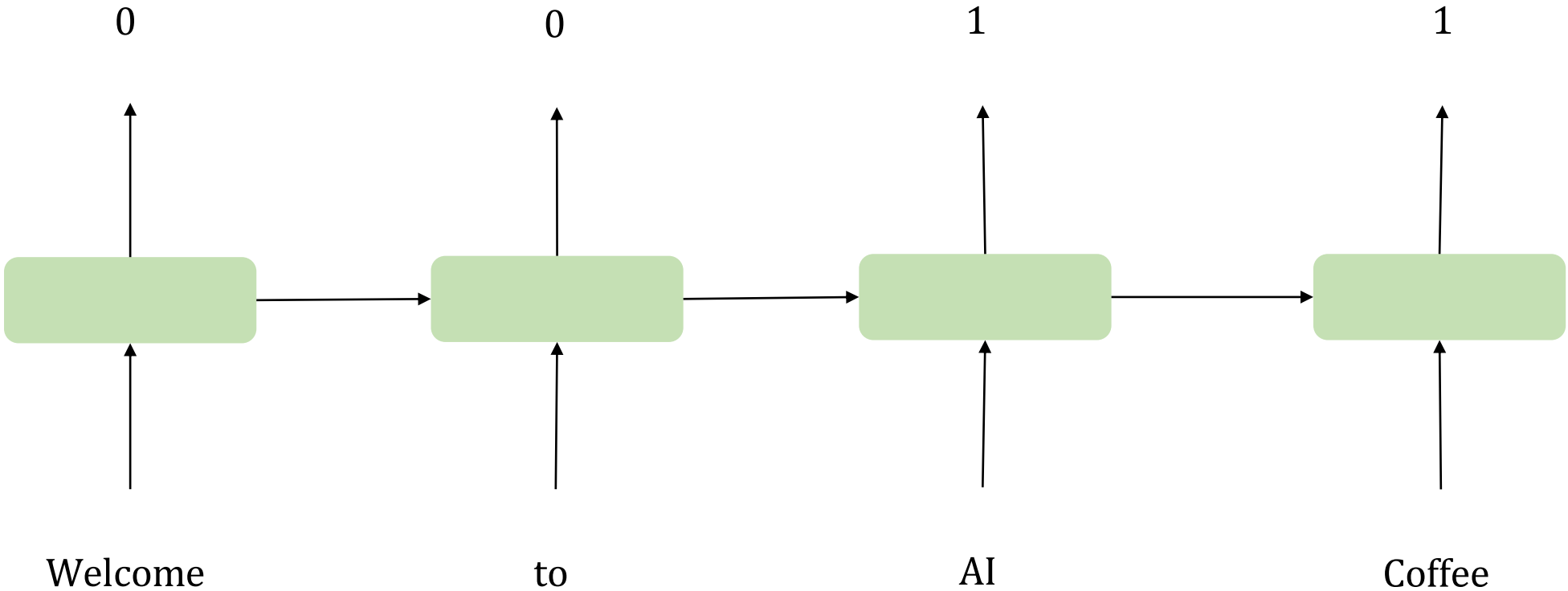
Weight and Biases Are Shared Across Every Input



No matter how many times we unroll a recurrent neural network, we never increase number of weights and bias to train

Different types of RNNs

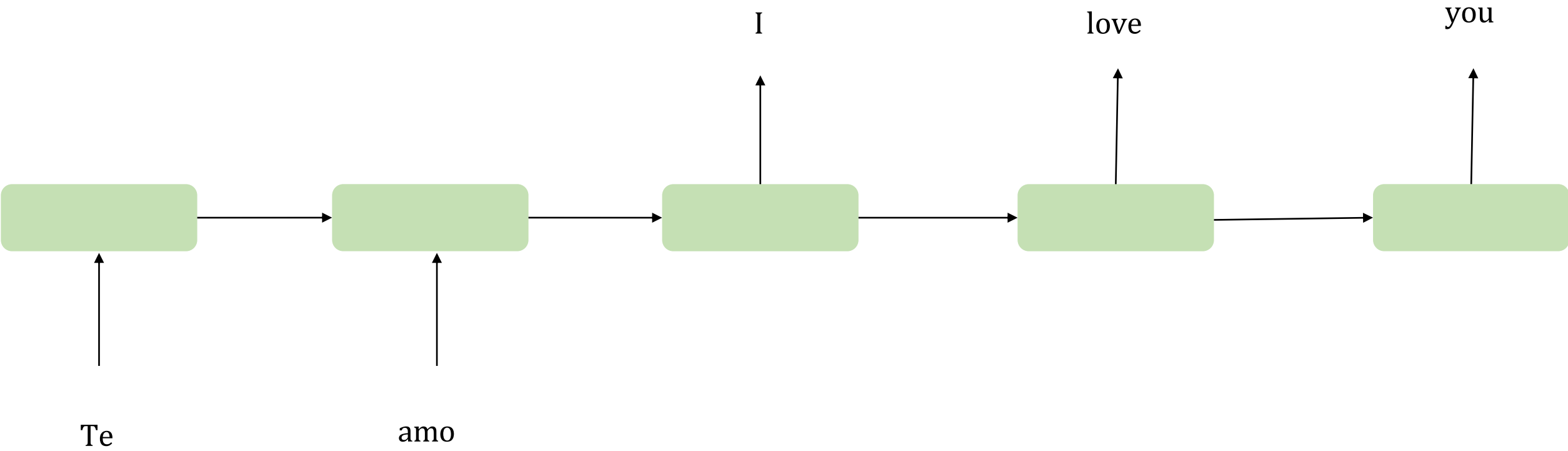
Many to many
(equal input-output)



Named entity recognition, Part of speech recognition,...

Different types of RNNs

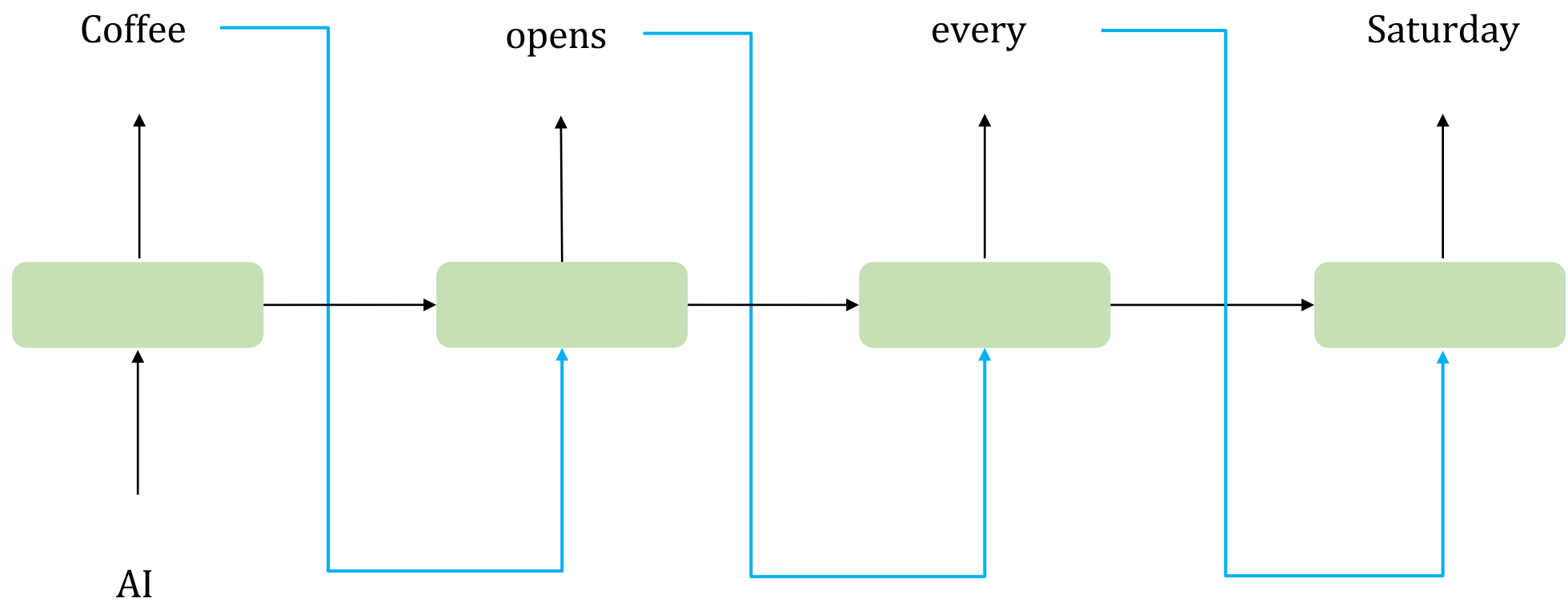
Many to many
(unequal input-output)



Machine translation, Speech recognition,...

Different types of RNNs

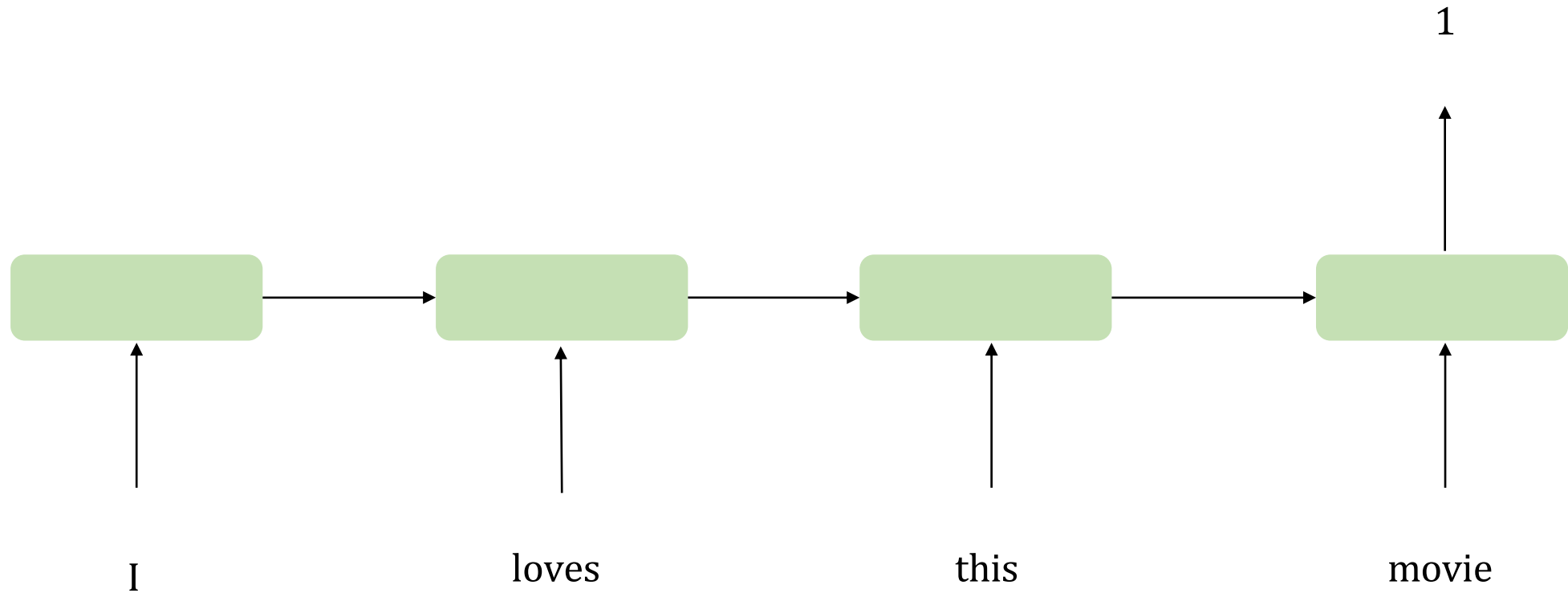
One to many



Novel sequence generation, Image captioning,...

Different types of RNNs

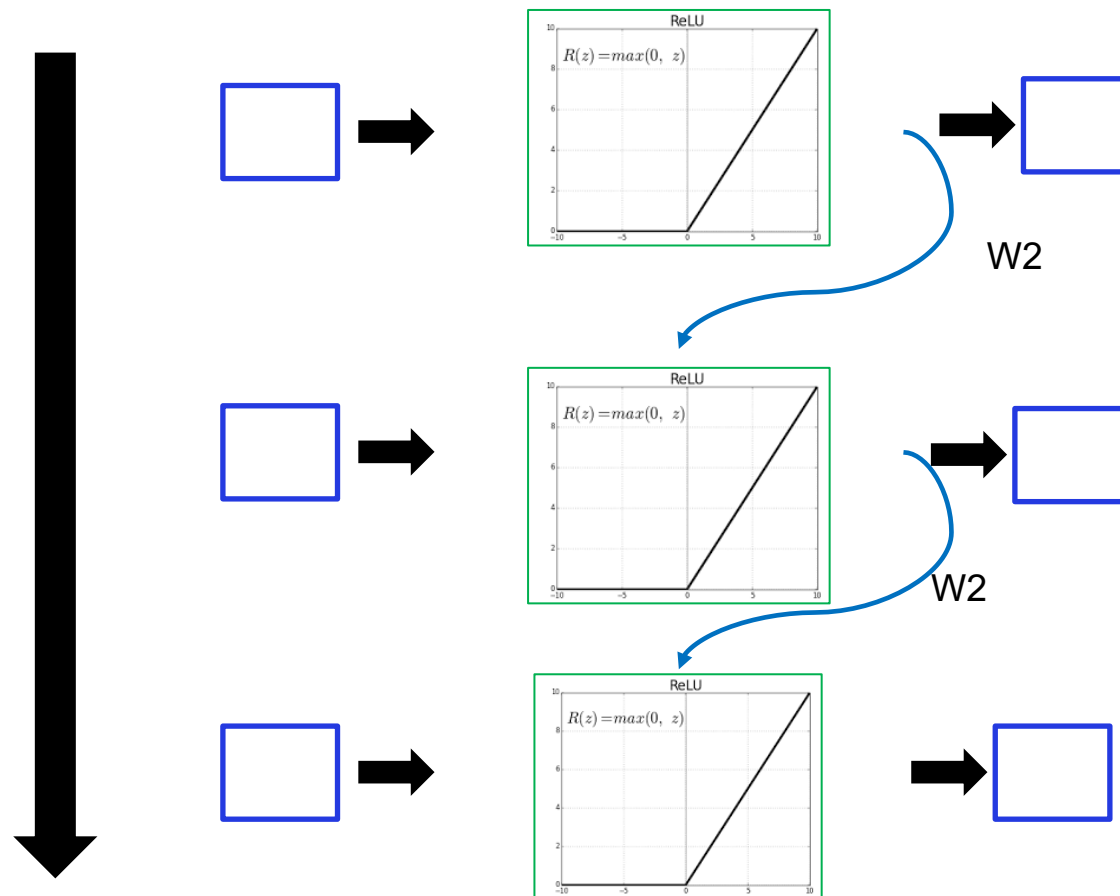
Many to one



Sentiment classification, Movie rating prediction

Why RNN is not Use Often

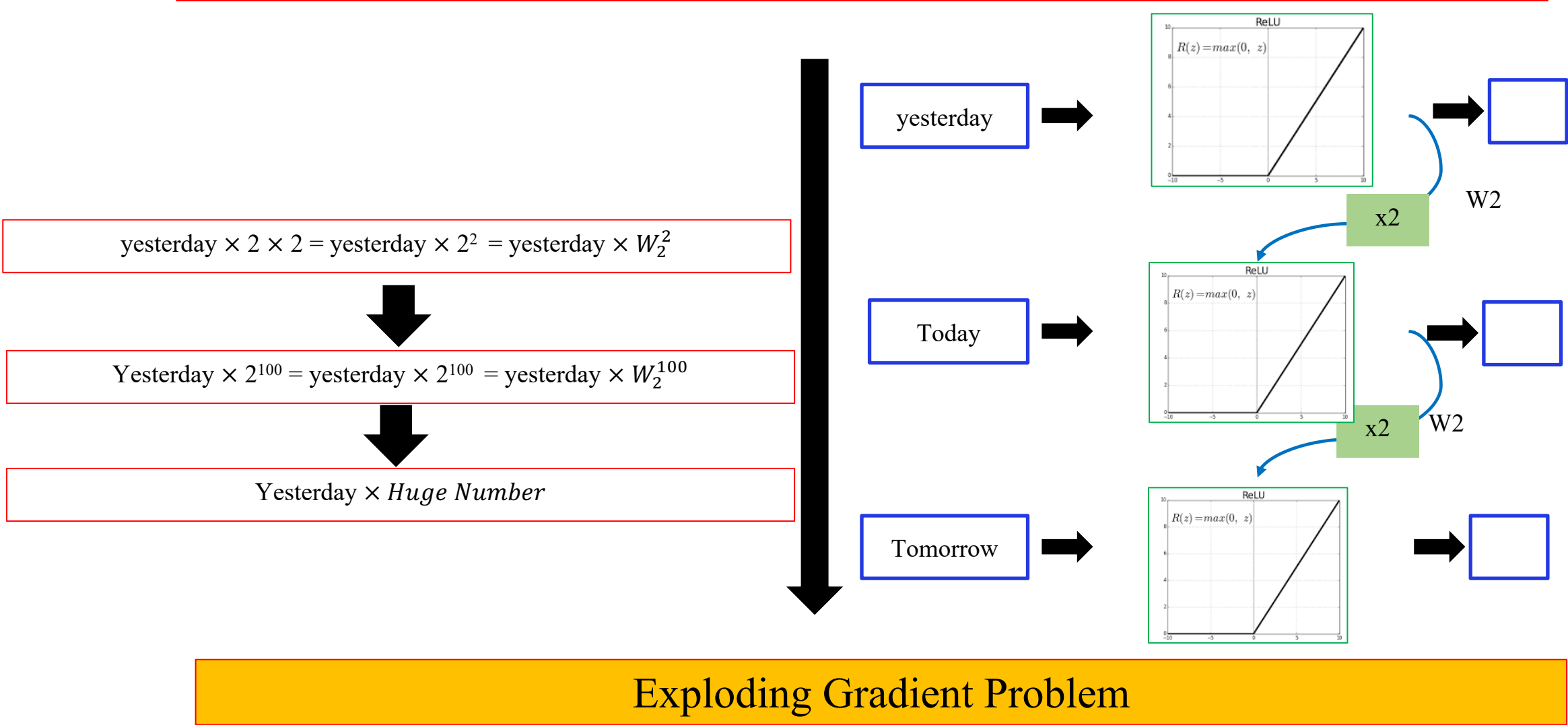
The more we unroll the recurrent neural network, the harder it is to train



Vanishing / Exploding Gradient Problem

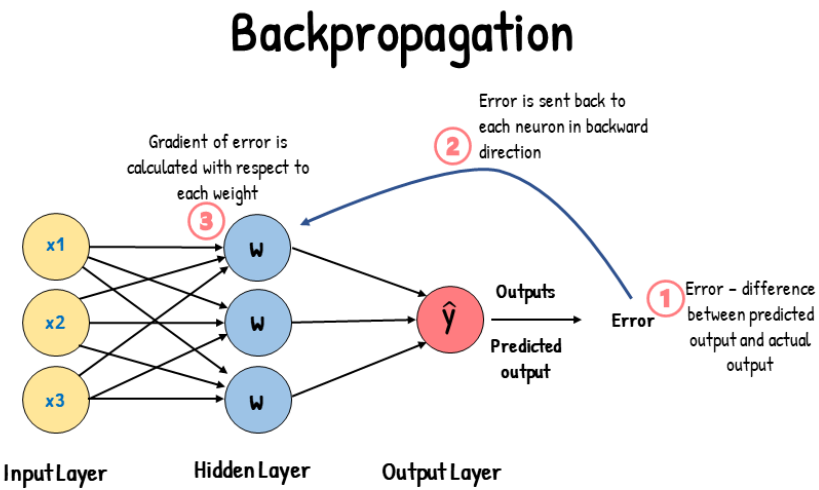
Why RNN is not use Often

If we set $W_2 \in [1, +\infty]$, Let's start with $W_2 = 2$



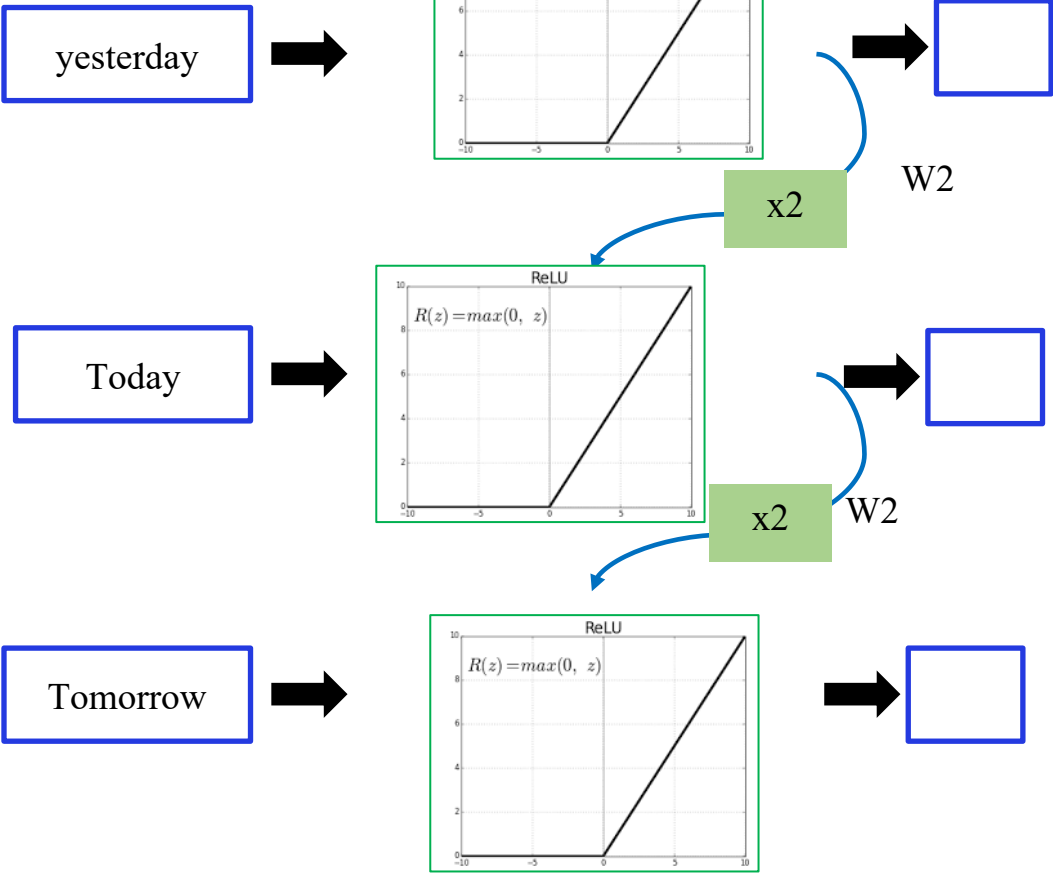
Why RNN is not use Often

If we set $W2 \in [1, +\infty]$, Let's start with $W2 = 2$



W is a huge numbers

Overshoot the optimal solution



Exploding Gradient Problem

Why RNN is not use Often

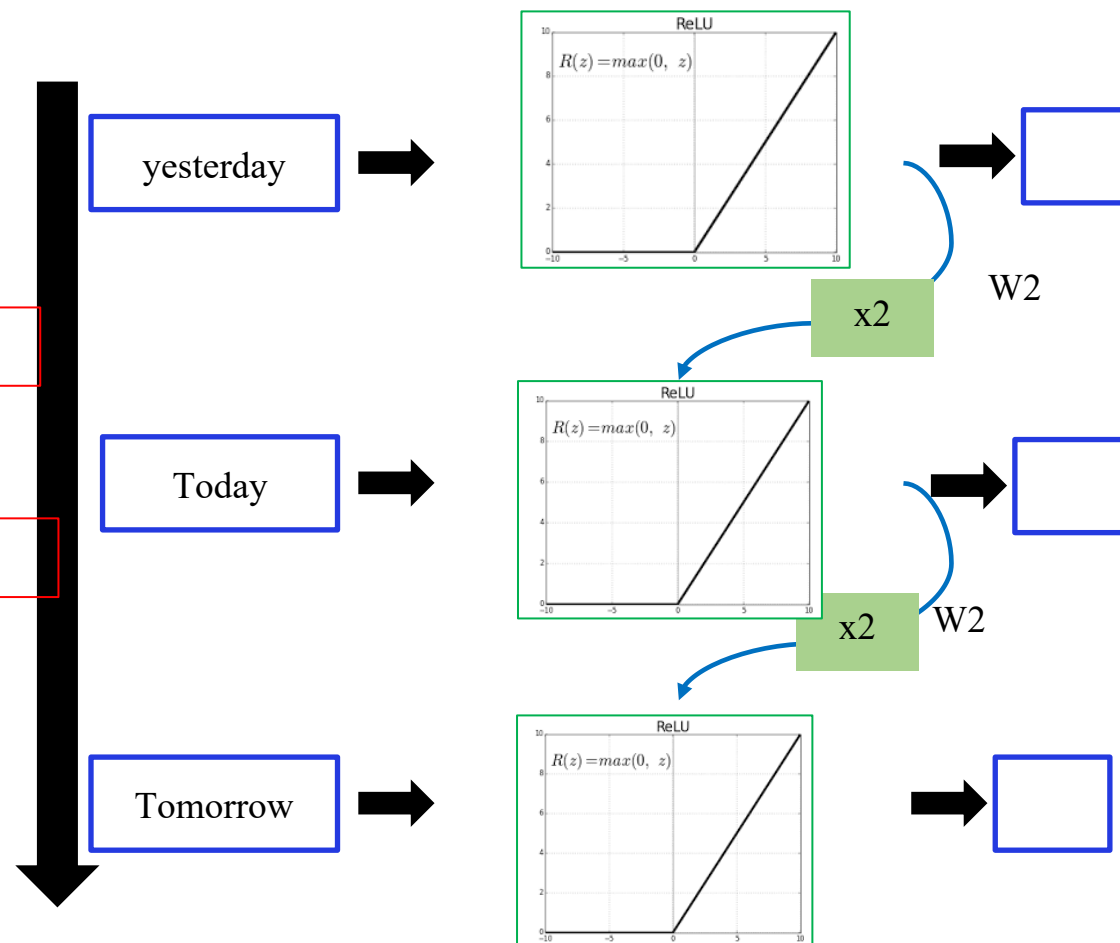
If we set $W_2 \in [0,1]$, Let's start with $W_2 = 0.5$



$$\text{yesterday} \times 0.5 \times 0.5 = \text{yesterday} \times 0.5^2 = \text{yesterday} \times W_2^2$$

$$\text{Yesterday} \times 0.5^{100} = \text{yesterday} \times 0.5^{100} = \text{yesterday} \times W_2^{100}$$

Yesterday $\times$ Really Small Number



Vanishing Gradient Problem

How to avoid Exploding and Vanishing Gradient Problem in RNN

I grew up in **France**... I speak fluent *French*

RNN fail to handel

Outline



➤ **Review: Recurrent Neural Network (RNN)**

➤ **Long-Short Term Memory (LSTM)**

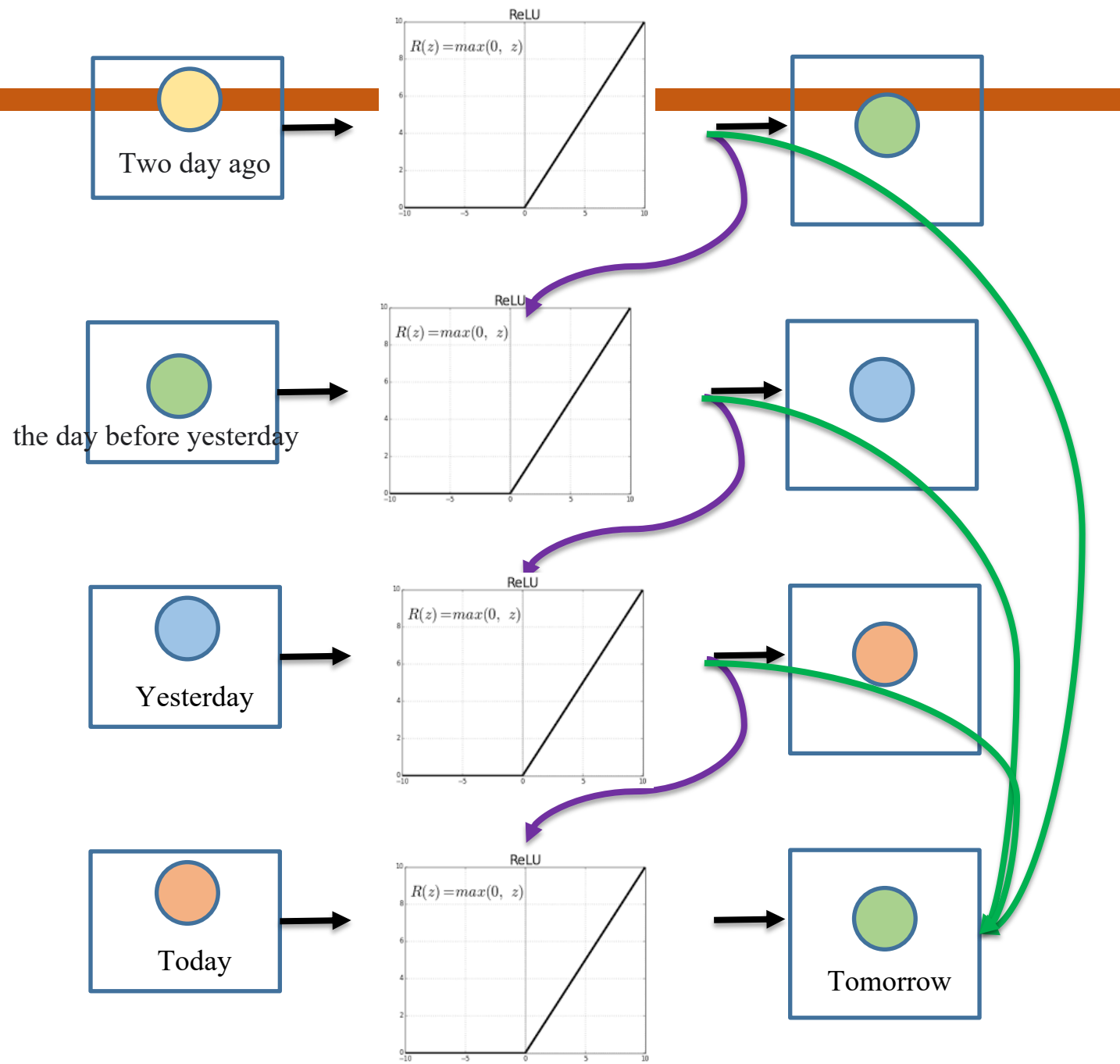
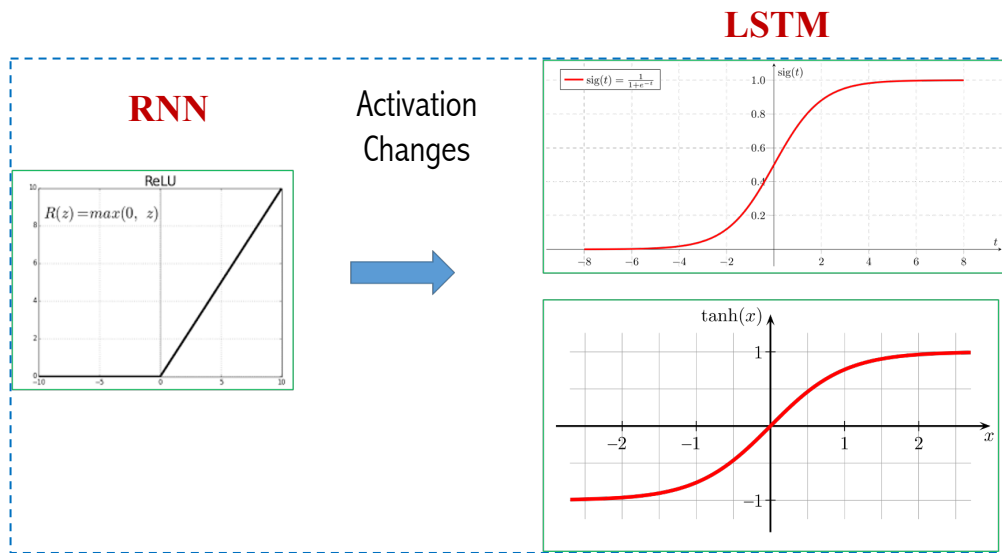
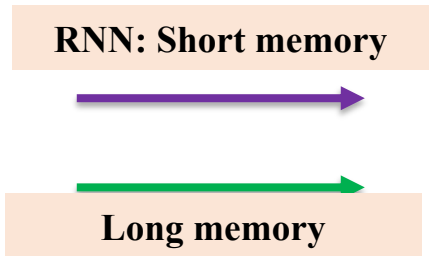
➤ **Gated Recurrent Unit (GRU)**

➤ **Time Series – Basic Knowledge**

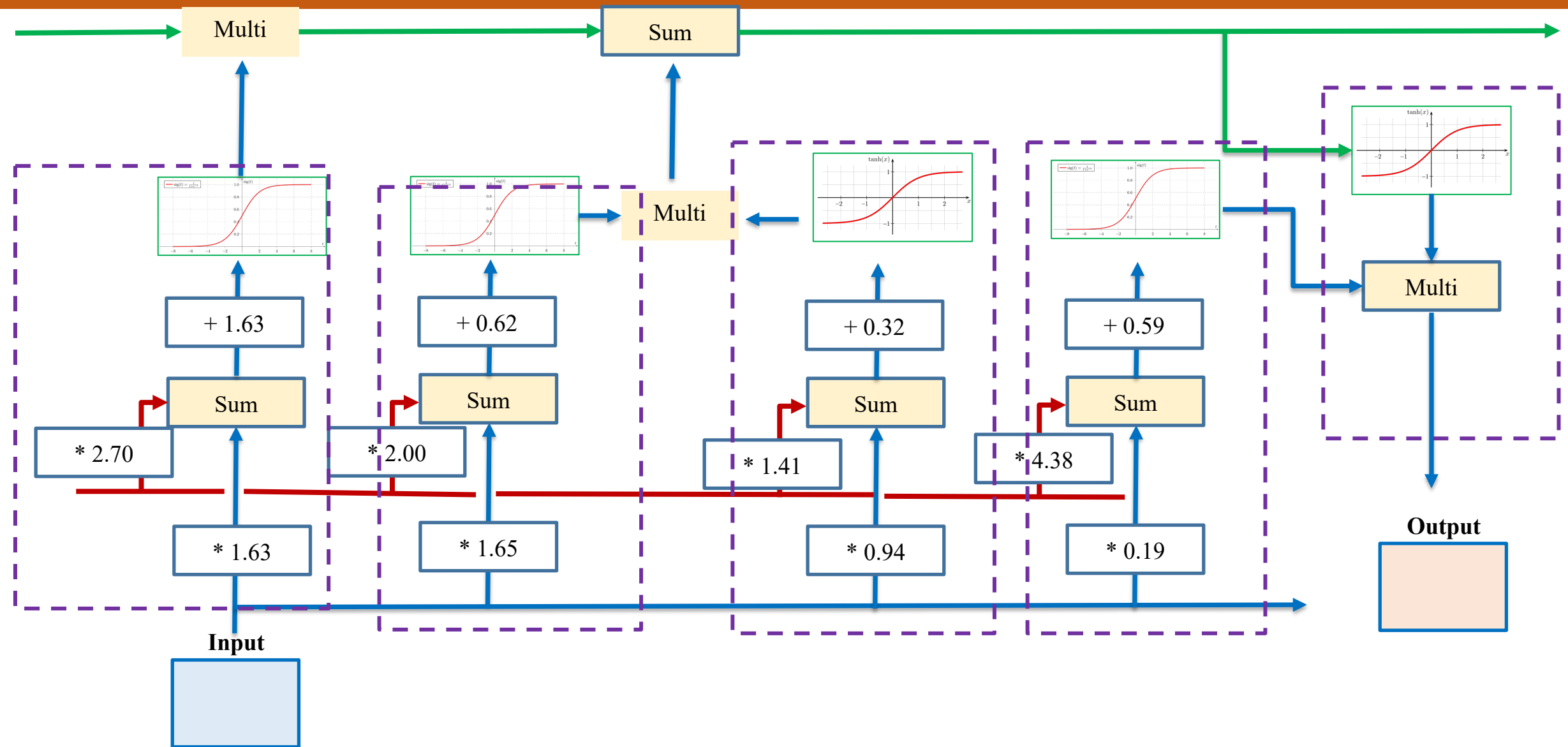
➤ **LSTM, GRU for Time Series Data**

➤ **LSTM, GRU for Text Data**

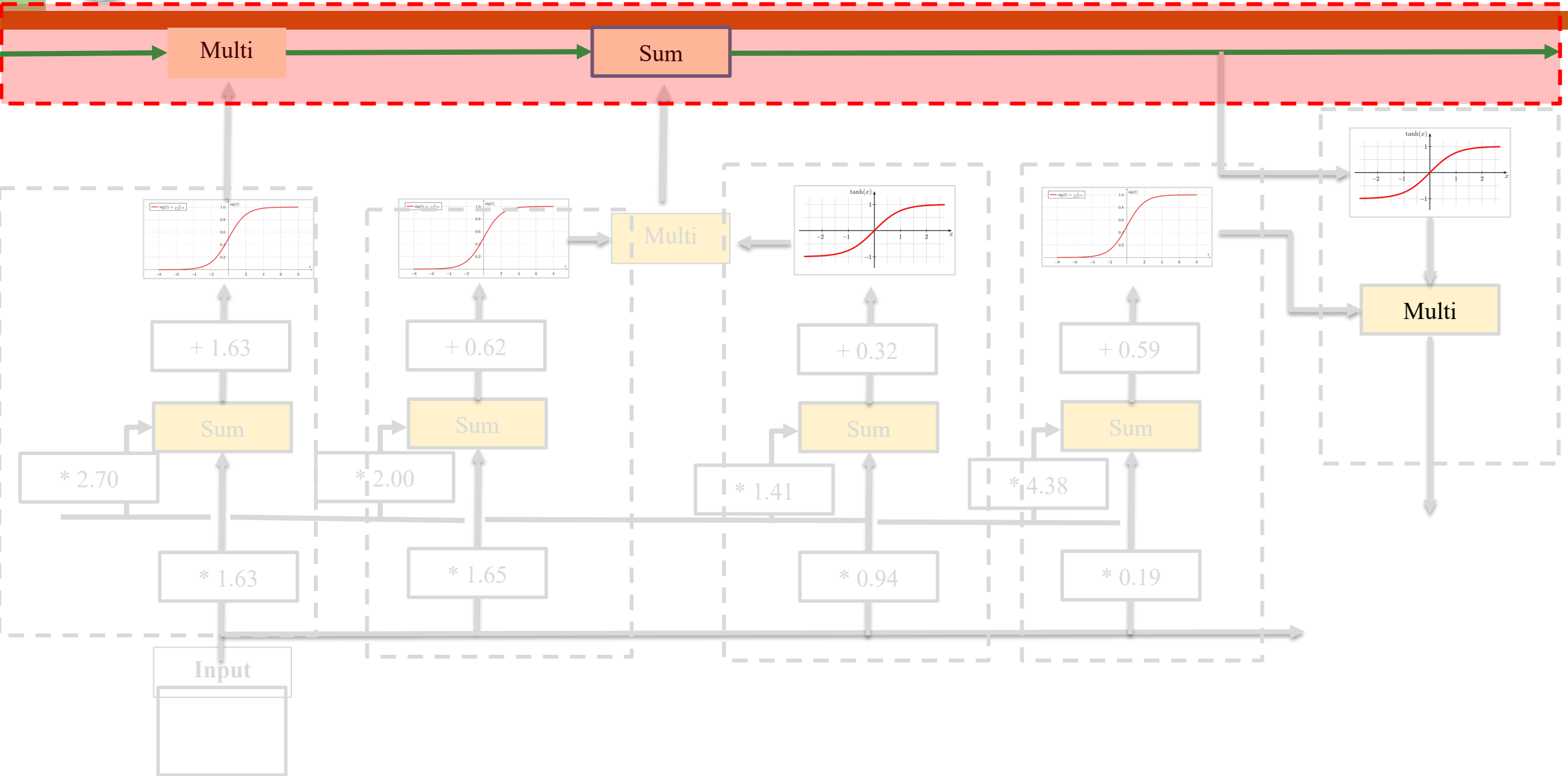
LSTM

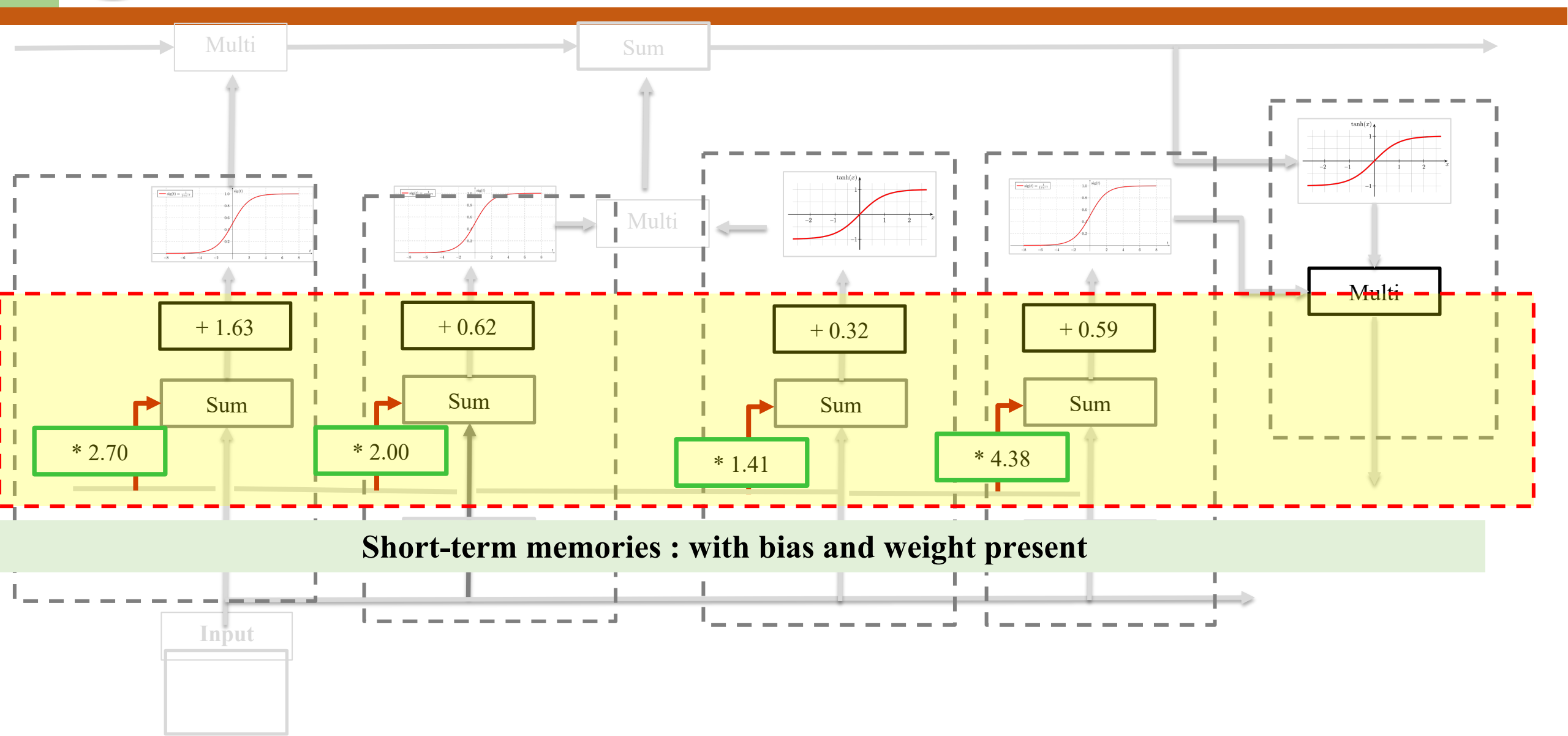


Three Stages in a Single LSTM Unit



Long-term memories : no bias and weight present => avoid causing exploding and vanishing gradient

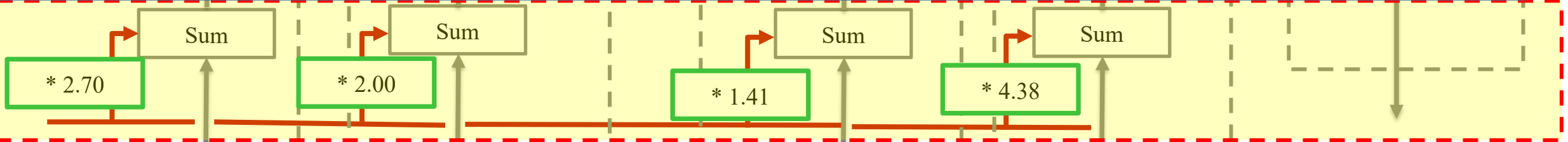




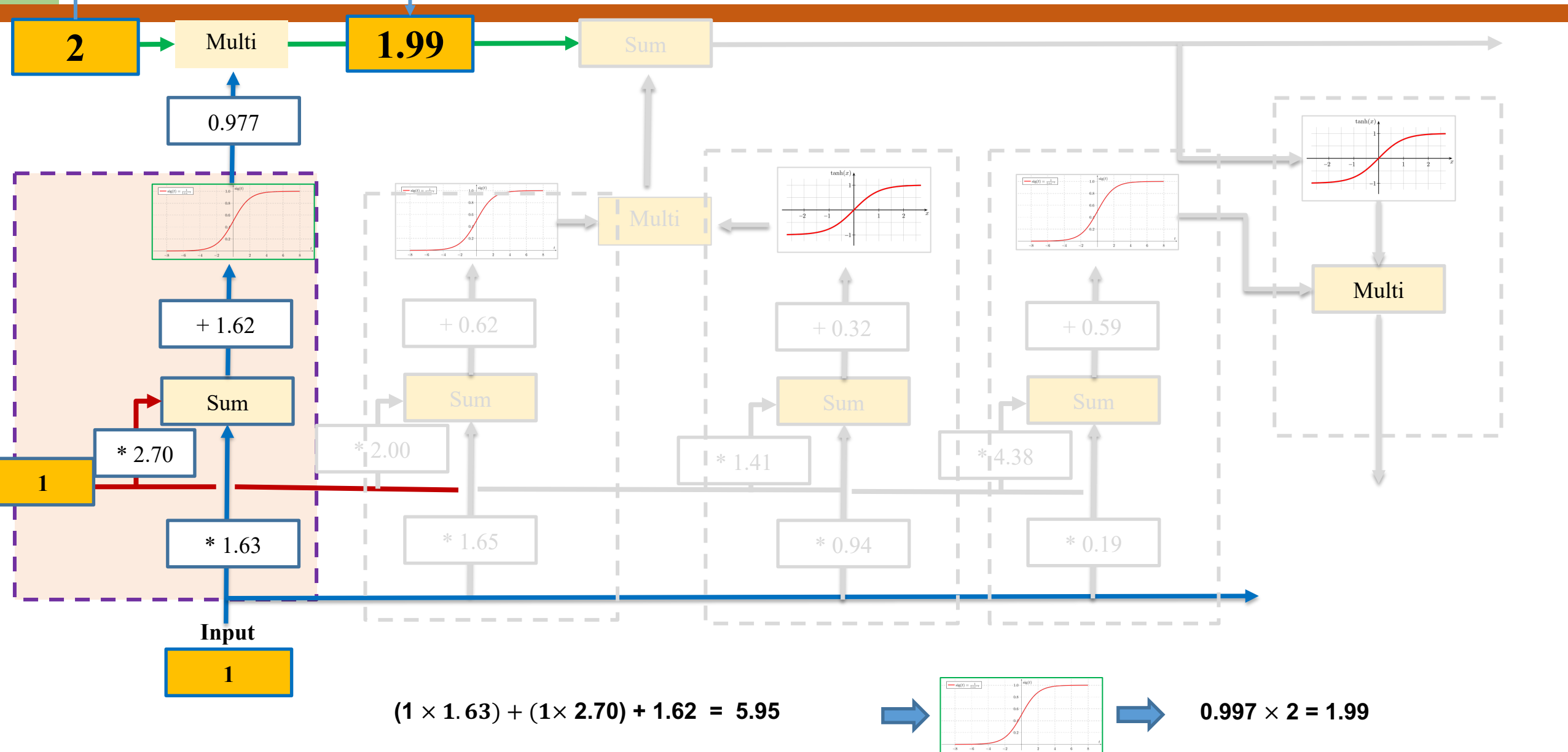
Long-term memories : no bias and weight present => avoid causing exploding and vanishing gradient



How do long-term memory and short-term memories work?

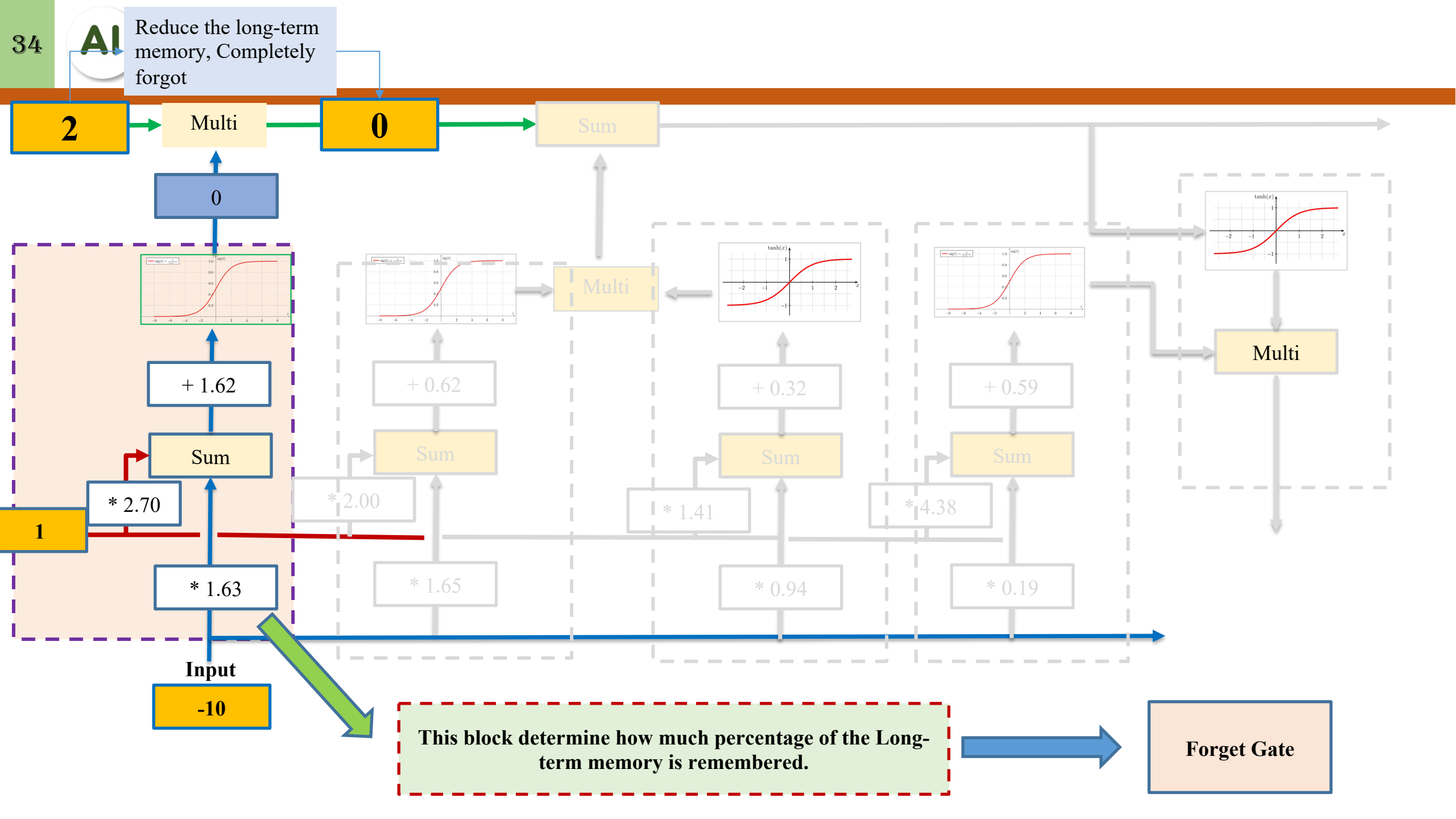


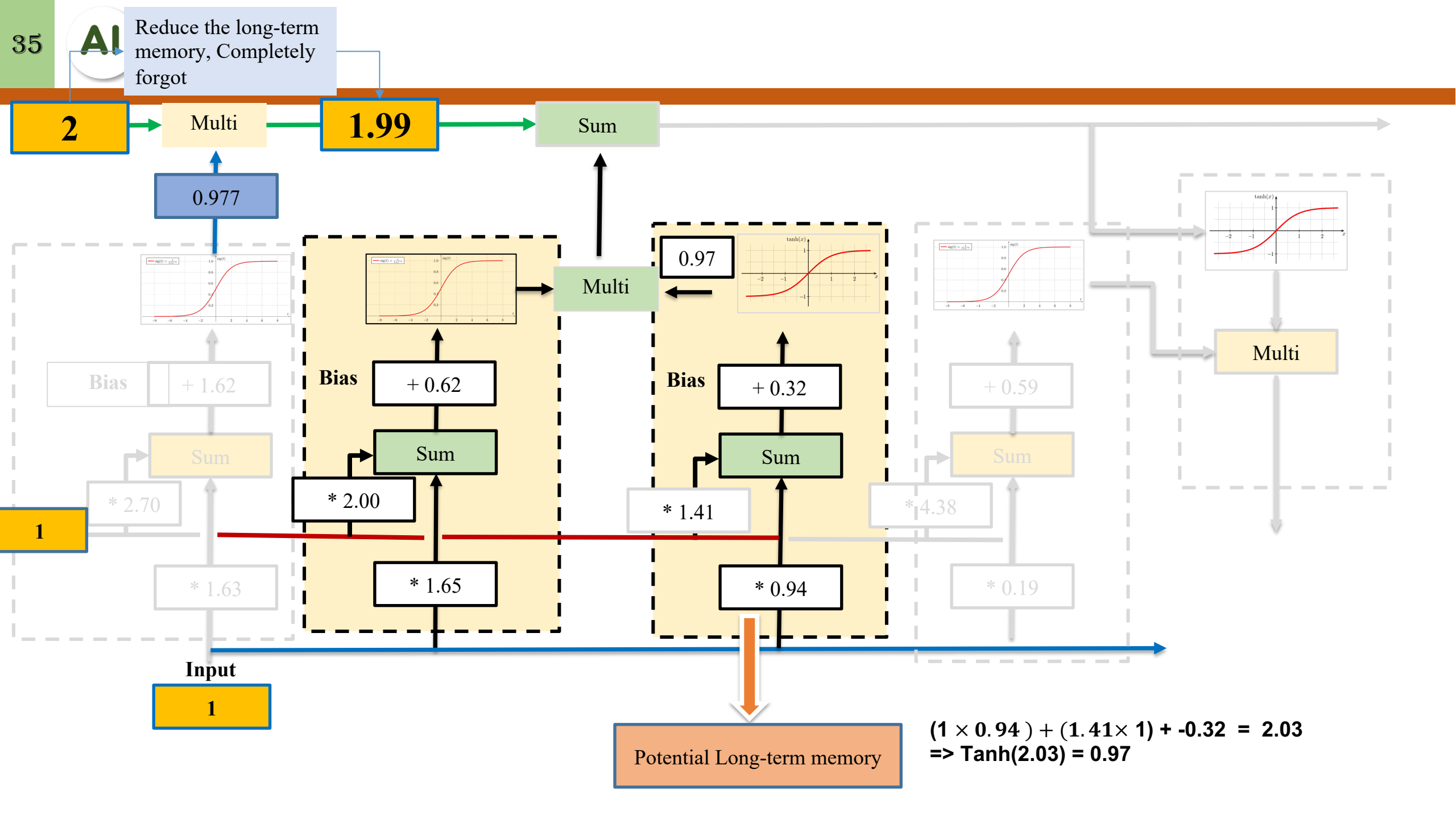
Short-term memories : with bias and weight present





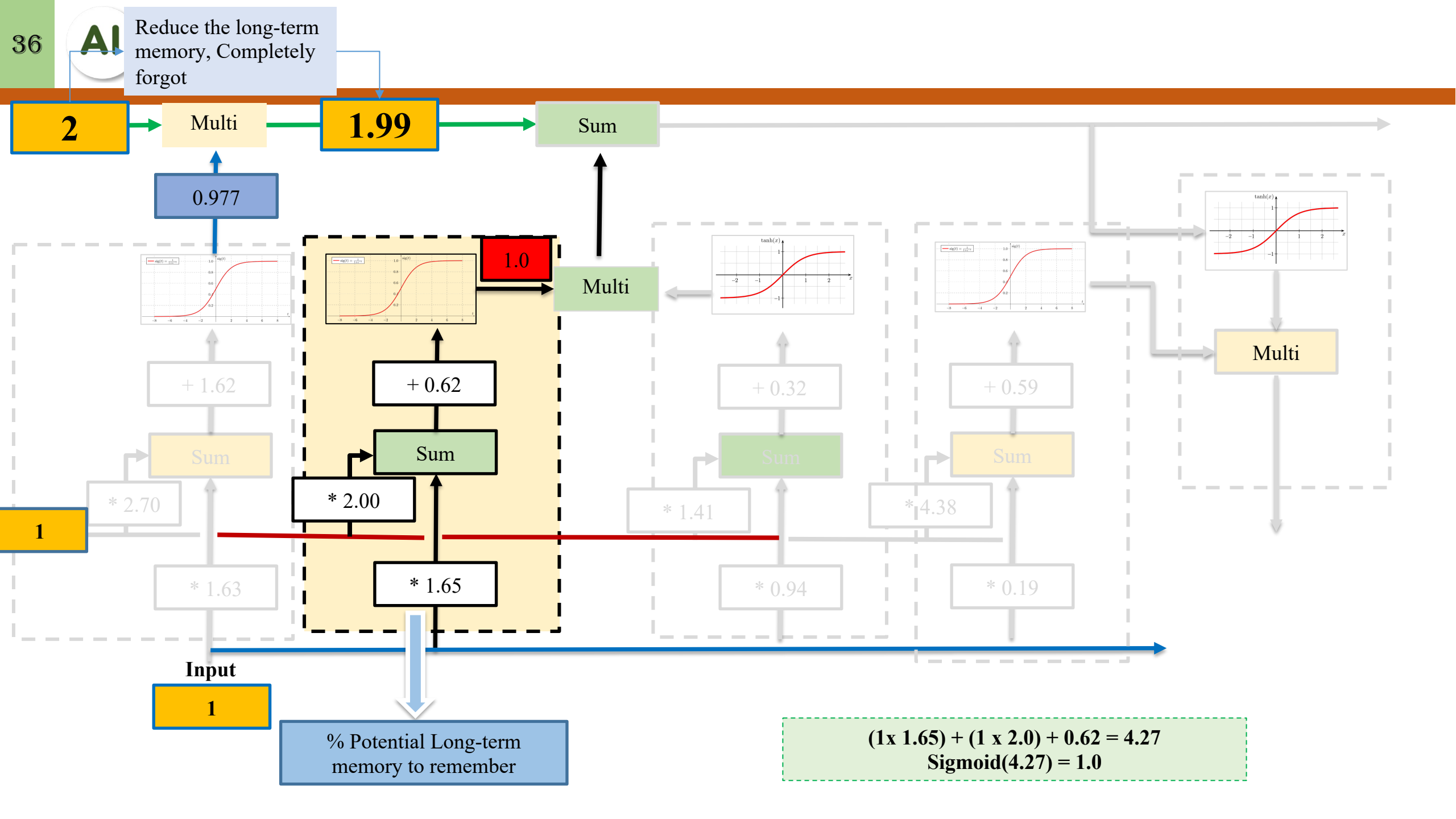
Reduce the long-term memory, Completely forgot

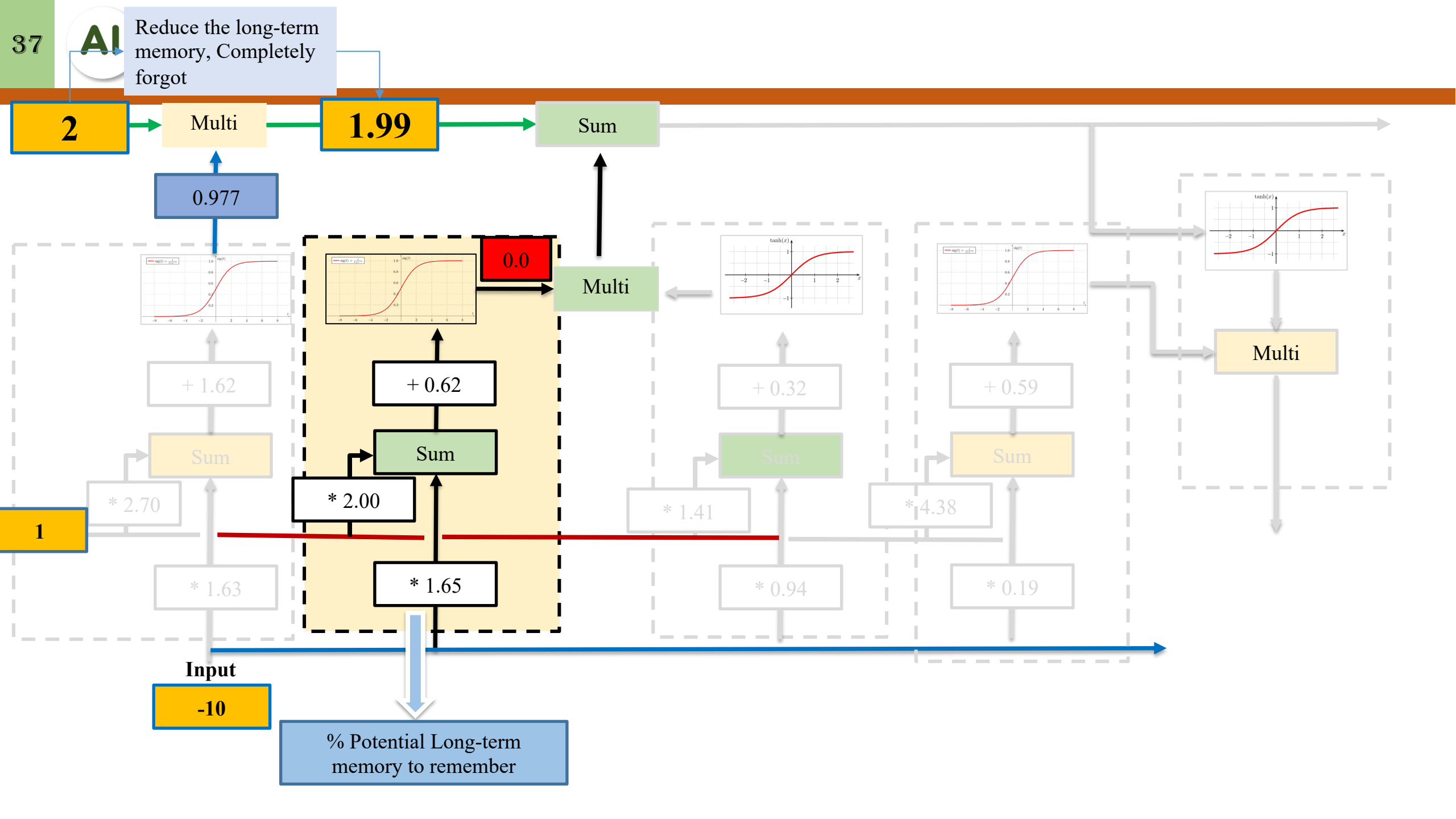


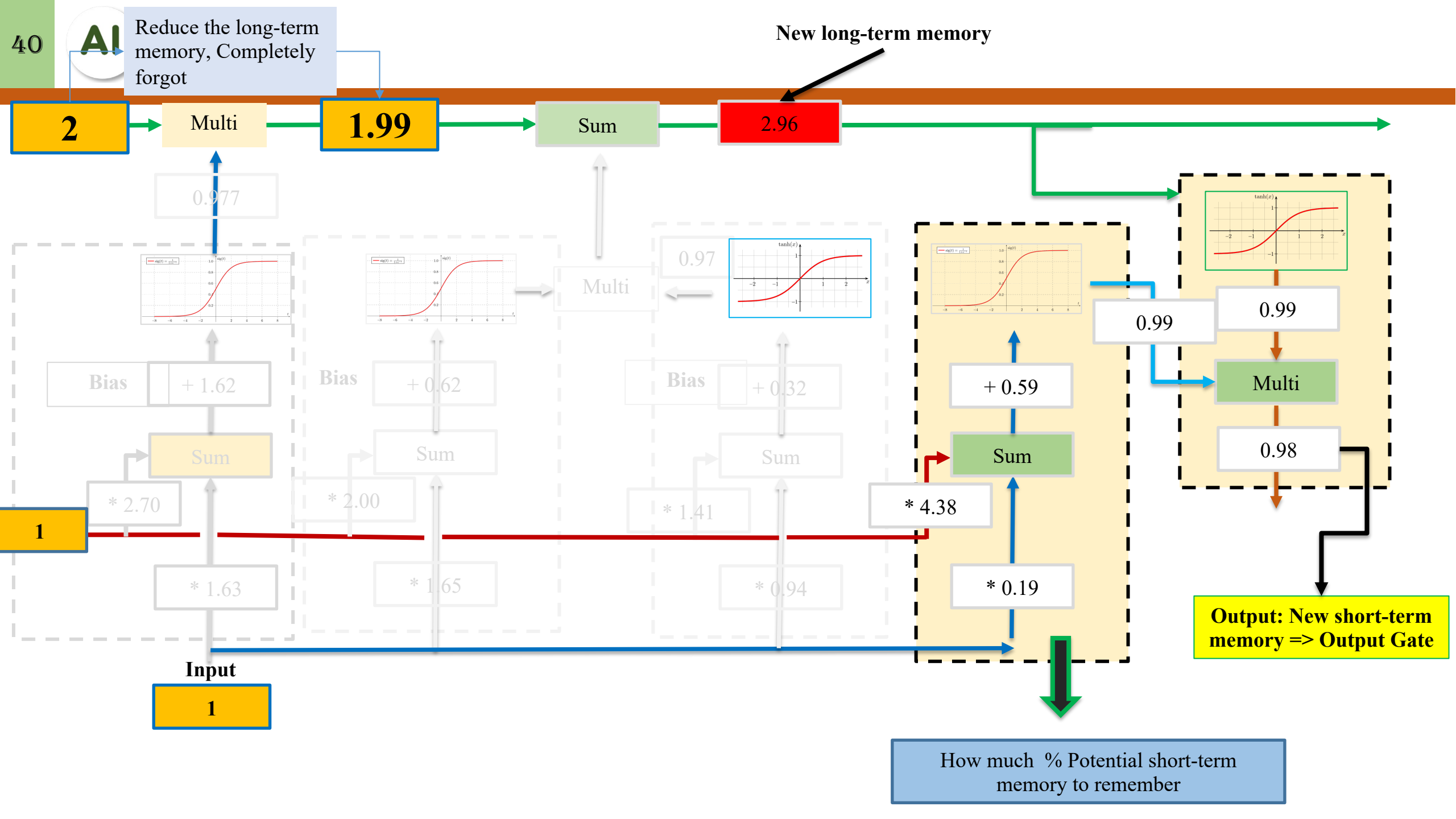




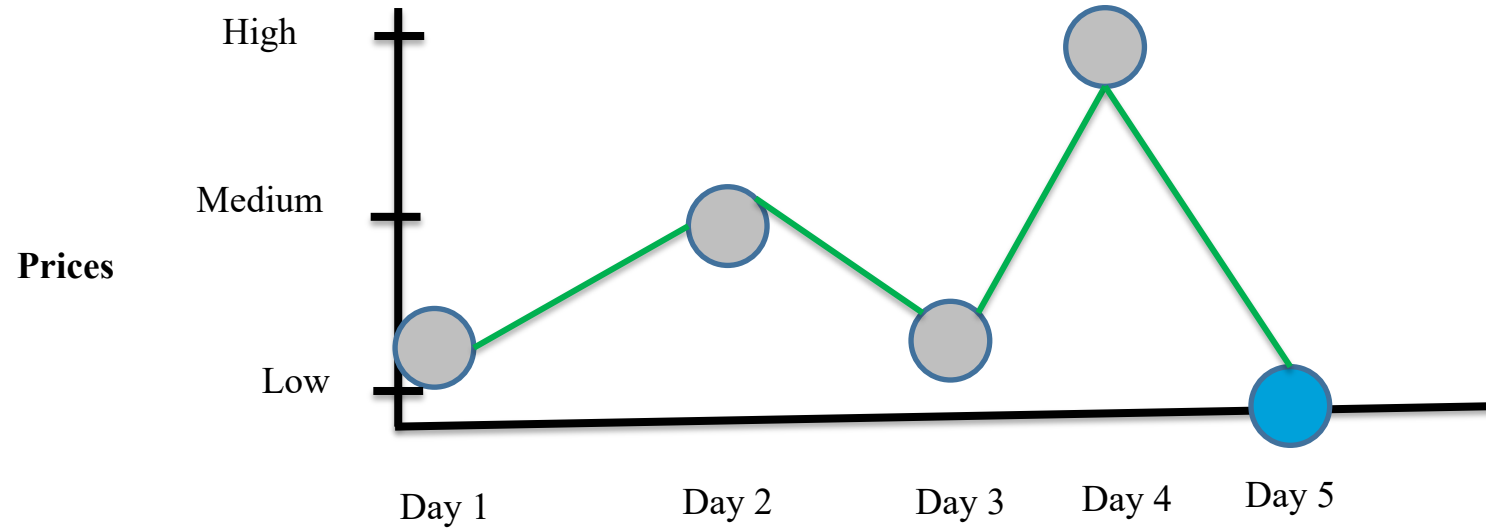
Reduce the long-term memory, Completely forgot







LSTM Unit: Example



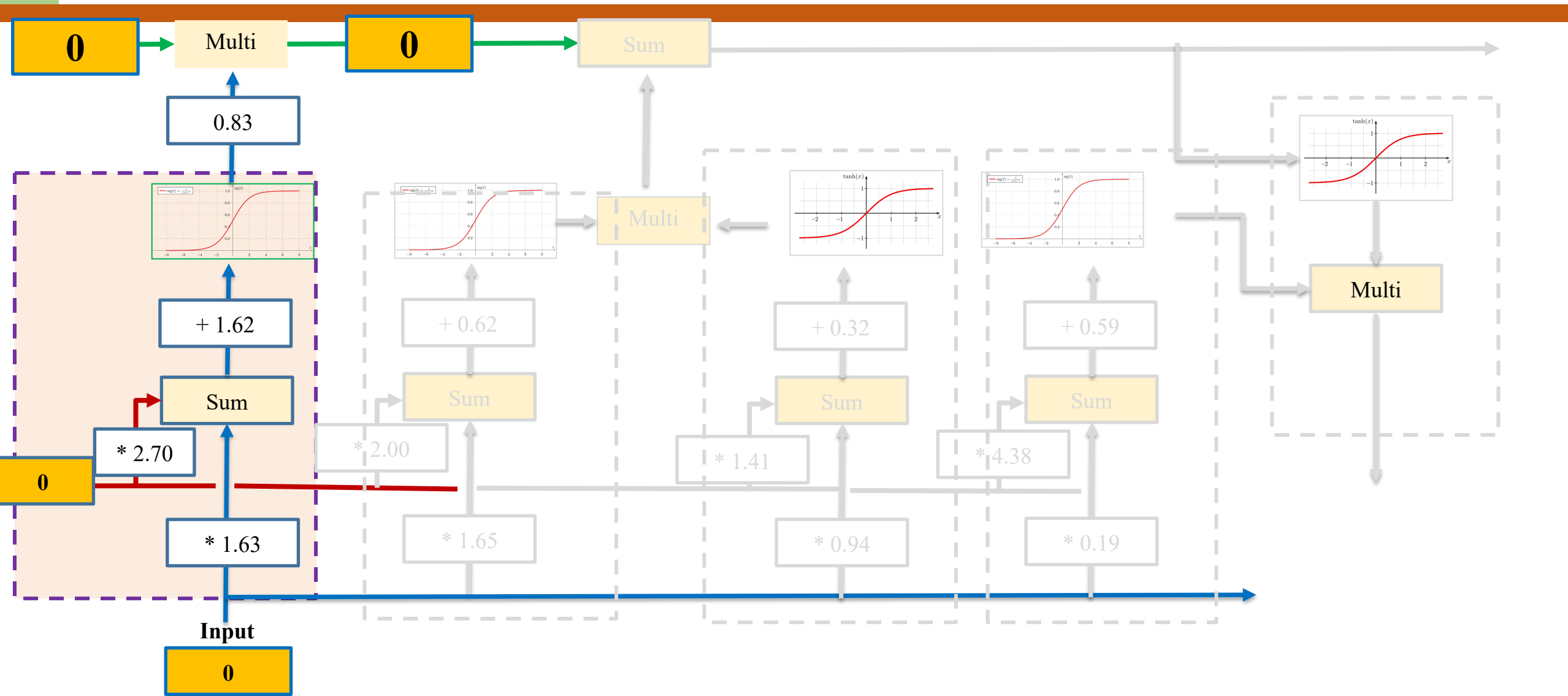
Input: day 1, day 2, day 3, day 4

Output: day 5

Algorithm: Long-short term memory



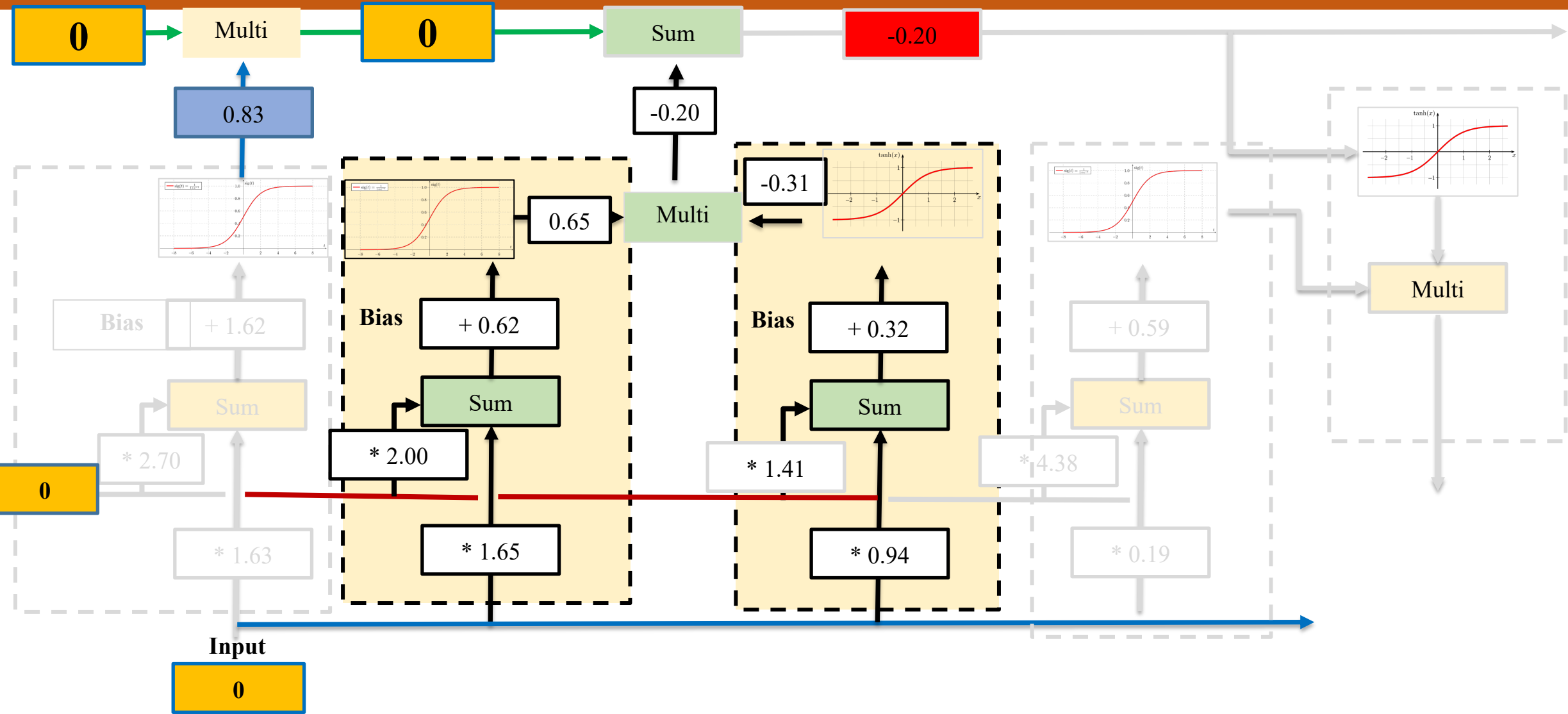
Day 1 input





Day 1 input

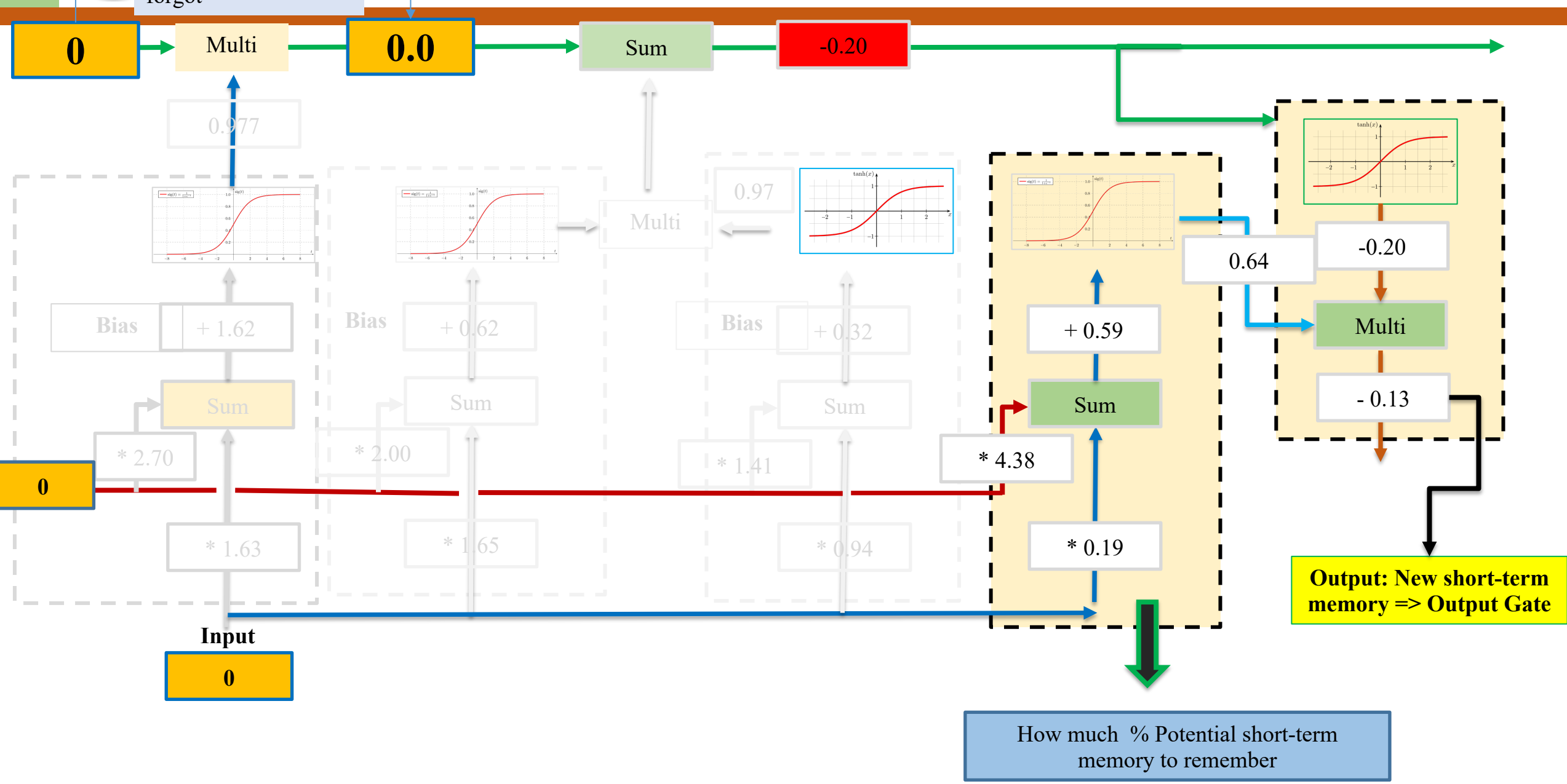
New long-term memory

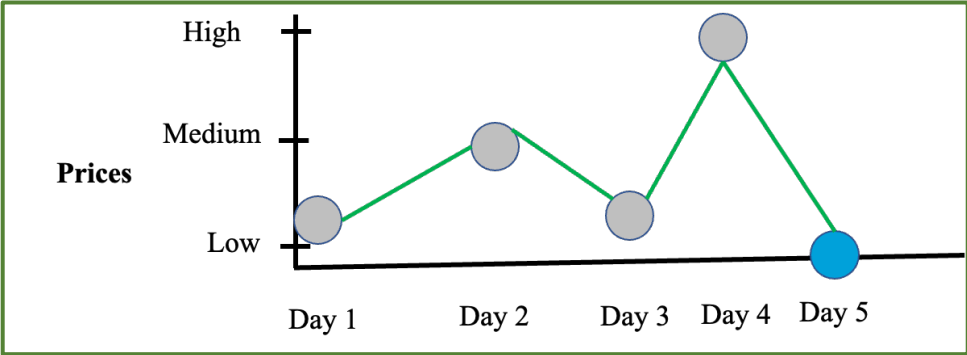
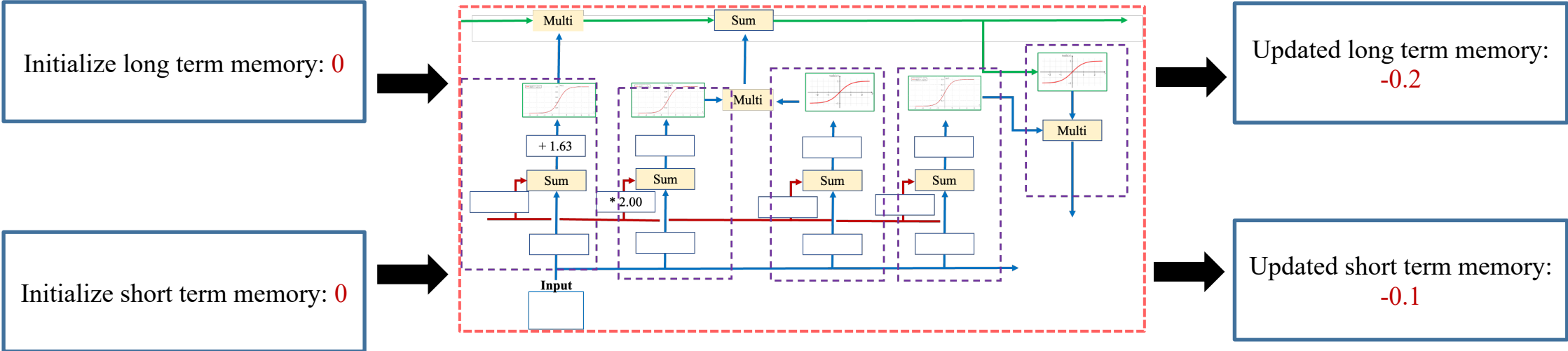


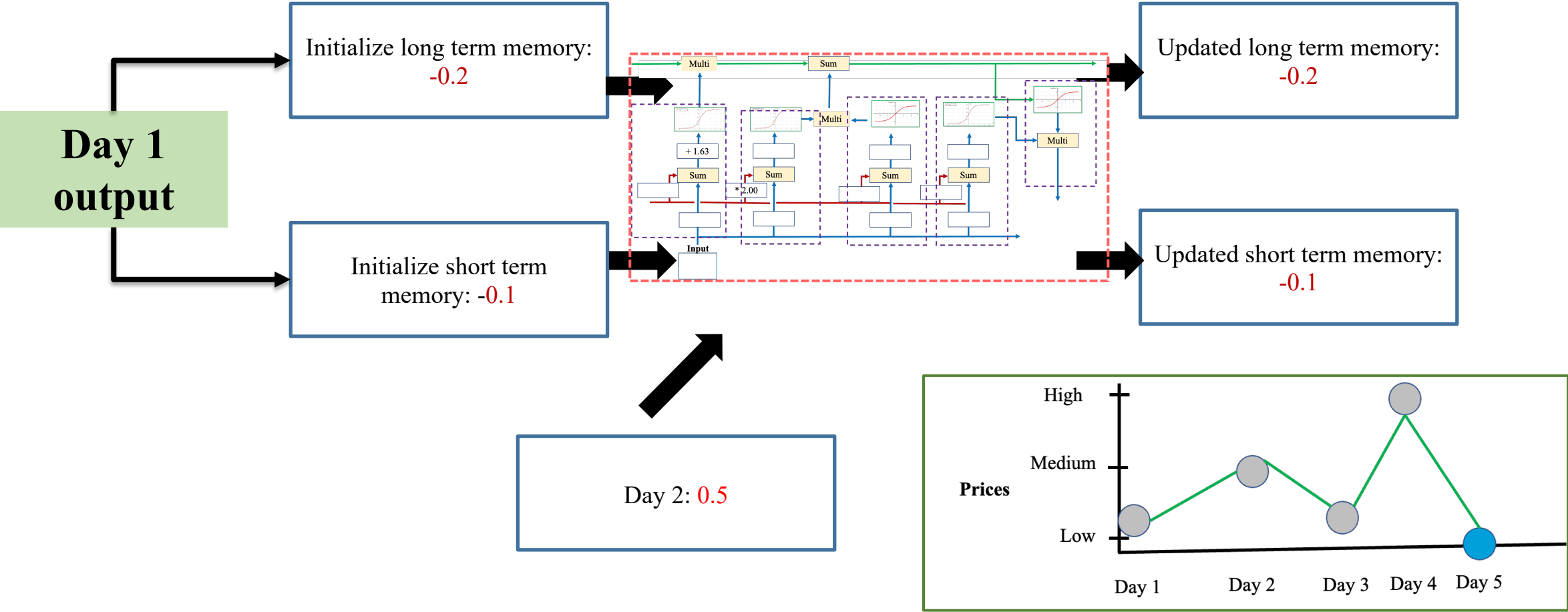


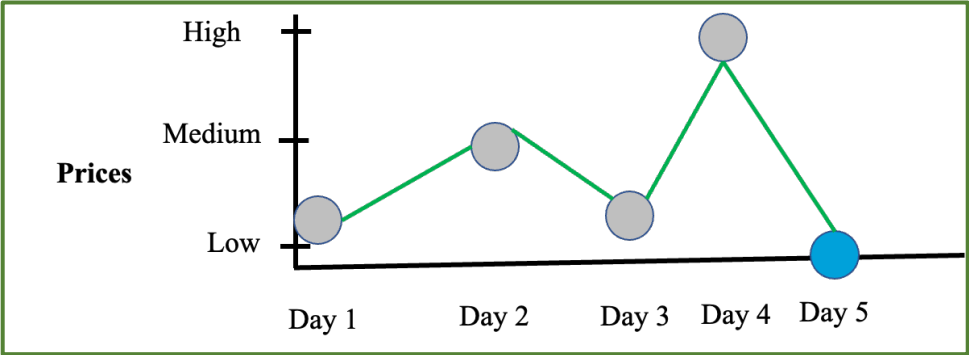
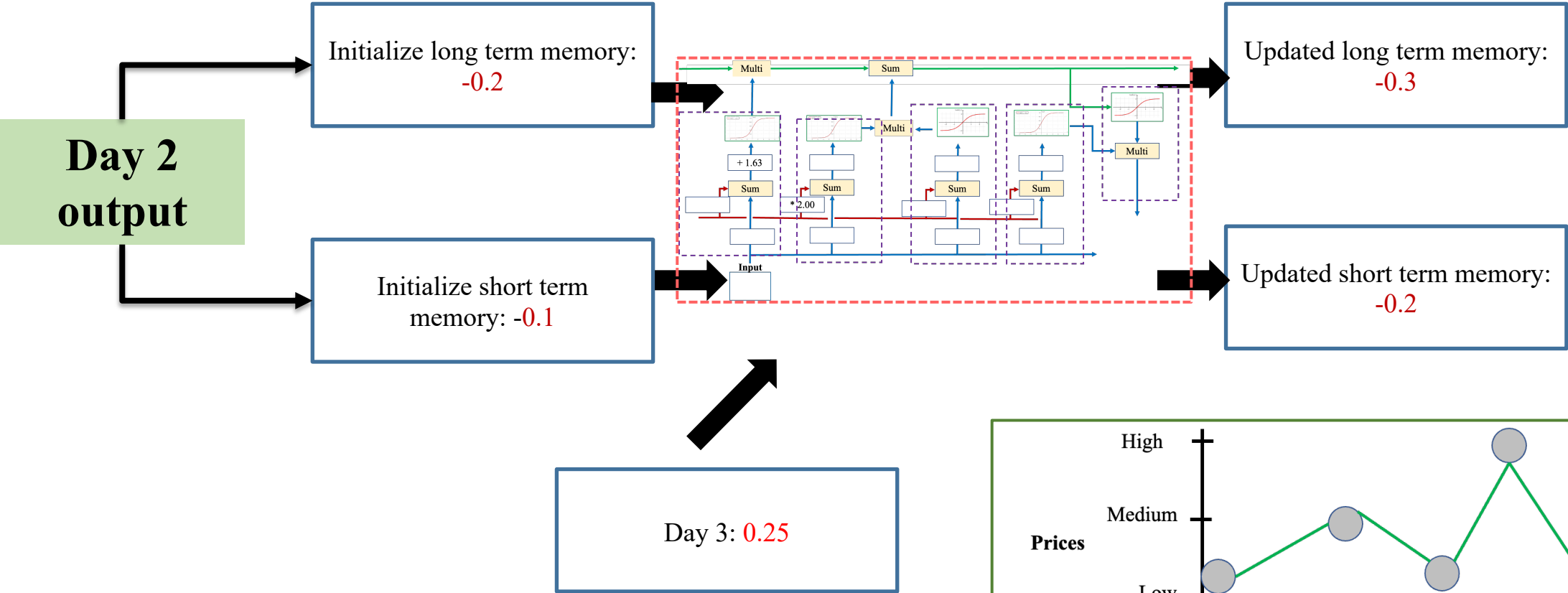
Reduce the long-term memory, Completely forgot

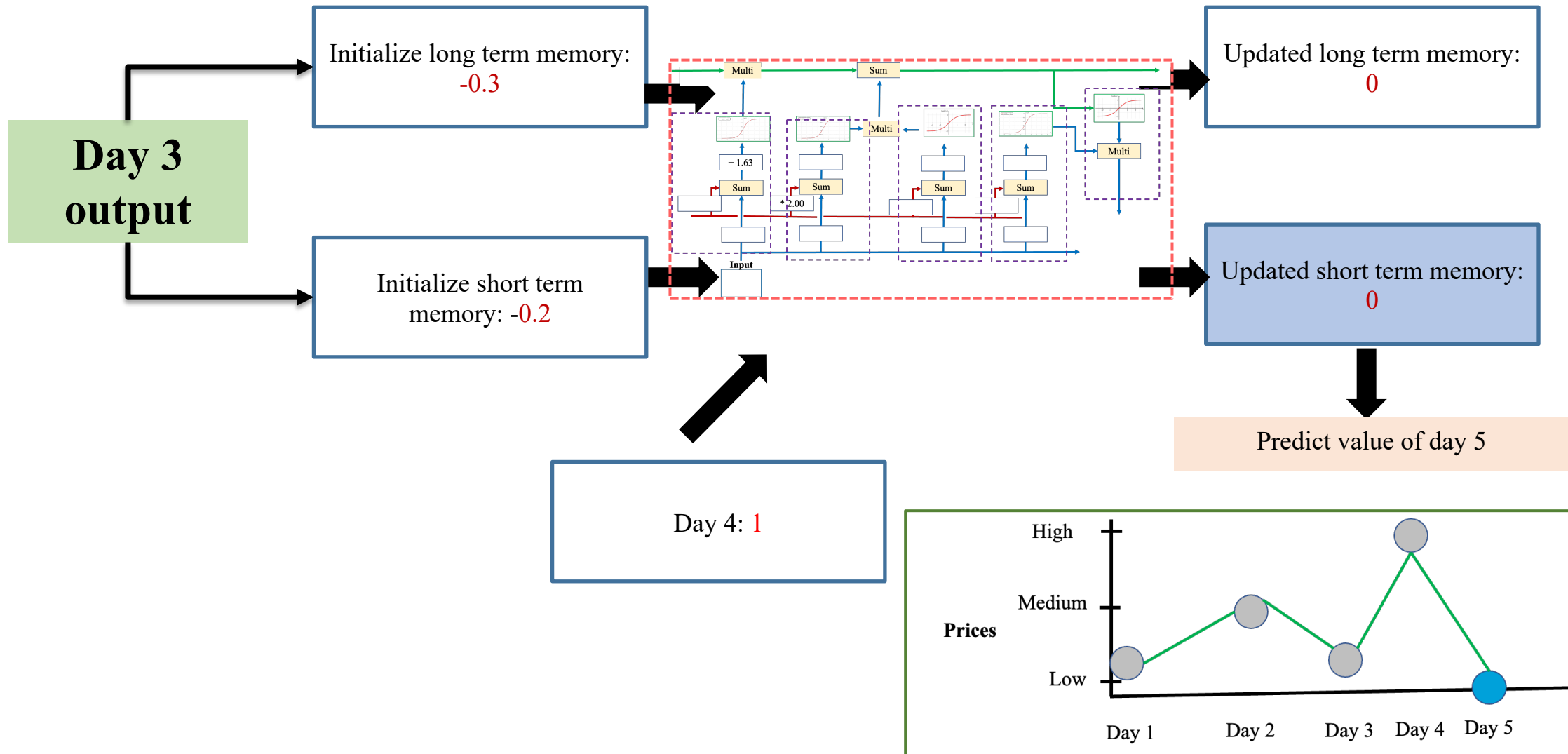
Day 1 input



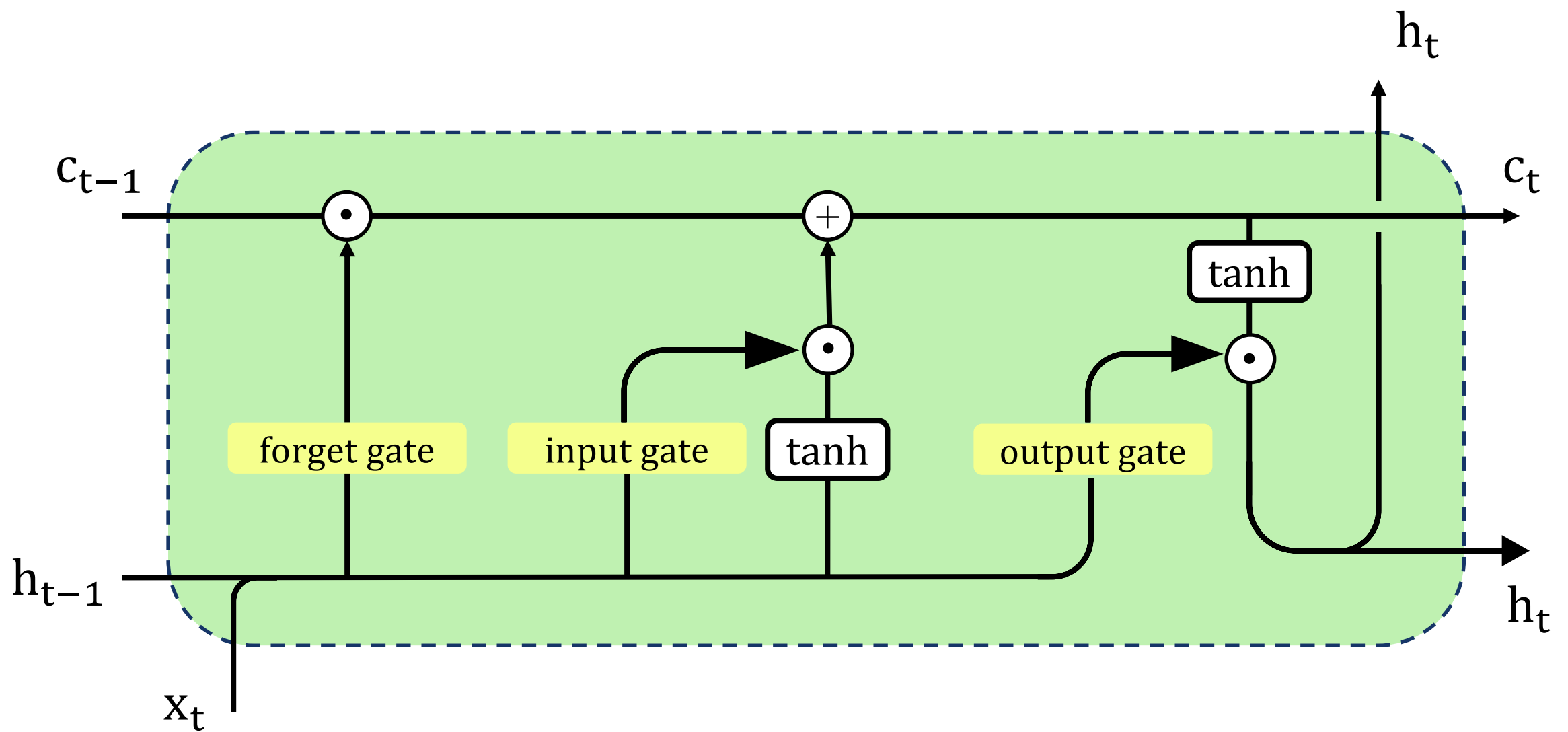




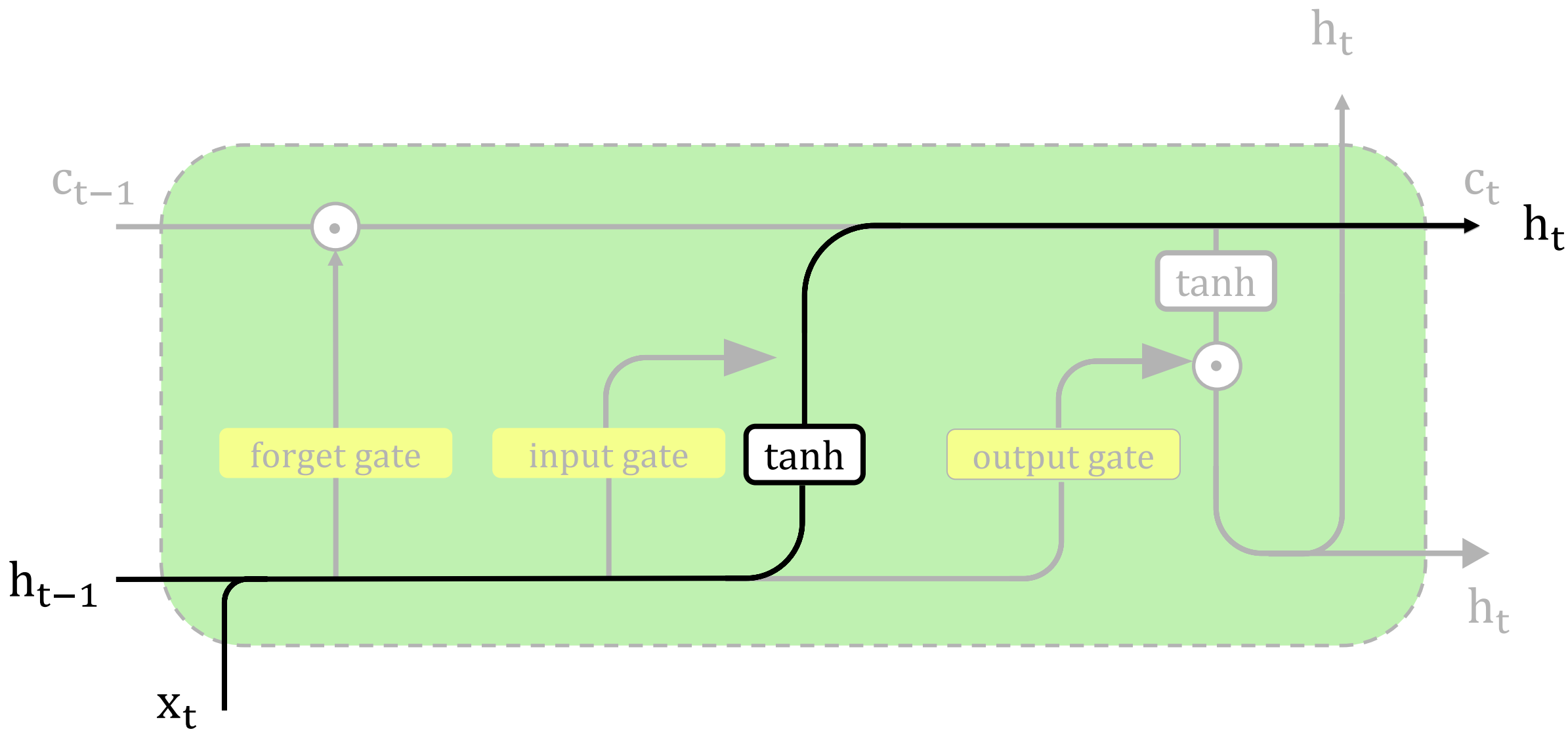




Long-Short Term Memory: Summarize

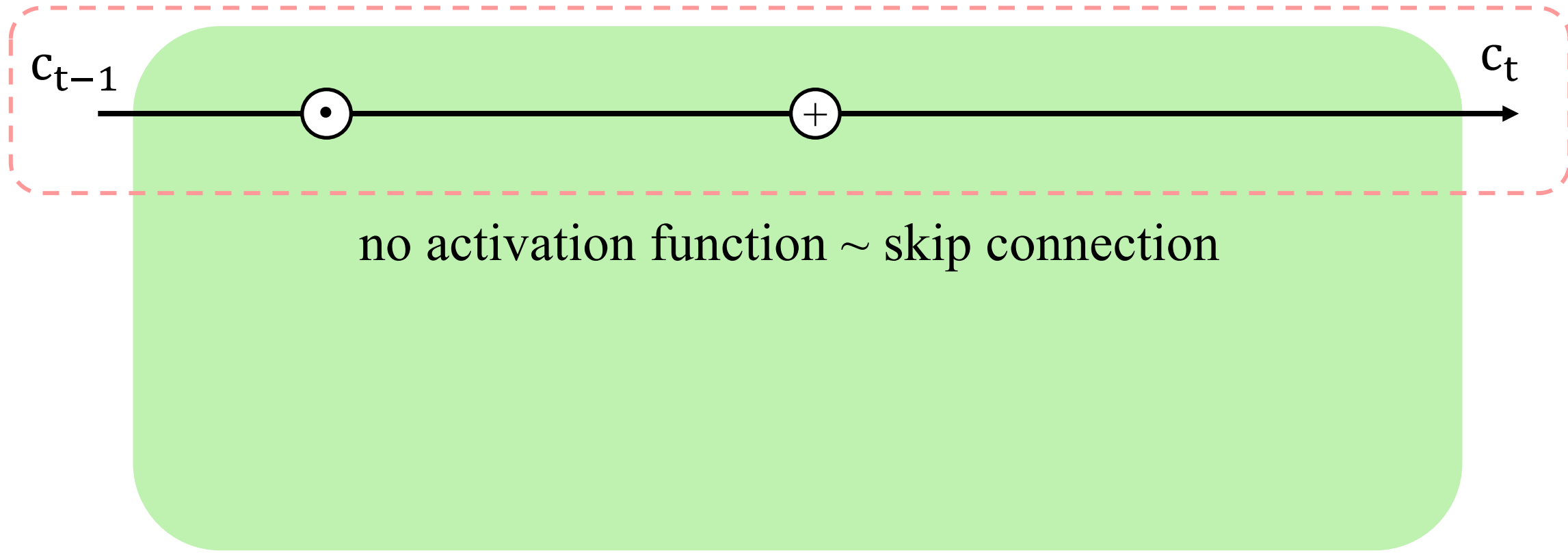


Long-Short Term Memory: Summarize

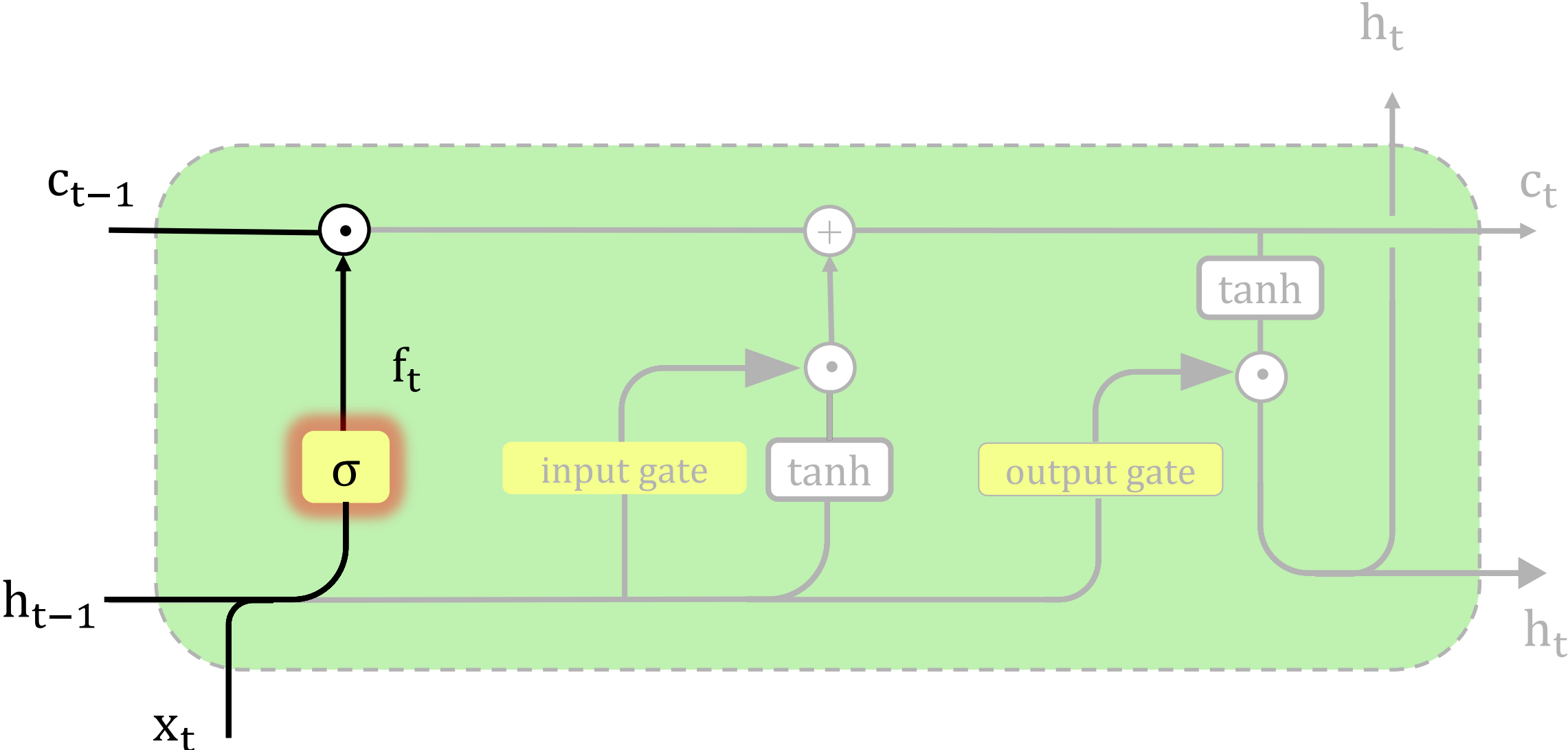


Long-Short Term Memory

Cell states brings information from the past to the future



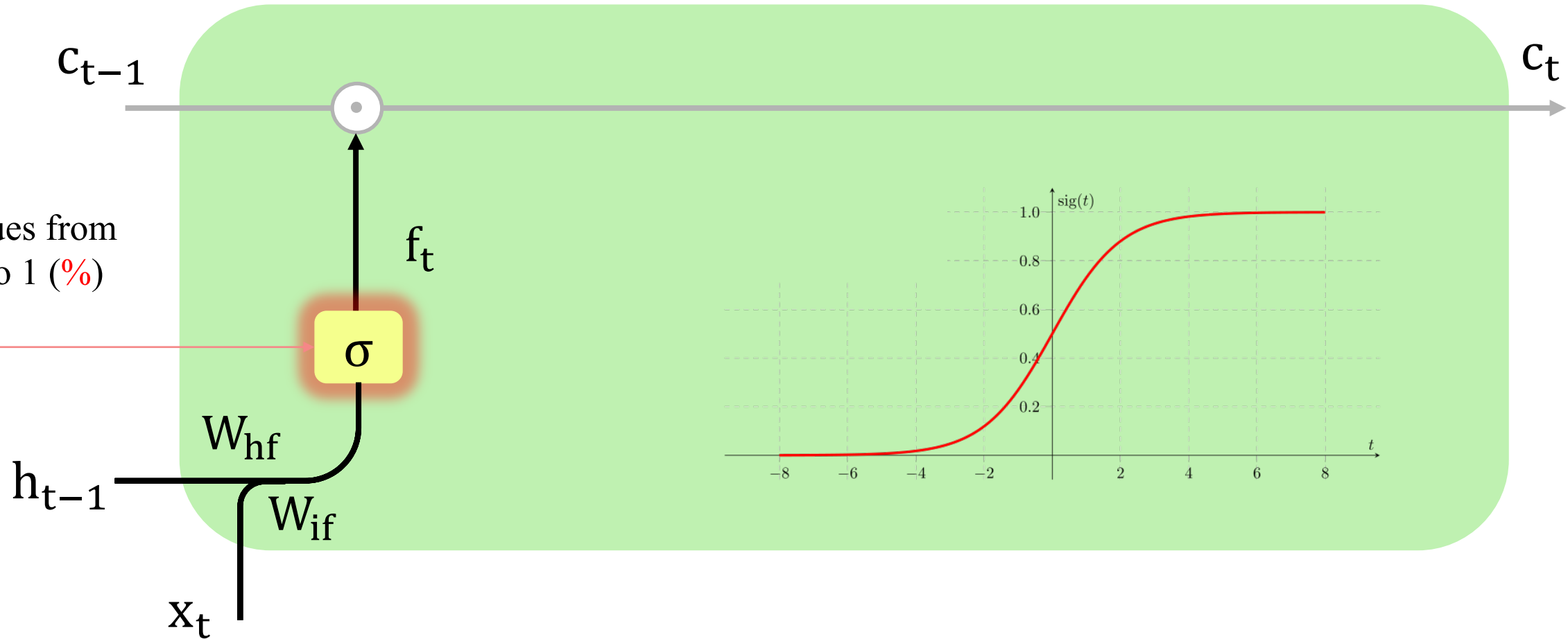
LSTM – Forget Gate



LSTM – Forget Gate

$$f_t = \sigma (W_{if}x_t + b_{if} + W_{hf}h_{t-1} + b_{hf})$$

values from
0 to 1 (%)

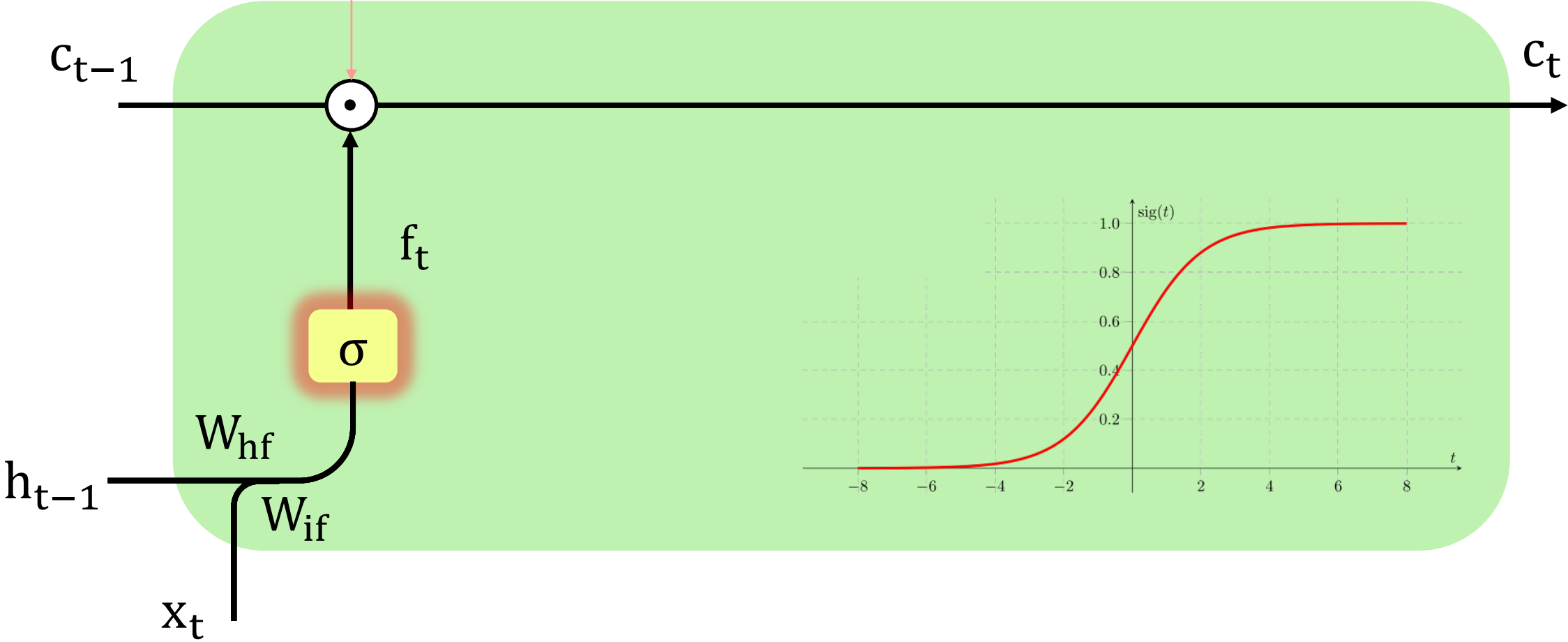


LSTM – Forget Gate

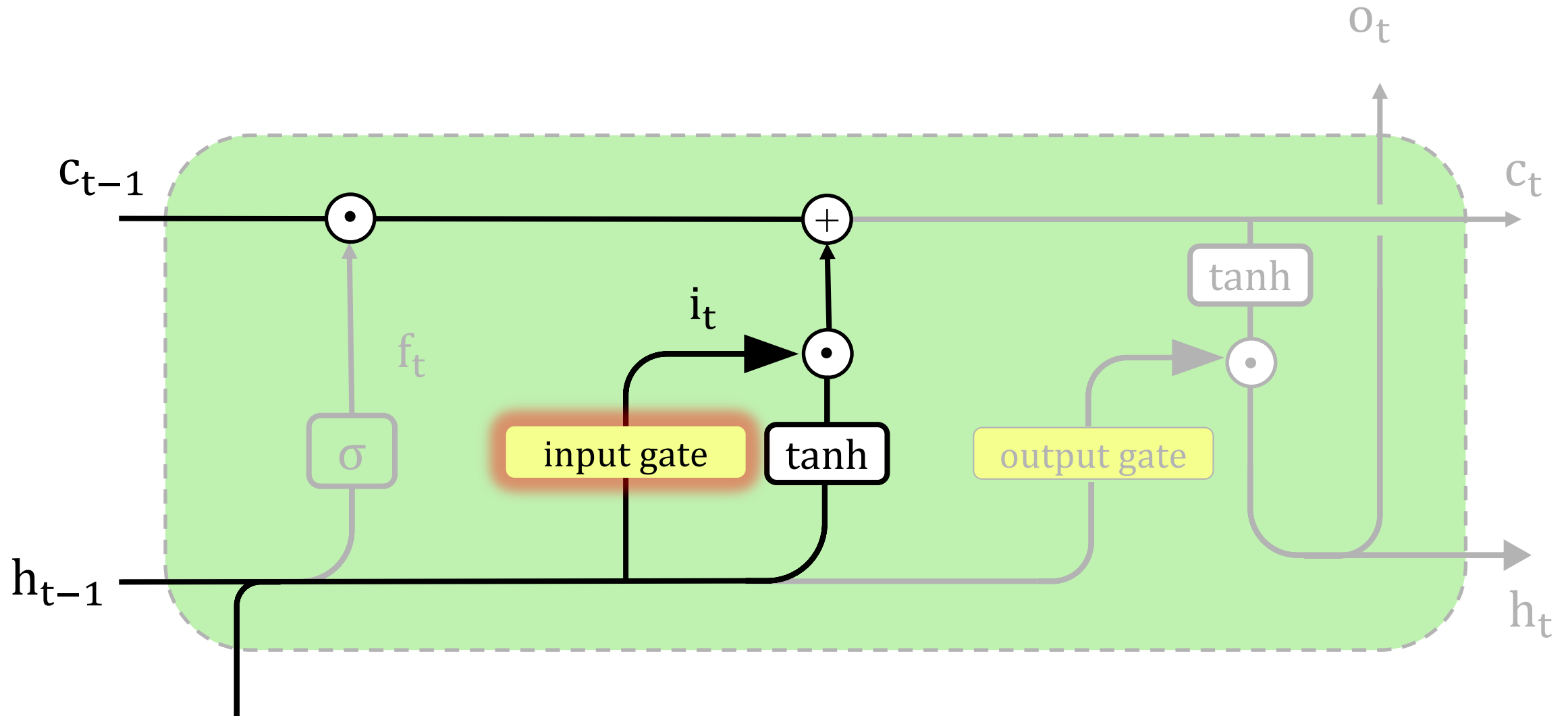
$$f_t = \sigma (W_{if}x_t + b_{if} + W_{hf}h_{t-1} + b_{hf})$$

$$c_t = f_t \odot c_{t-1}$$

always decrease



Input gate



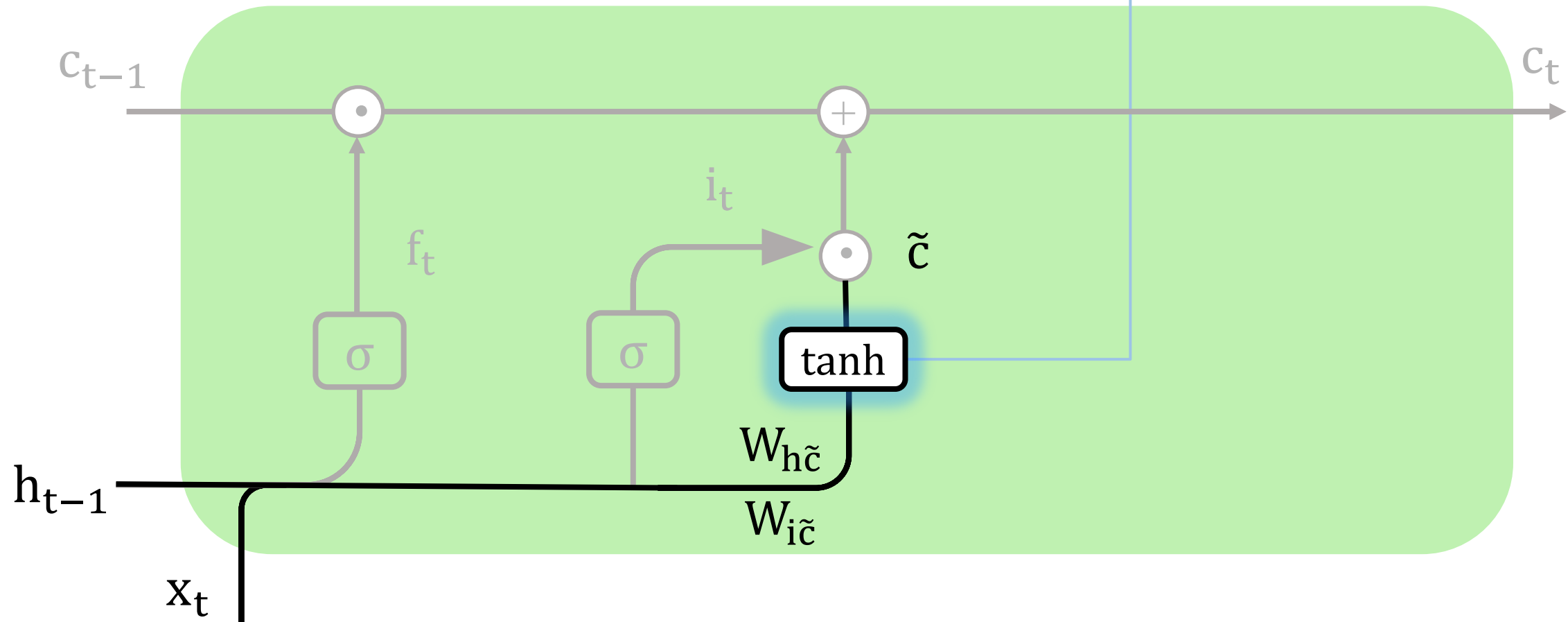
Input gate

$$\tilde{c}_t = \tanh(W_{i\tilde{c}}x_t + b_{i\tilde{c}} + W_{h\tilde{c}}h_{t-1} + b_{h\tilde{c}})$$

$$i_t = \sigma(W_{ii}x_t + b_{ii} + W_{hi}h_{t-1} + b_{hi})$$

input

bad or good
current information

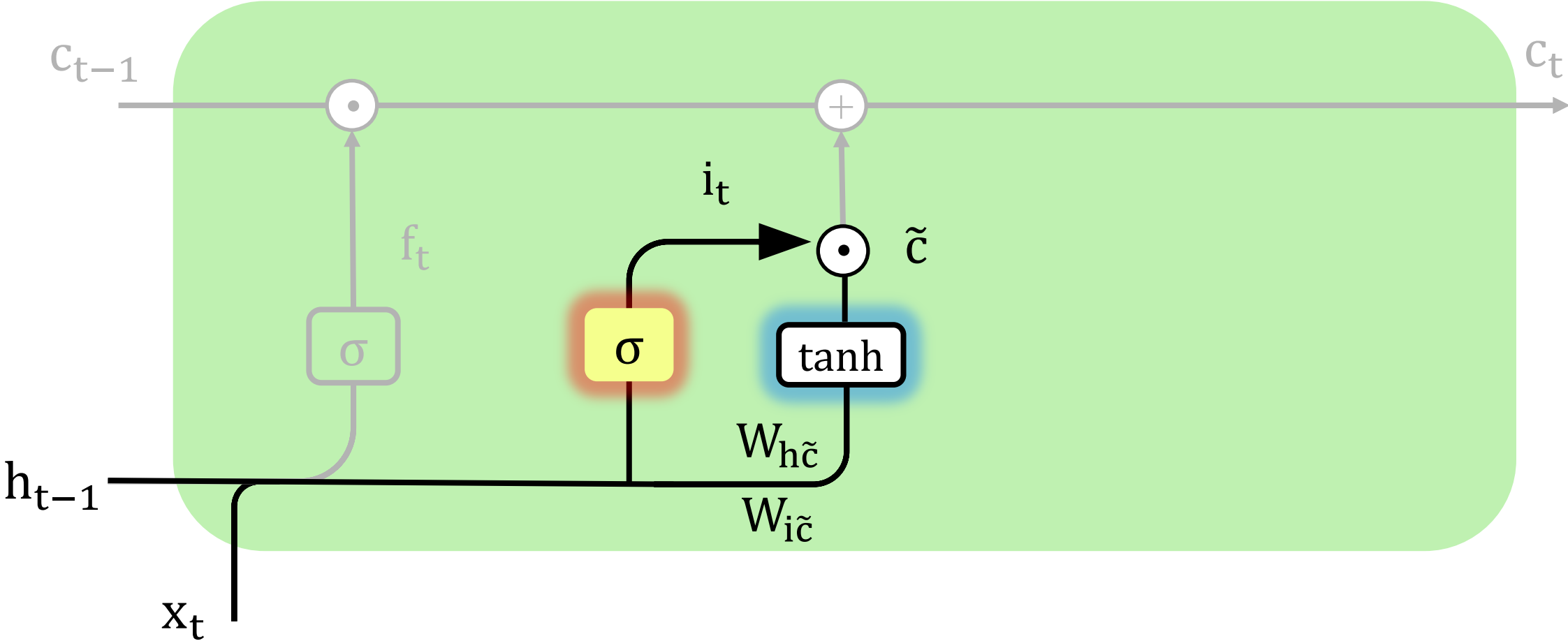


Input gate

$$\tilde{c}_t = \tanh(W_{i\tilde{c}}x_t + b_{i\tilde{c}} + W_{h\tilde{c}}h_{t-1} + b_{h\tilde{c}})$$

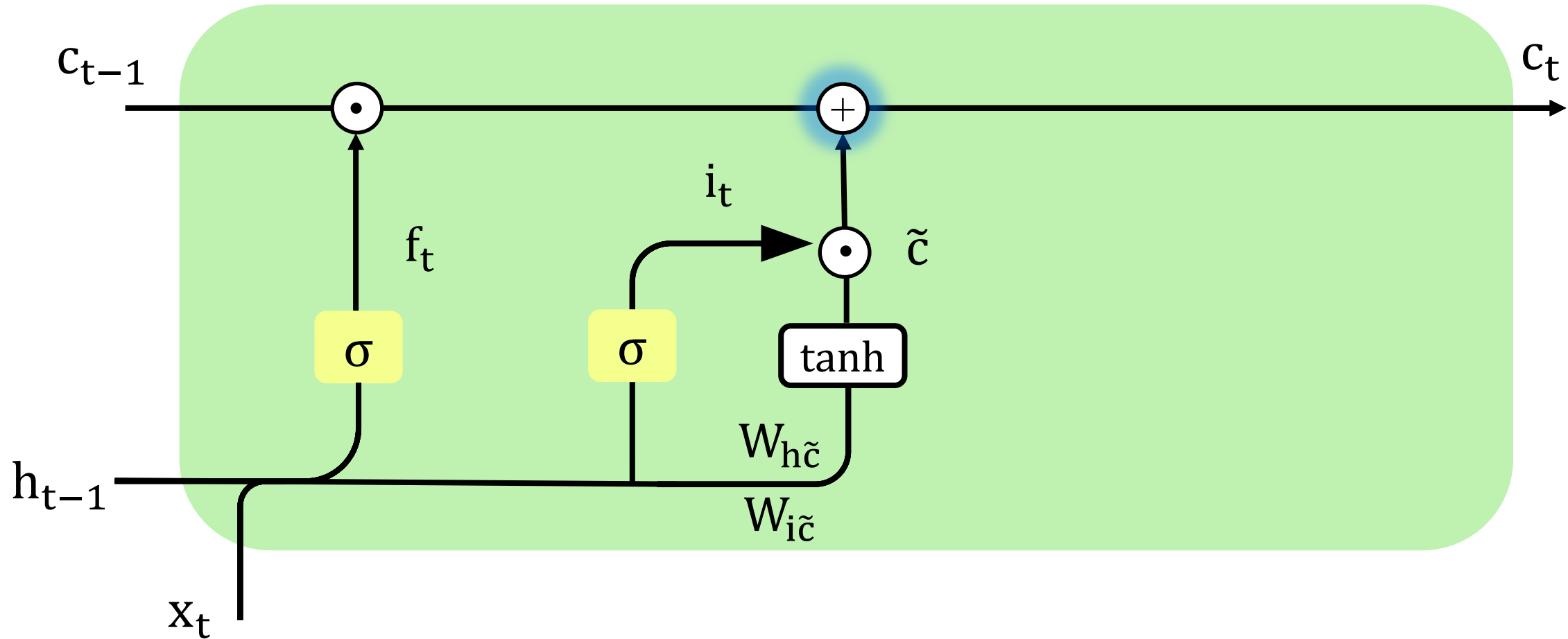
$$i_t = \sigma(W_{ii}x_t + b_{ii} + W_{hi}h_{t-1} + b_{hi})$$

(%) bad or good
current information



Input gate

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad \text{add (\%) current information to Long-term}$$

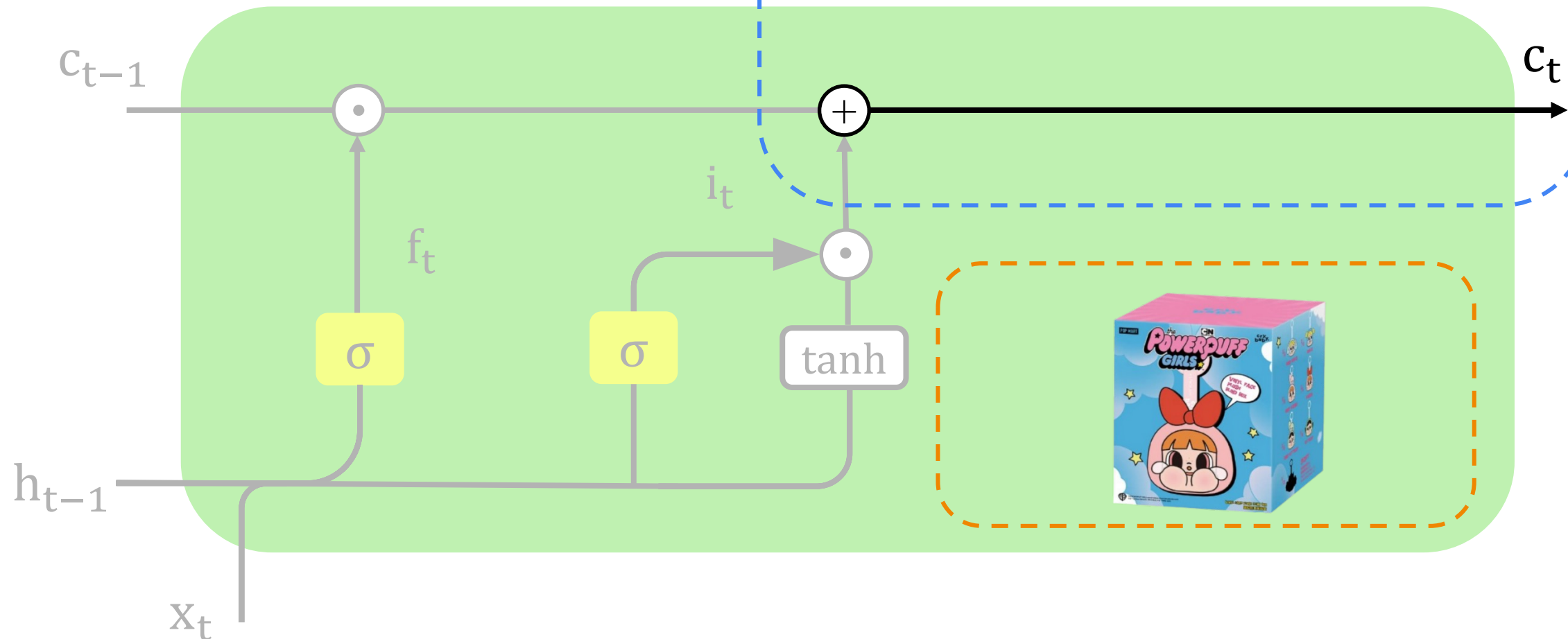


$$\tilde{c}_t = \tanh(W_{i\tilde{c}}x_t + b_{i\tilde{c}} + W_{h\tilde{c}}h_{t-1} + b_{h\tilde{c}})$$

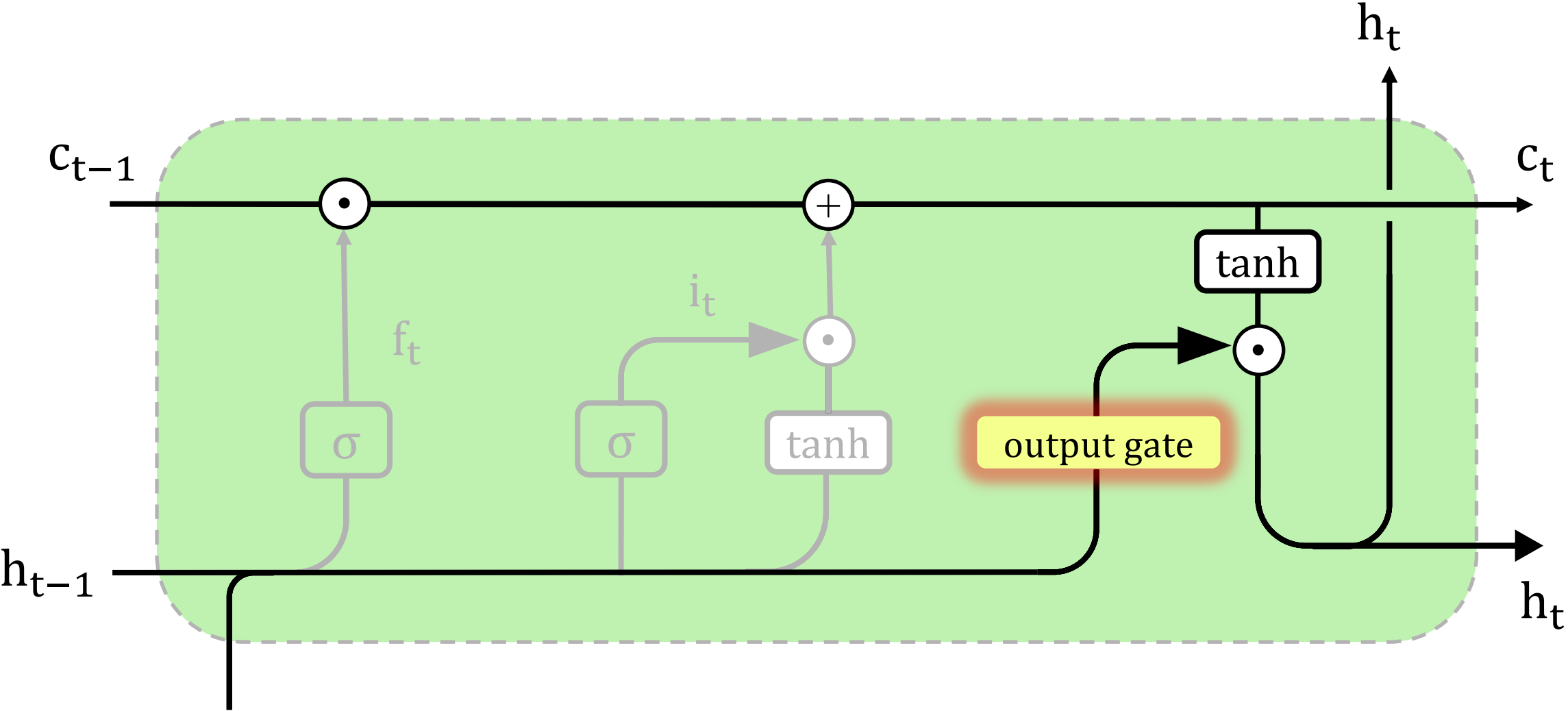
$$i_t = \sigma(W_{ii}x_t + b_{ii} + W_{hi}h_{t-1} + b_{hi})$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

We will use this Long-term to create Short-term and predict the output



Output gate

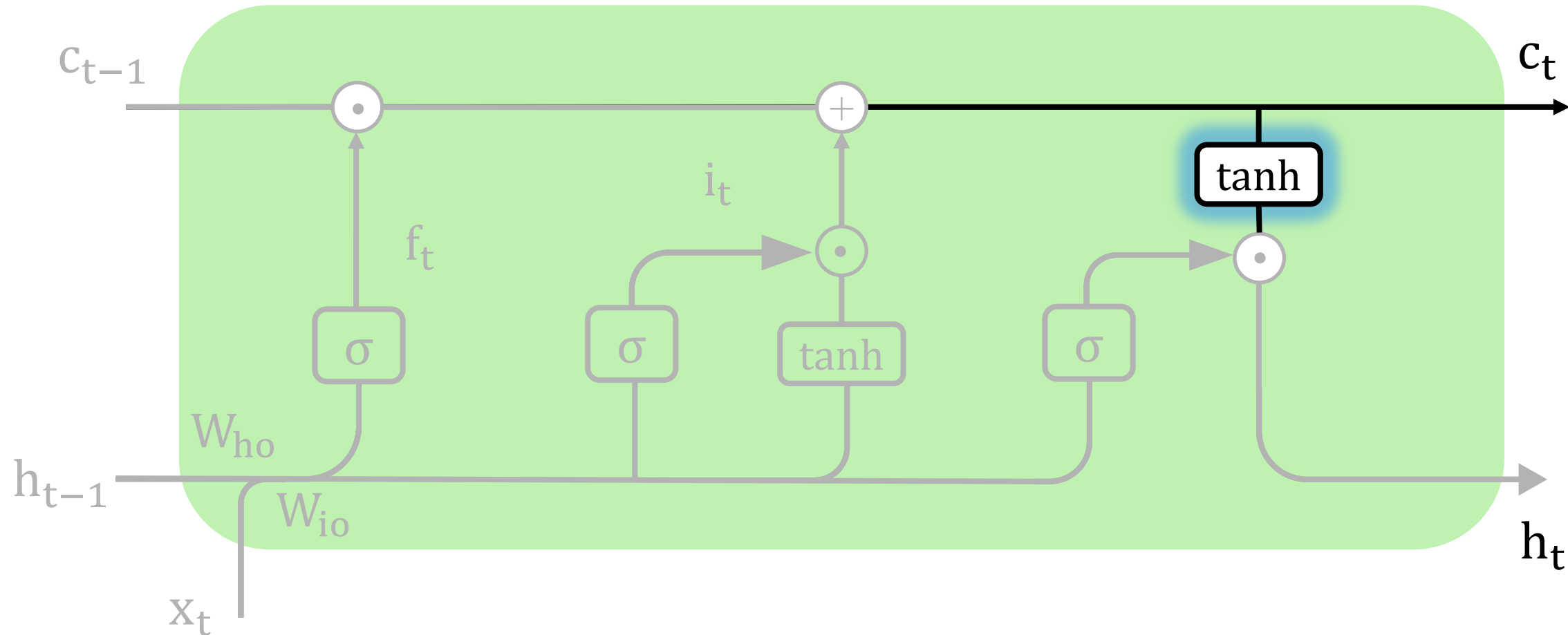


Output gate

$$o_t = \sigma(W_{io}x_t + b_{io} + W_{ho}h_{t-1} + b_{ho})$$

$$h_t = o_t \odot \tanh(c_t)$$

which Long-term information should be used for Short-term?

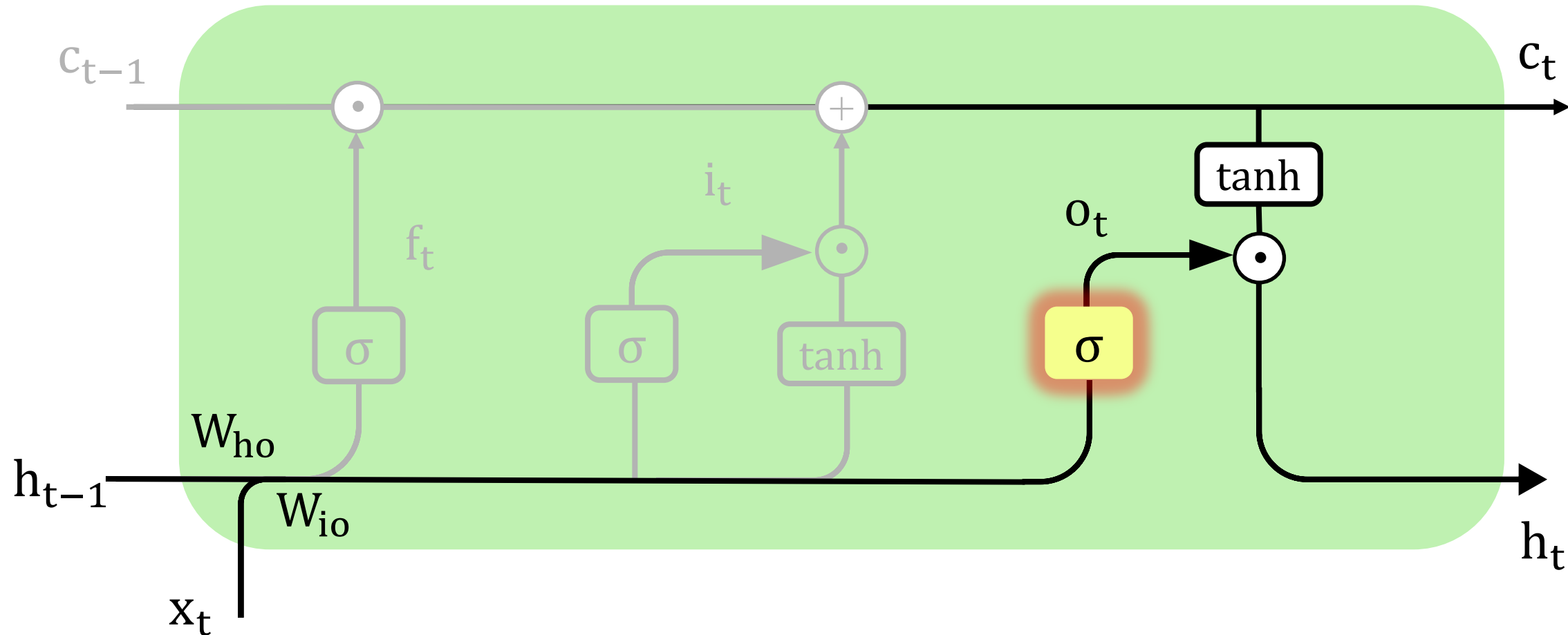


Output gate

$$o_t = \sigma(W_{io}x_t + b_{io} + W_{ho}h_{t-1} + b_{ho})$$

$$h_t = o_t \odot \tanh(c_t)$$

(%) which Long-term information should be used for Short-term?

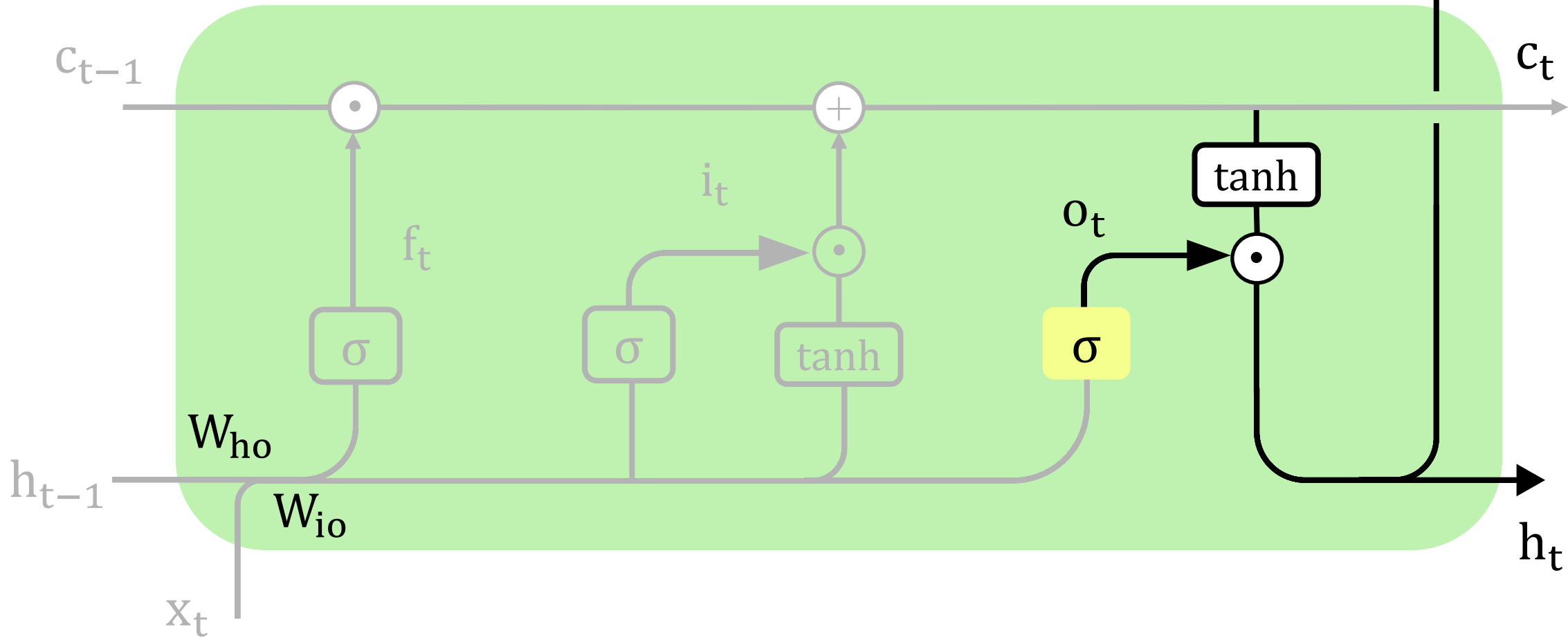


Output gate

$$o_t = \sigma(W_{io}x_t + b_{io} + W_{ho}h_{t-1} + b_{ho})$$

$$h_t = o_t \odot \tanh(c_t)$$

this is also output of LSTM unit



LSTM: Summarize

$$f_t = \sigma(W_{if}x_t + b_{if} + W_{hf}h_{t-1} + b_{hf})$$

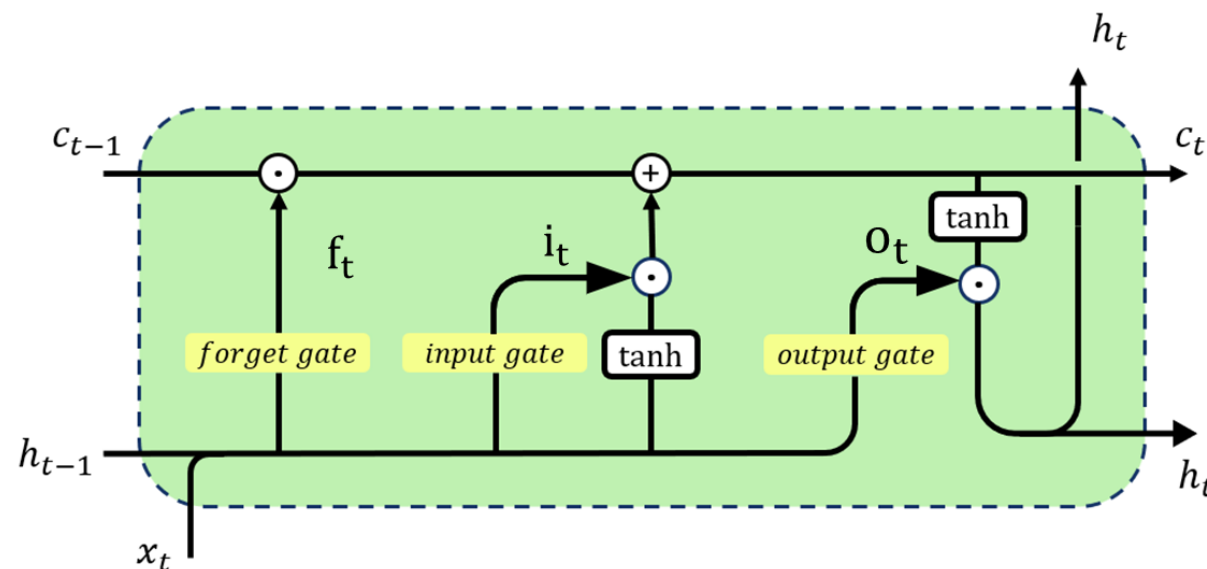
$$i_t = \sigma(W_{ii}x_t + b_{ii} + W_{hi}h_{t-1} + b_{hi})$$

$$o_t = \sigma(W_{io}x_t + b_{io} + W_{ho}h_{t-1} + b_{ho})$$

$$\tilde{c}_t = \tanh(W_{i\tilde{c}}x_t + b_{i\tilde{c}} + W_{h\tilde{c}}h_{t-1} + b_{h\tilde{c}})$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

$$h_t = o_t \odot \tanh(c_t)$$



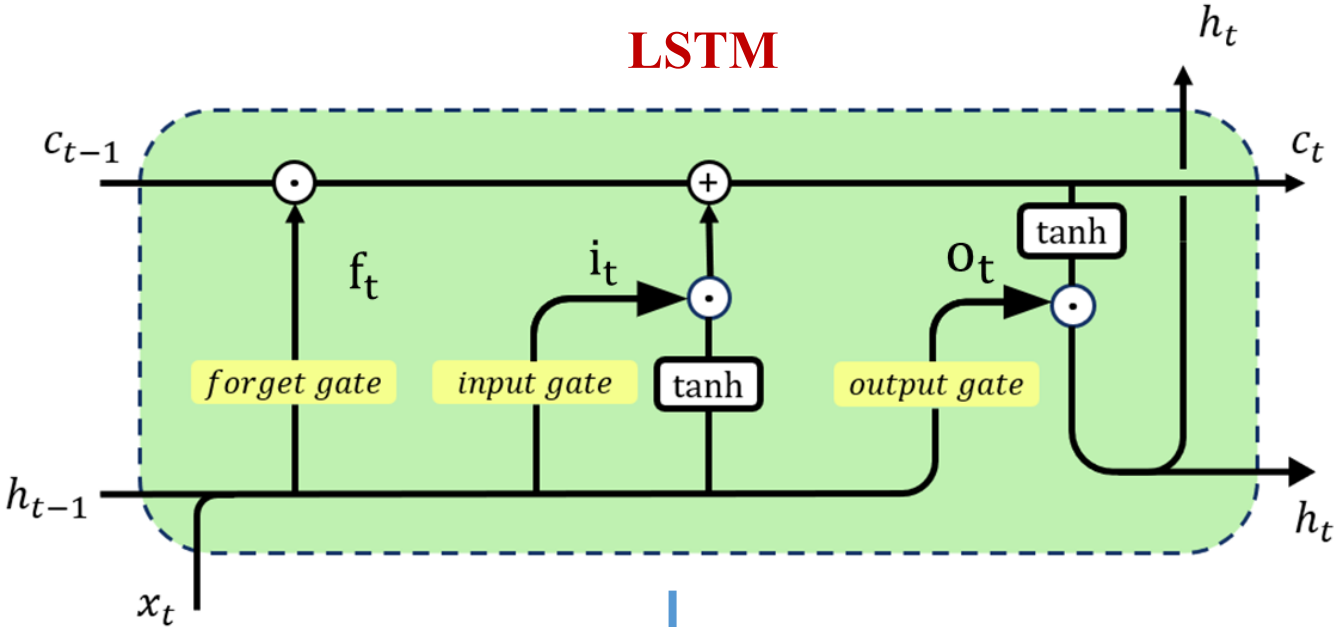
Outline

- **Review: Recurrent Neural Network (RNN)**
- **Long-Short Term Memory (LSTM)**
- **Gated Recurrent Unit (GRU)**
- **Time Series – Basic Knowledge**
- **LSTM, GRU for Time Series Data**
- **LSTM, GRU for Text Data**



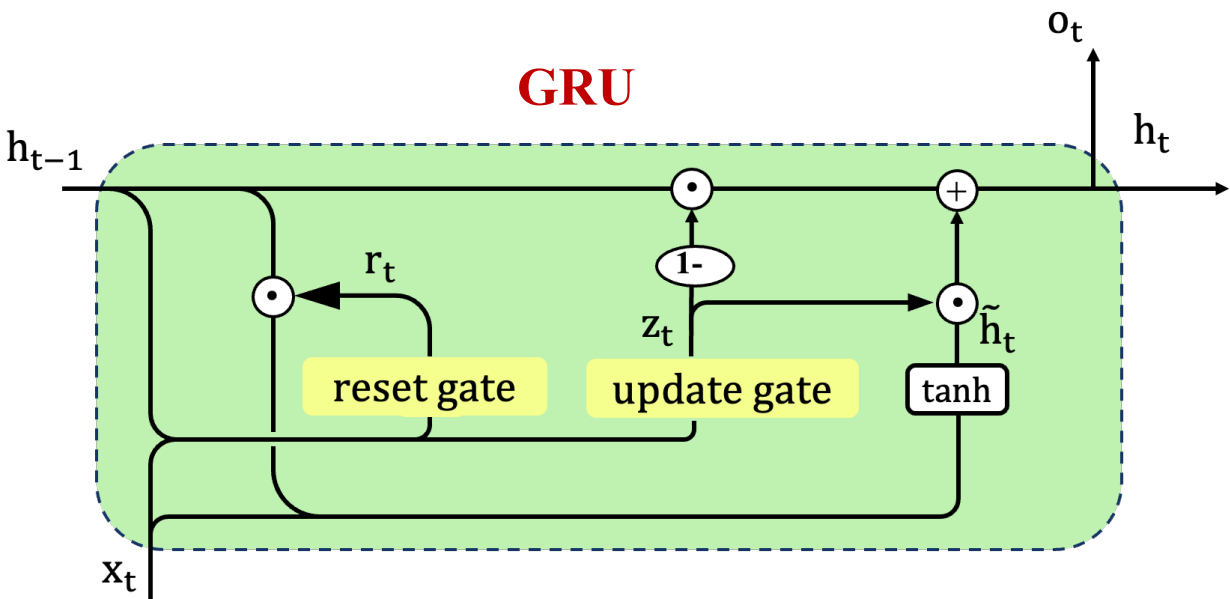
Gated Recurrent Unit

LSTM



GRU has a simpler architecture than **LSTM**, with fewer parameters, which can make it easier to train and more computationally efficient

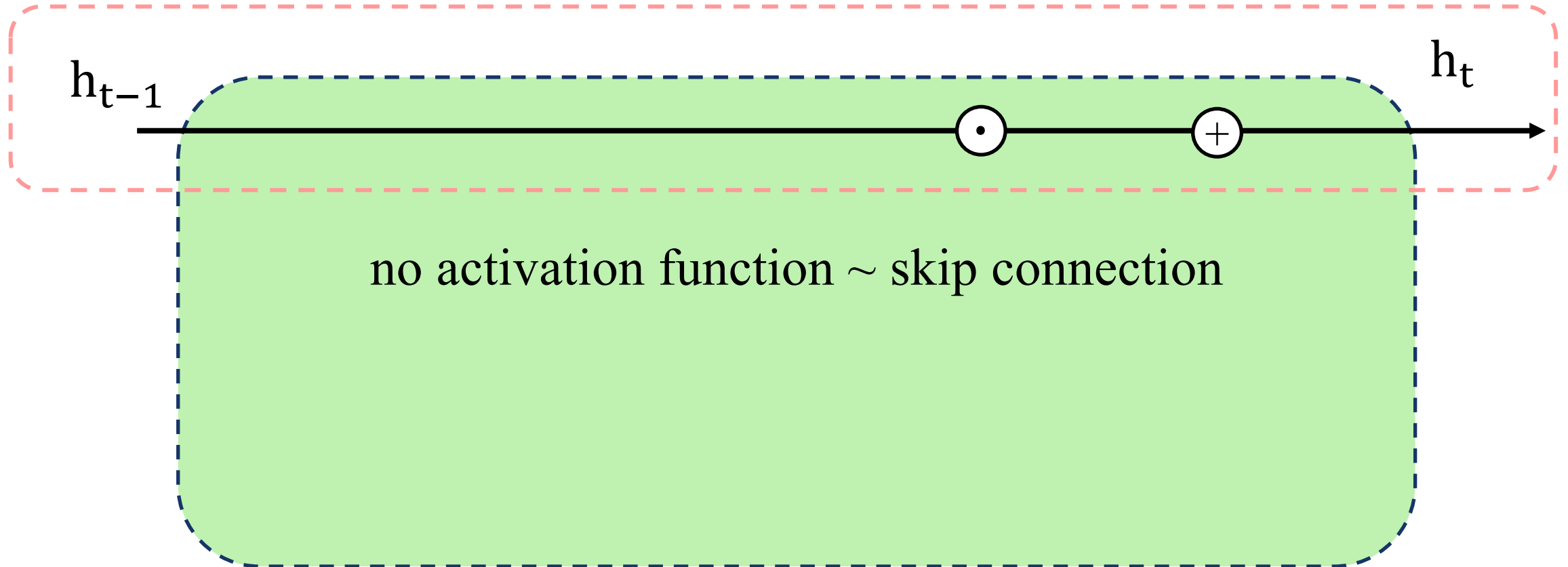
GRU



Gated Recurrent Unit

✗ Cell states

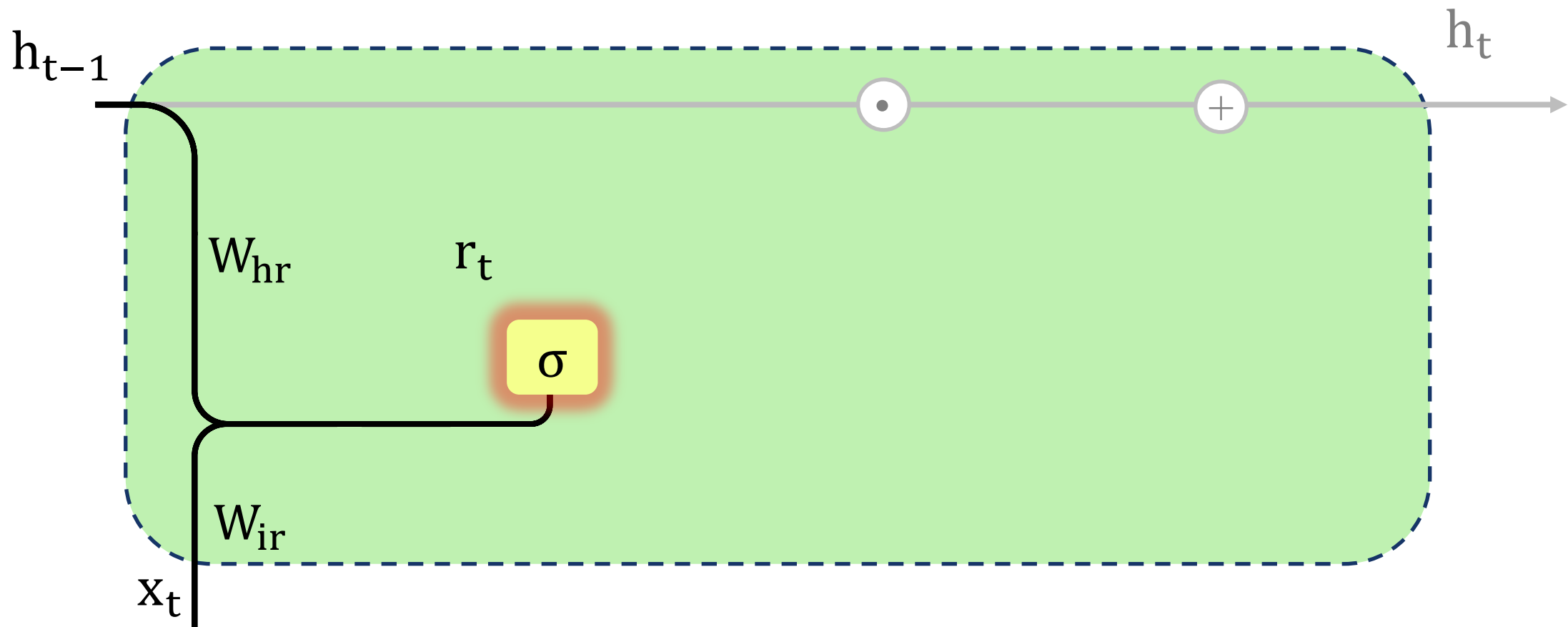
brings information from the past to the future



GRU – Reset Gate

$$r_t = \sigma(W_{ir}x_t + b_{ir} + W_{hr}h_{t-1} + b_{hr})$$

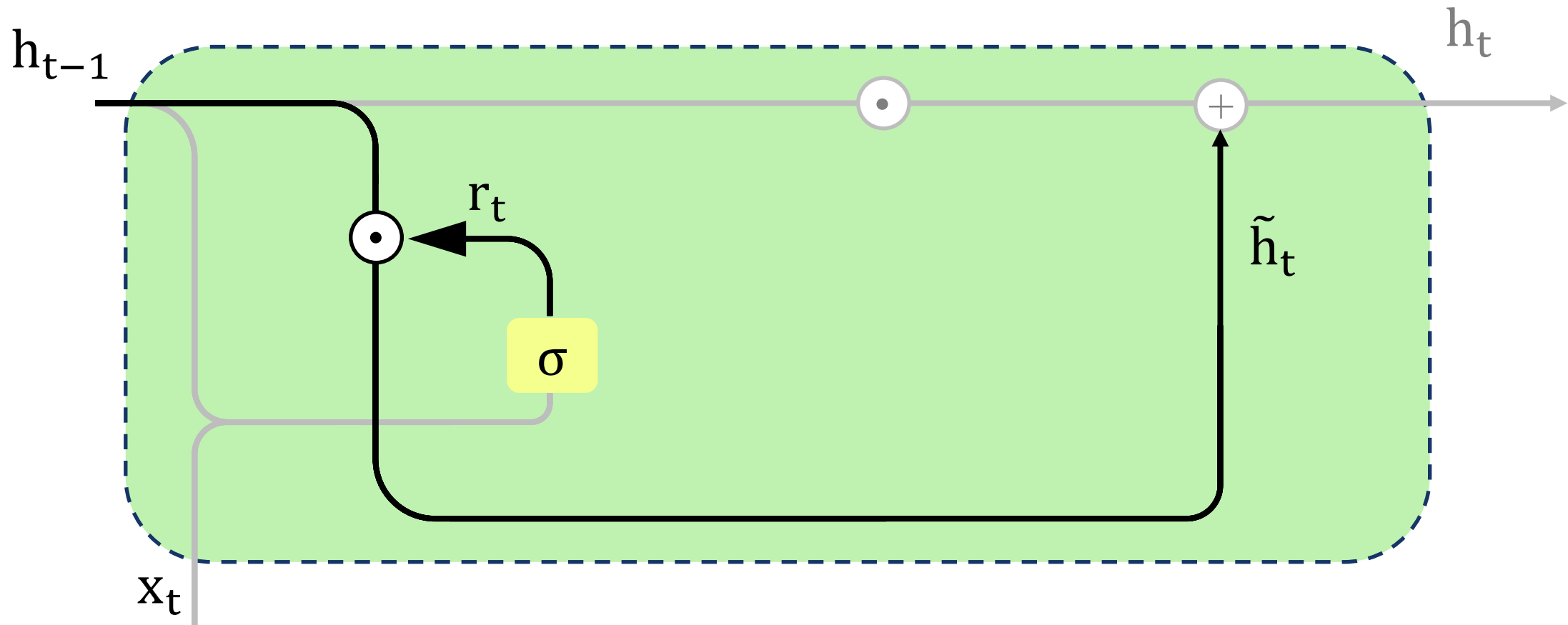
reset gate



Gated Recurrent Unit

$$r_t = \sigma(W_{ir}x_t + b_{ir} + W_{hr}h_{t-1} + b_{hr})$$

$$\tilde{h}_t = \tanh(W_{i\tilde{h}}x_t + W_{h\tilde{h}}(r_t \odot h_{t-1}) + b_{h\tilde{h}})$$

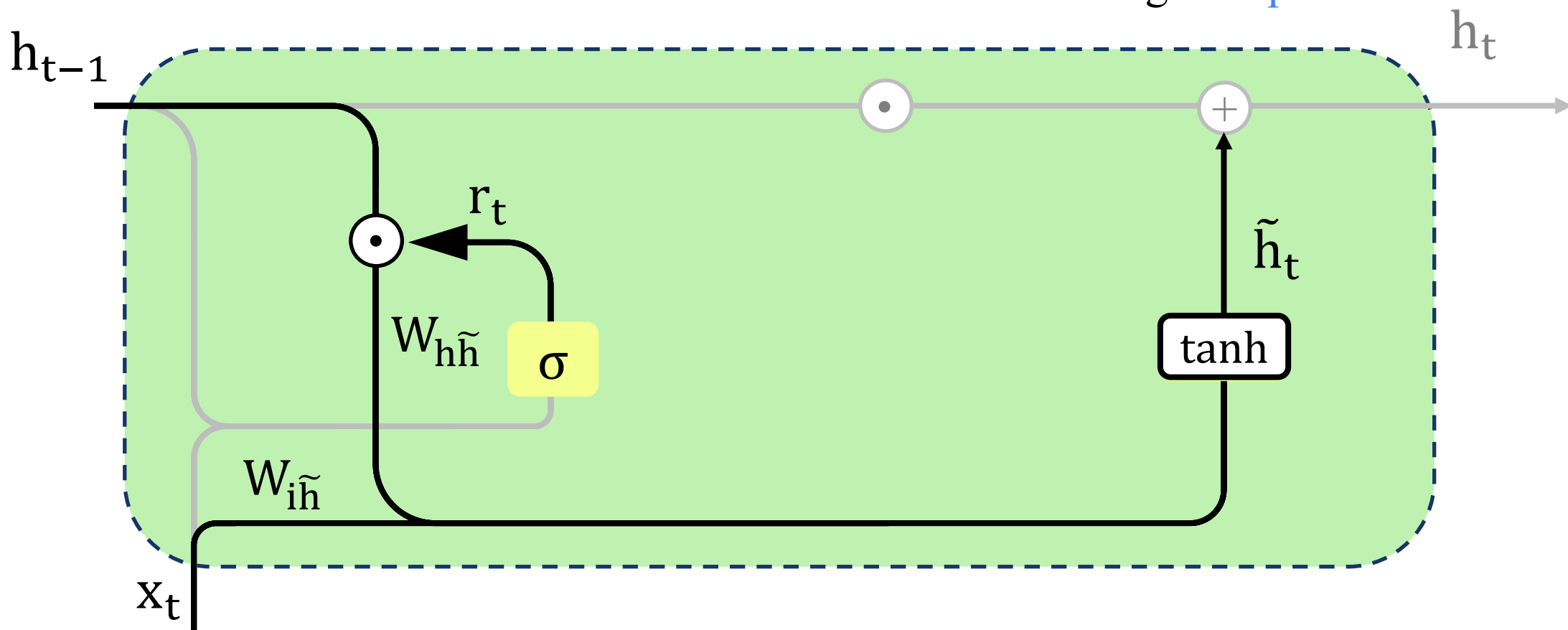


Gated Recurrent Unit

$$r_t = \sigma(W_{ir}x_t + b_{ir} + W_{hr}h_{t-1} + b_{hr})$$

$$\tilde{h}_t = \tanh(W_{i\tilde{h}}x_t + W_{h\tilde{h}}(r_t \odot h_{t-1}) + b_{h\tilde{h}})$$

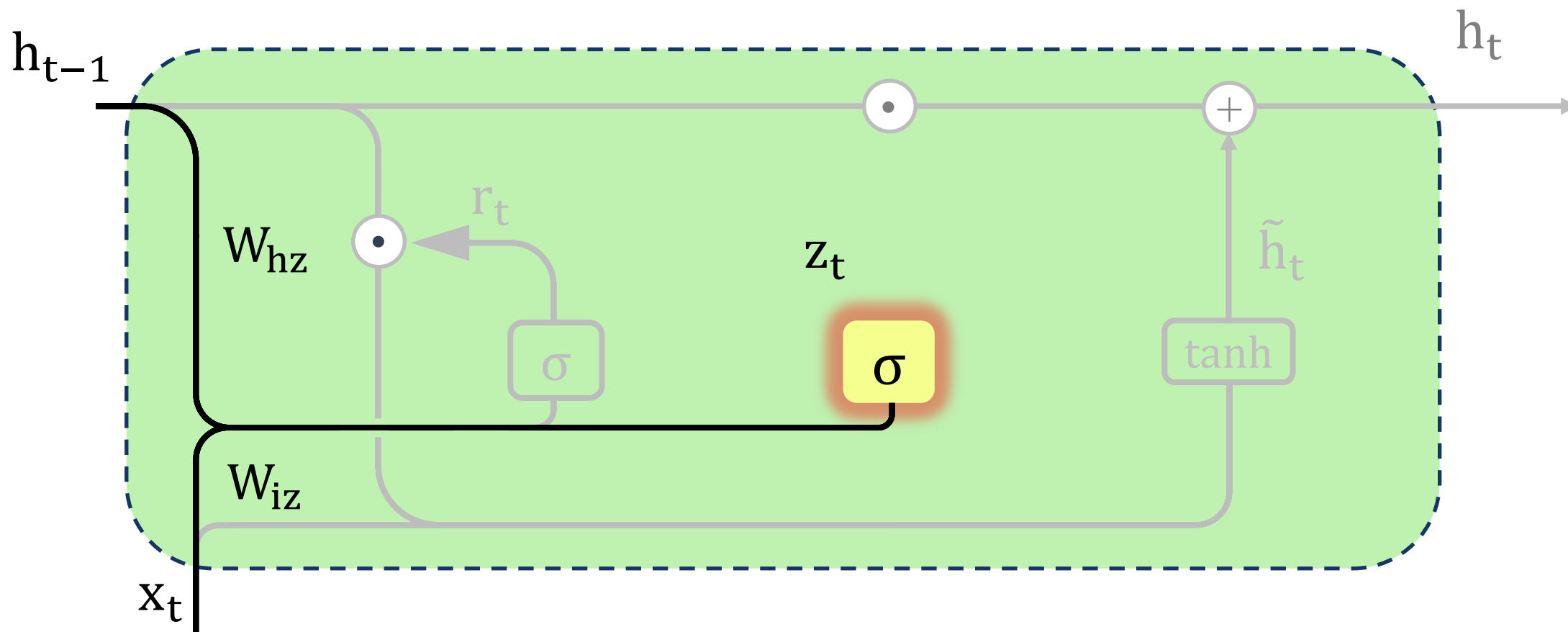
waiting for **update**



Gated Recurrent Unit

$$z_t = \sigma(W_{iz}x_t + b_{iz} + W_{hz}h_{t-1} + b_{hz})$$

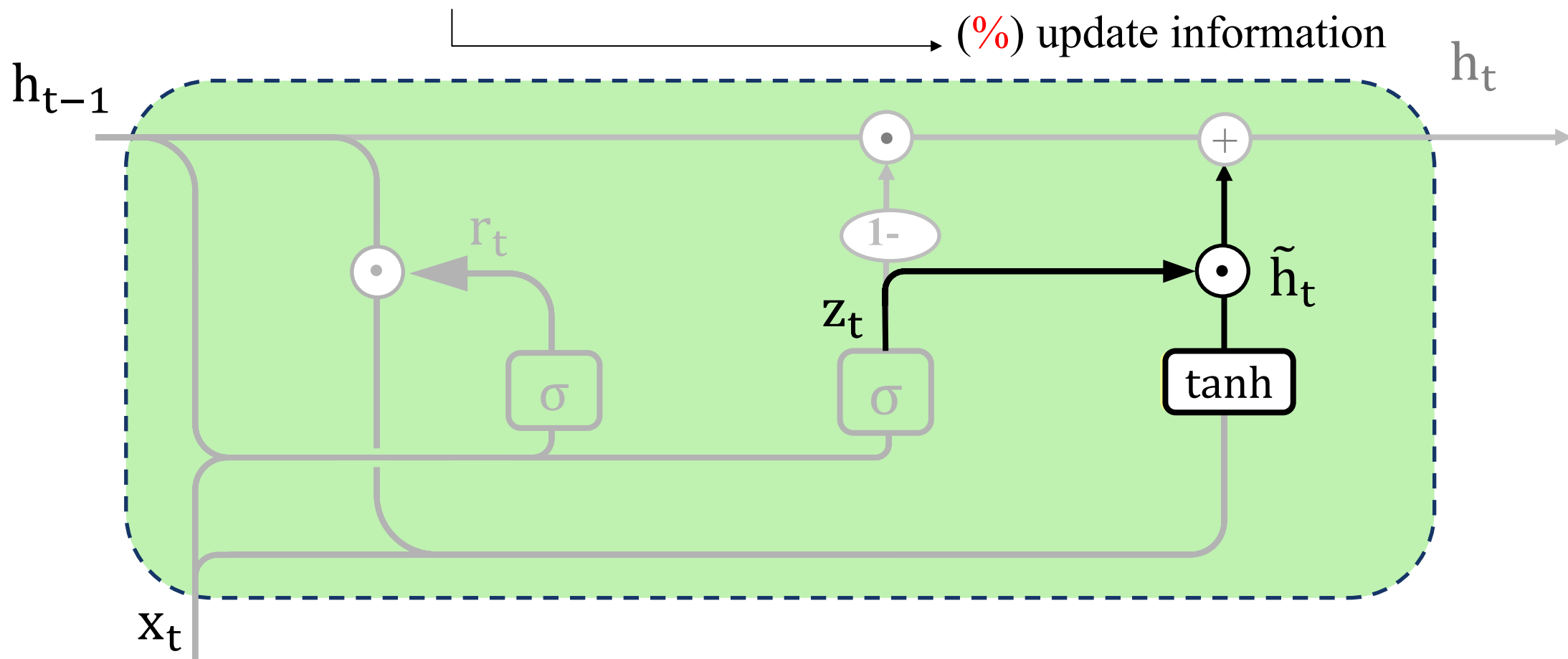
update gate



Gated Recurrent Unit

$$z_t = \sigma(W_{iz}x_t + b_{iz} + W_{hz}h_{t-1} + b_{hz})$$

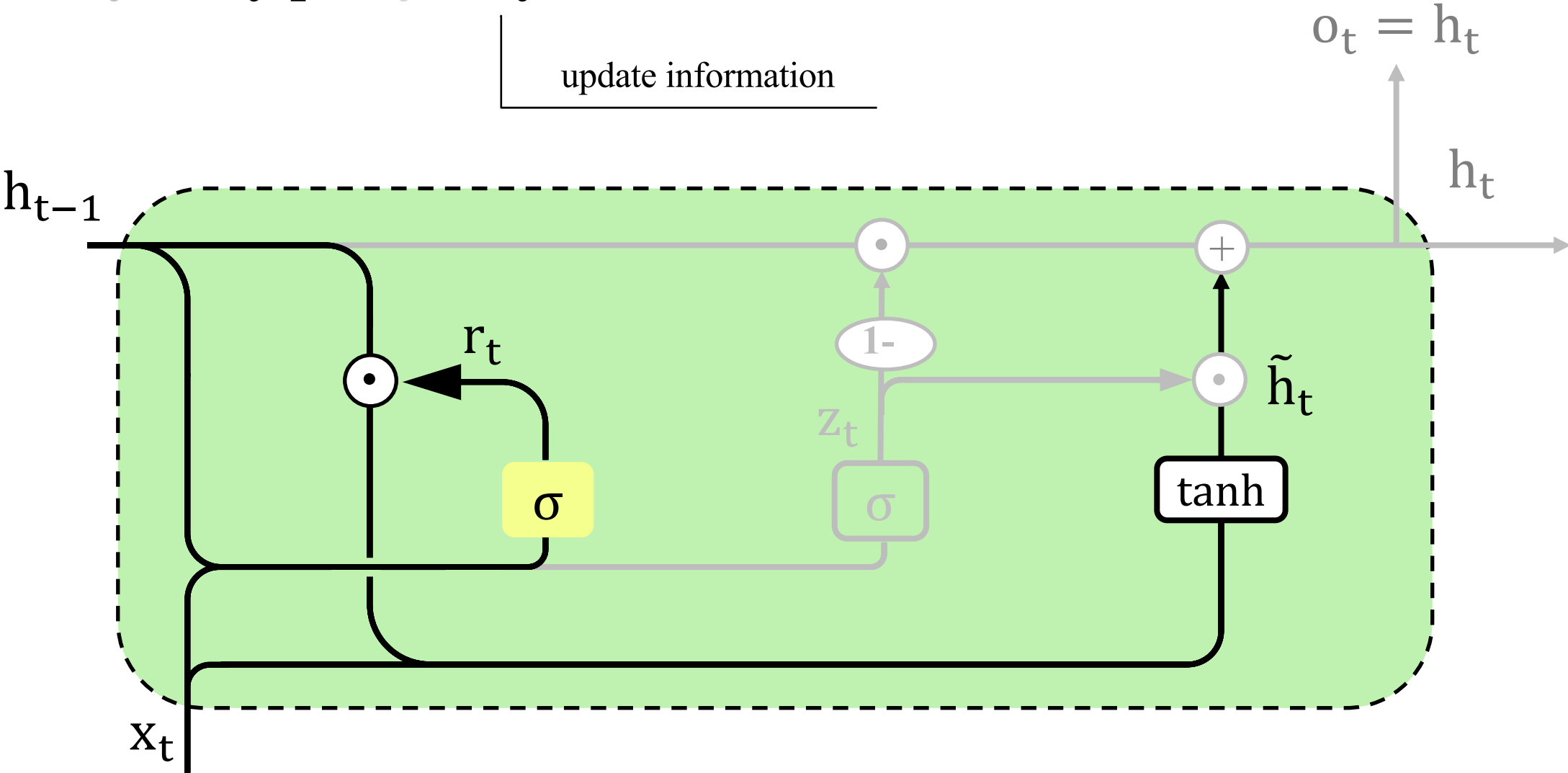
$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$



Gated Recurrent Unit

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

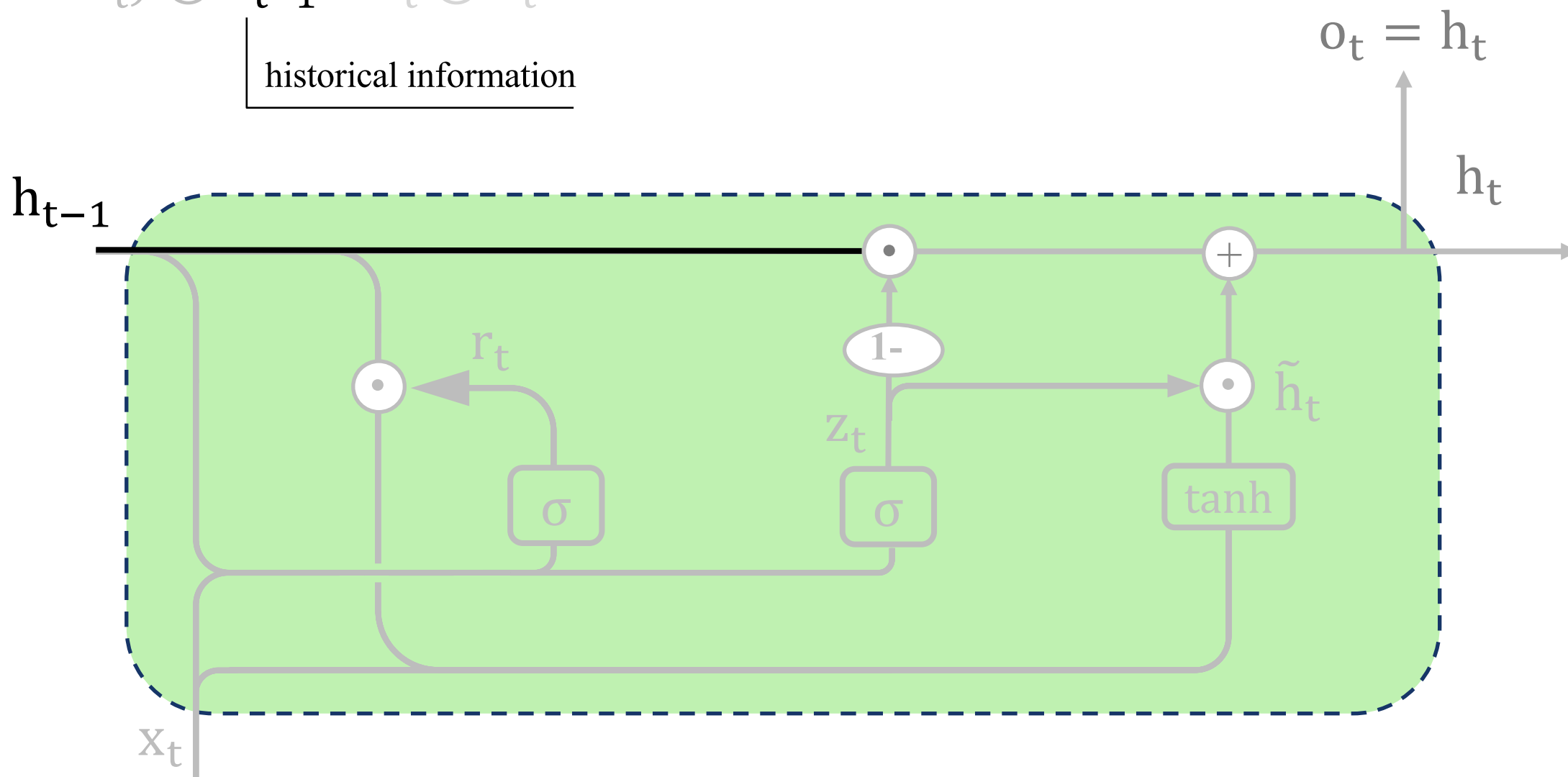
update information



Gated Recurrent Unit

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

historical information

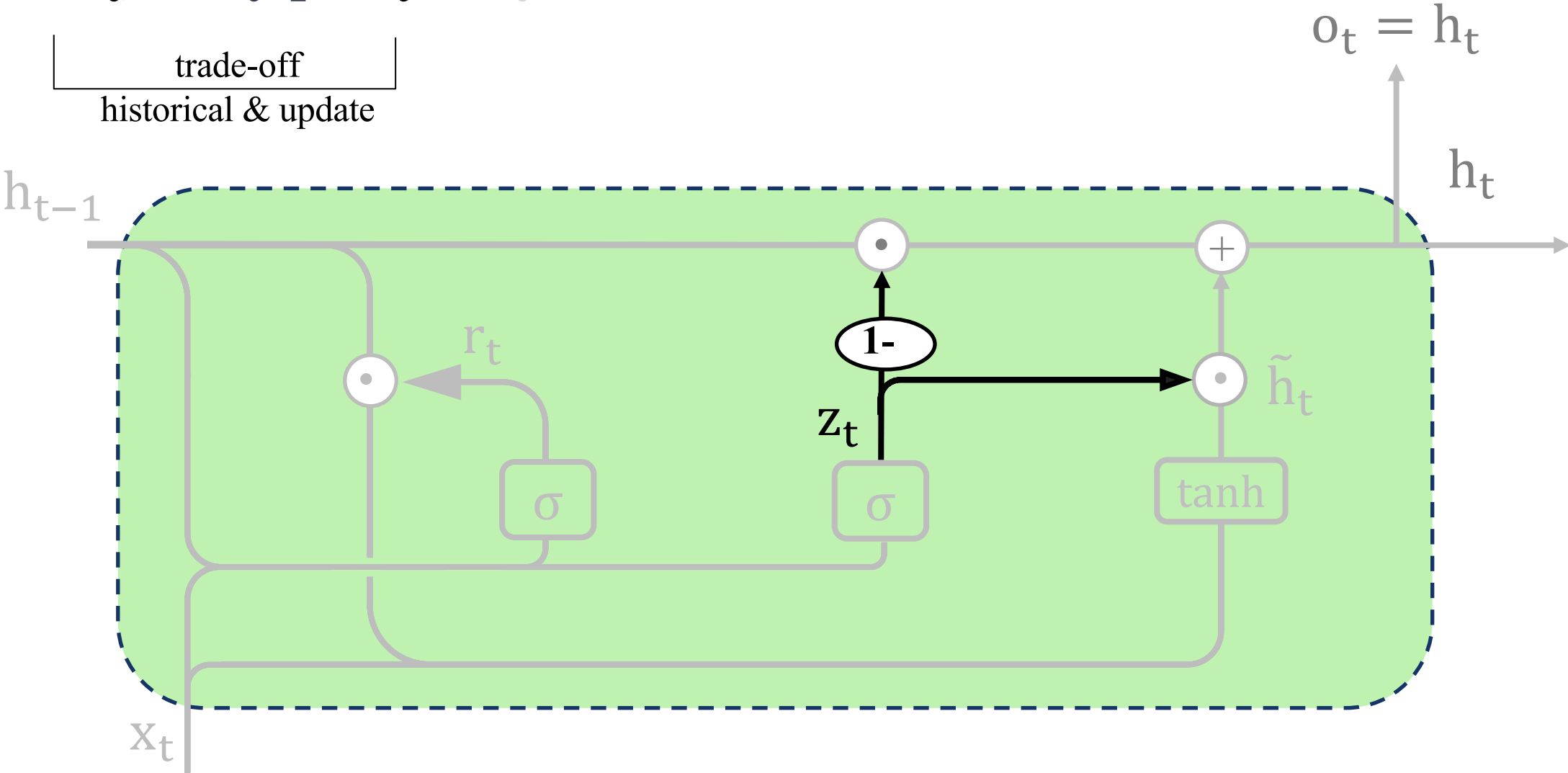


Gated Recurrent Unit

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

trade-off

historical & update



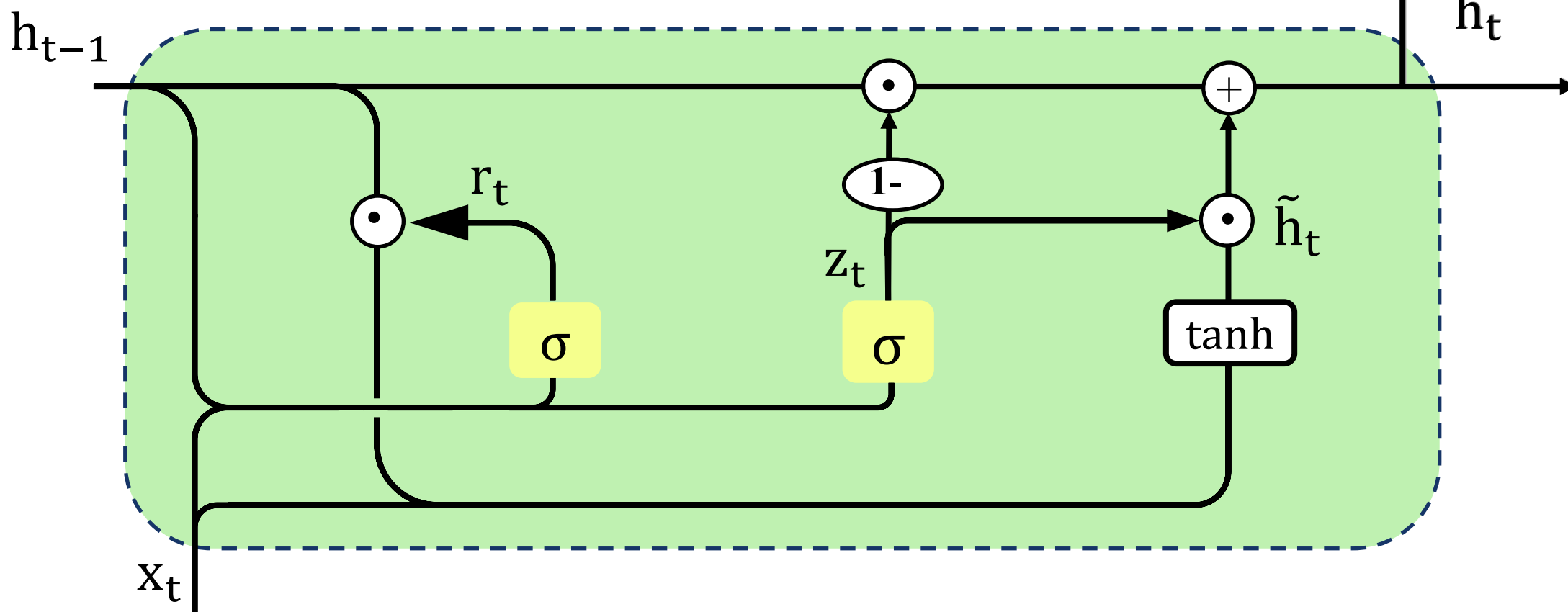
Gated Recurrent Unit

brings all the
information

from the past

to the future

$$o_t = h_t$$



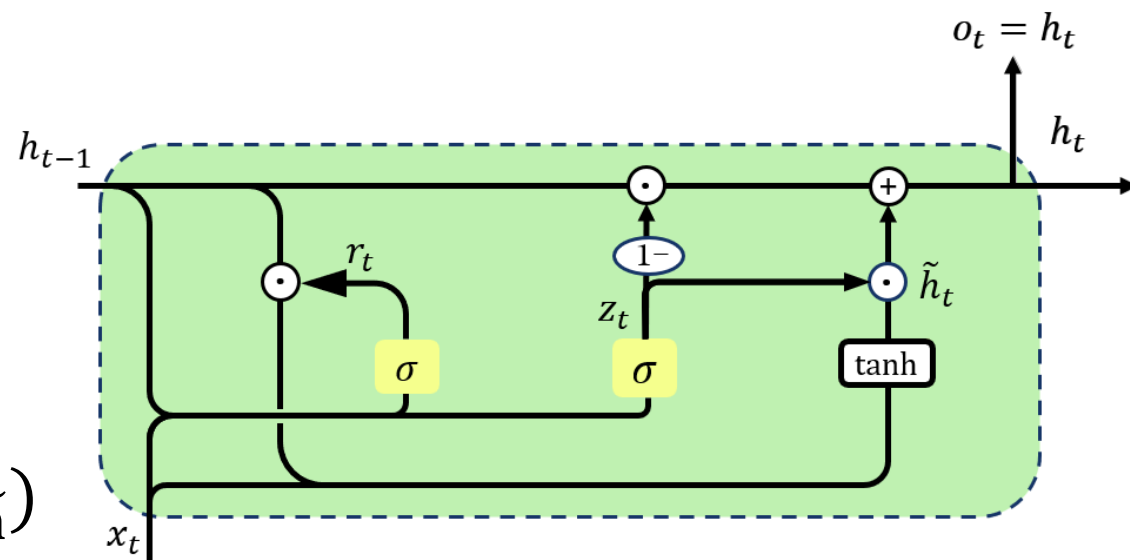
Gated Recurrent Unit

$$r_t = \sigma(W_{ir}x_t + b_{ir} + W_{hr}h_{t-1} + b_{hr})$$

$$z_t = \sigma(W_{iz}x_t + b_{iz} + W_{hz}h_{t-1} + b_{hz})$$

$$\tilde{h}_t = \tanh(W_{i\tilde{h}}x_t + W_{h\tilde{h}}(r_t \odot h_{t-1}) + b_{h\tilde{h}})$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$



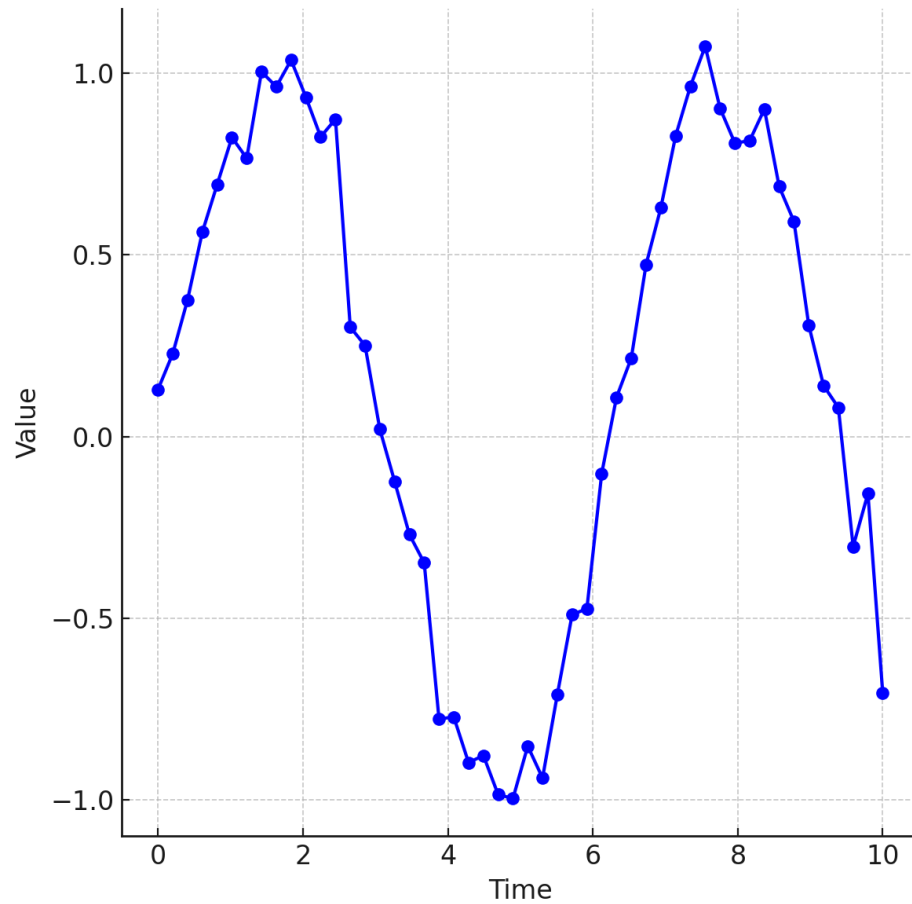
Outline

- **Review: Recurrent Neural Network (RNN)**
- **Long-Short Term Memory (LSTM)**
- **Gated Recurrent Unit (GRU)**
- **Time Series – Basic Knowledge**
- **LSTM, GRU for Time Series Data**
- **LSTM, GRU for Text Data**



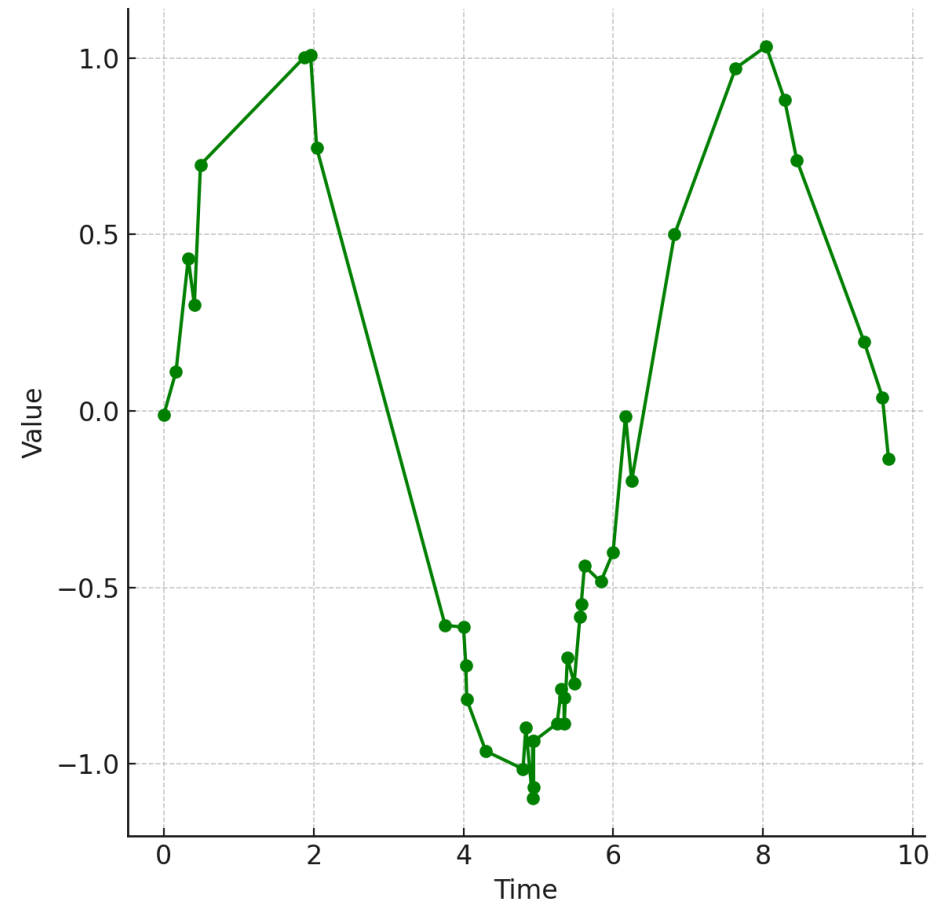
Time Series - Basic Knowledge

Evenly Sampled



Spacing of times is equal

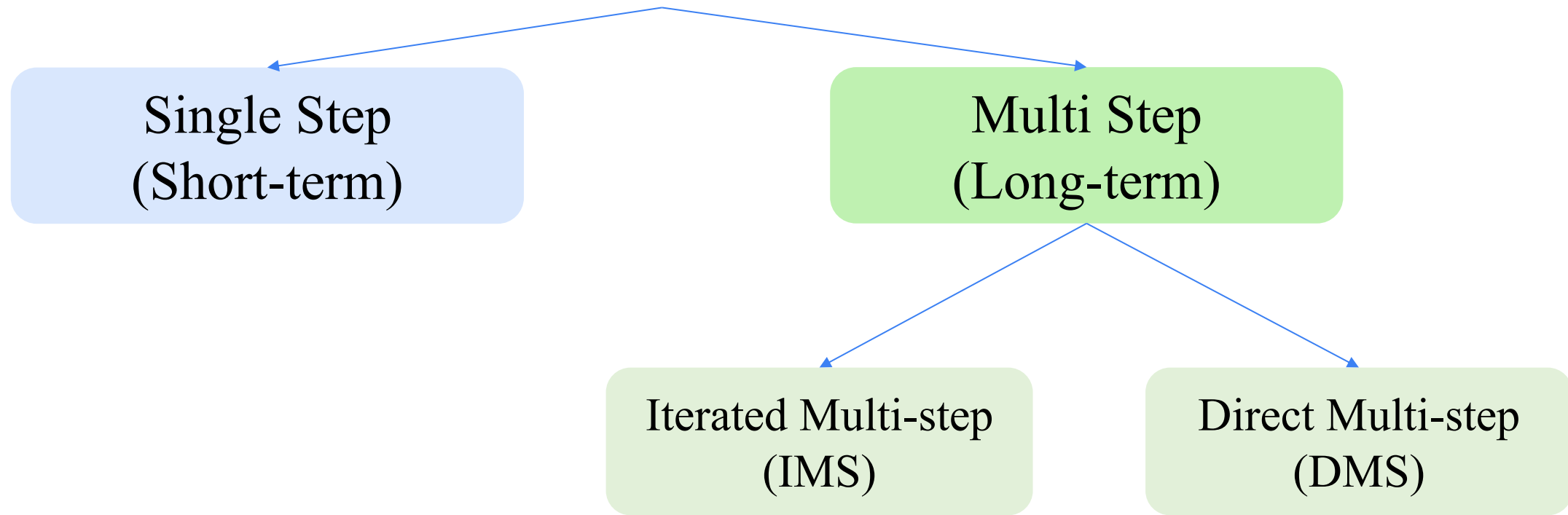
Unevenly Sampled



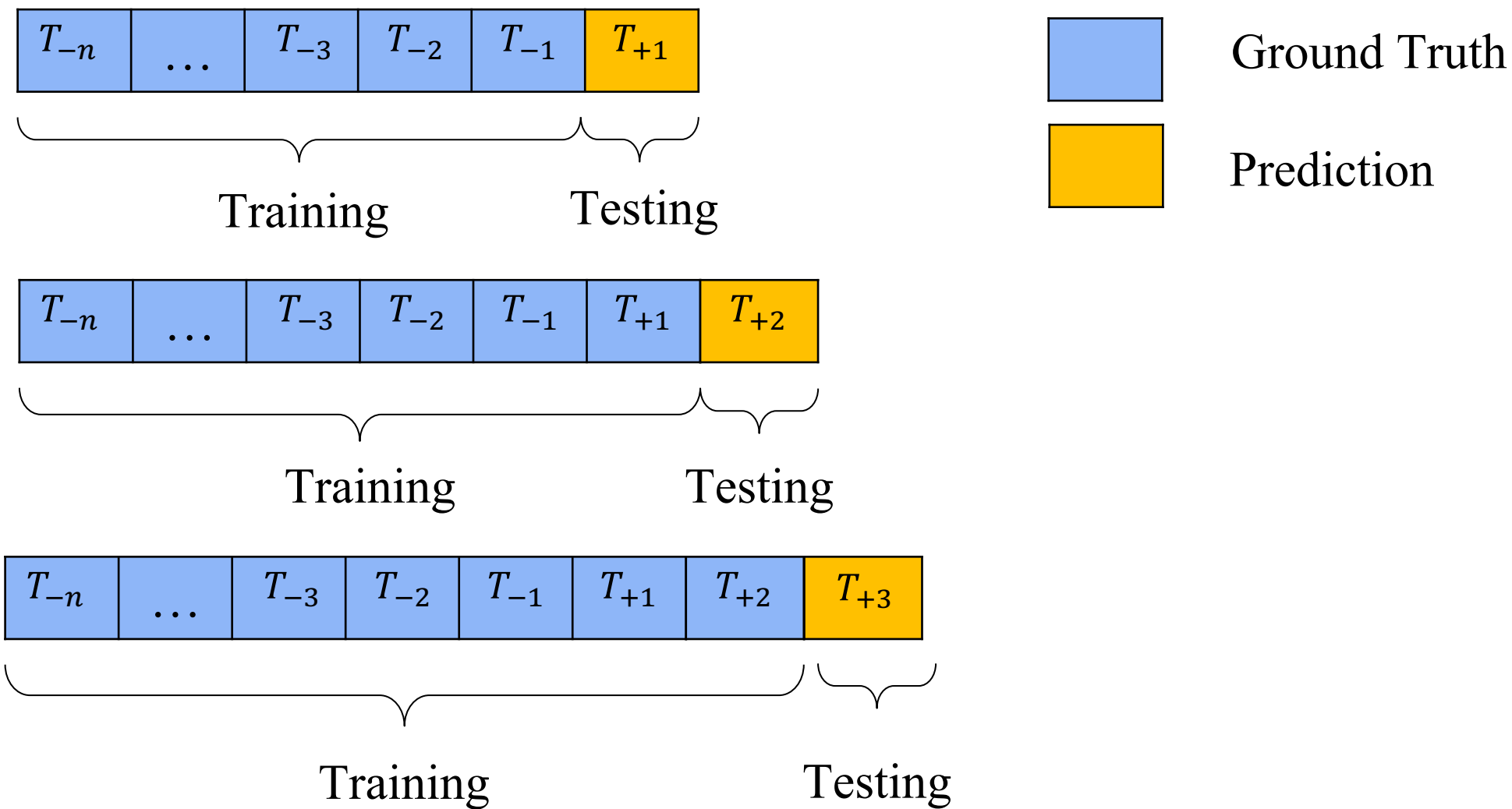
Spacing of times is not equal

Time Series - Basic Knowledge

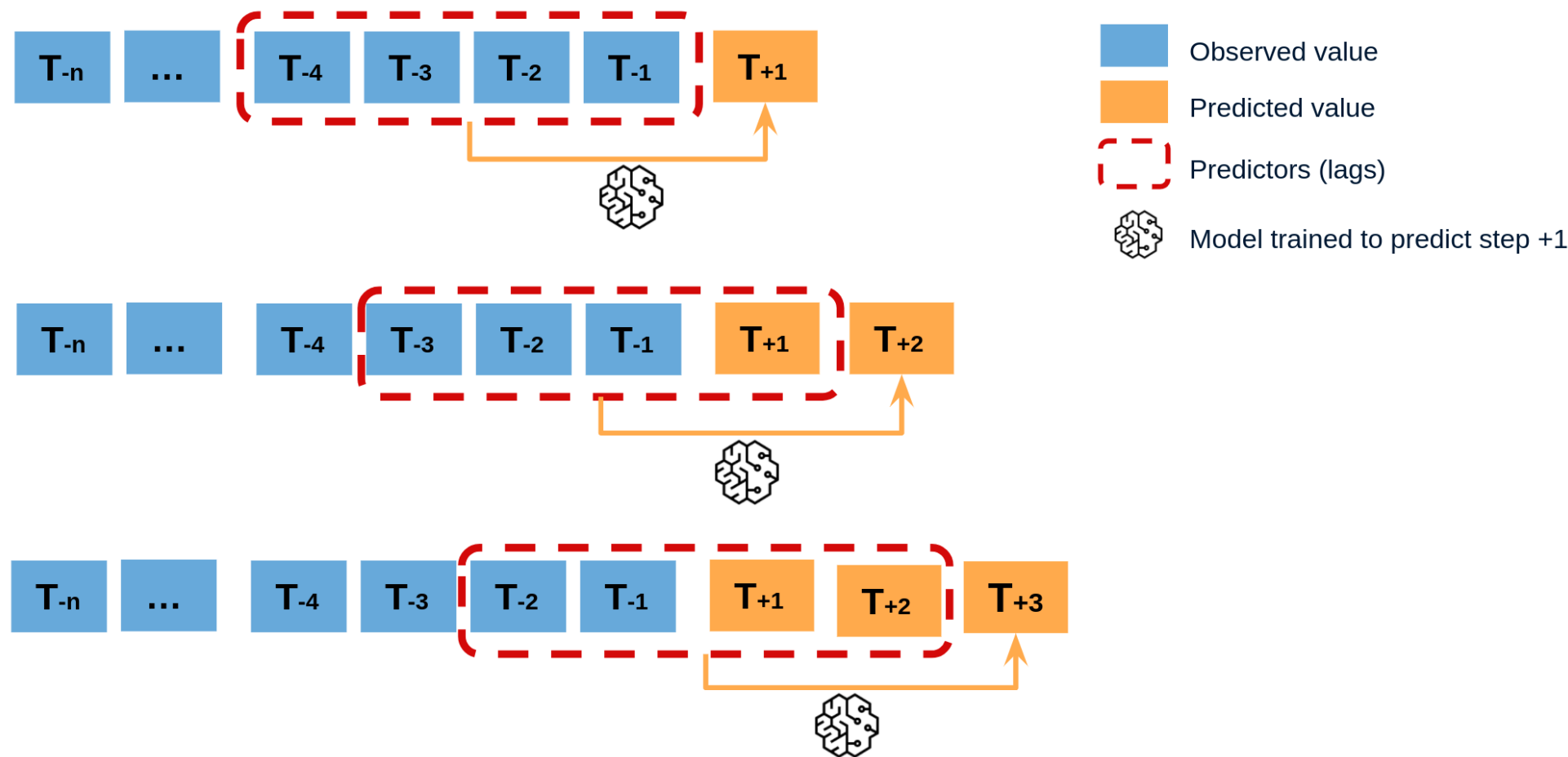
Forecasting



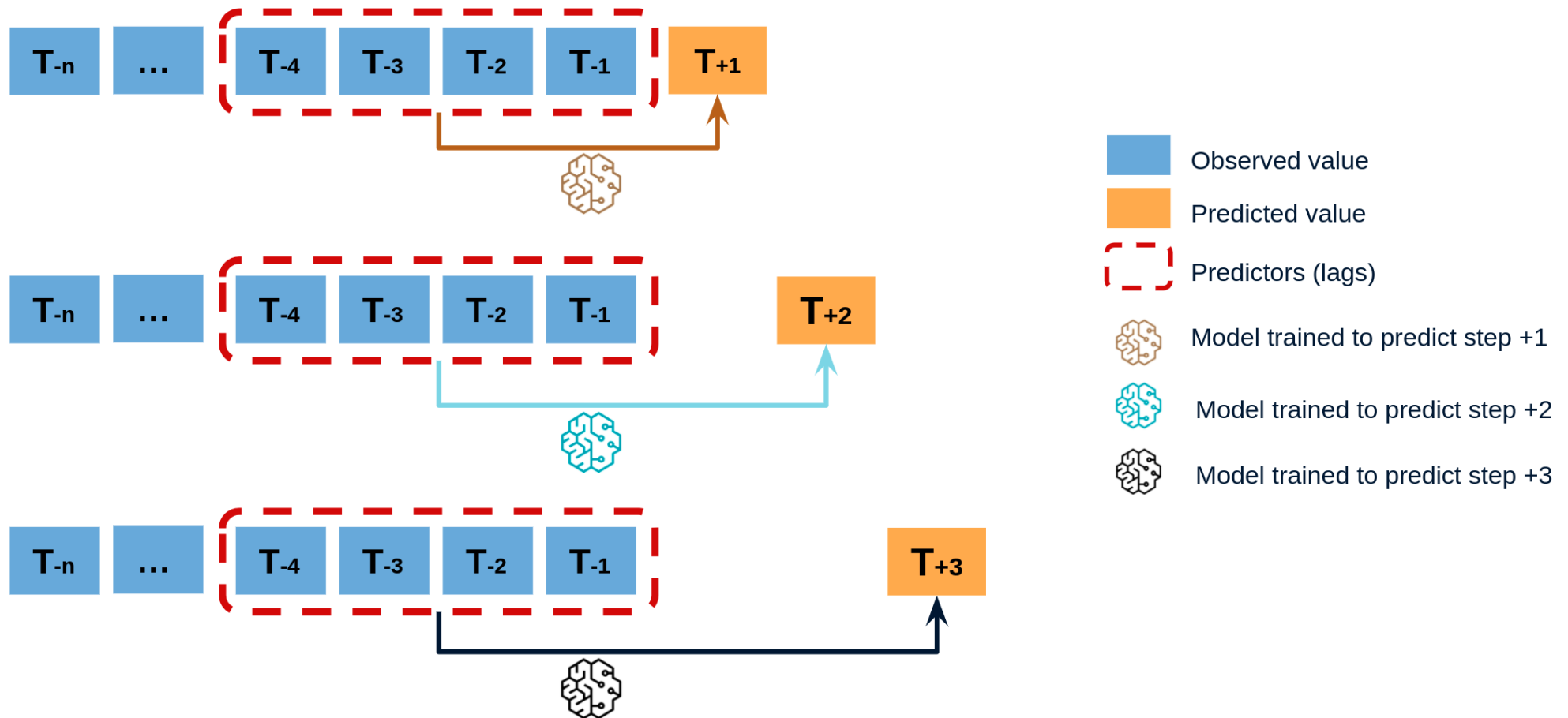
Single-step Forecasting



Iterated Multi-step Forecasting



Direct Multi-step Forecasting

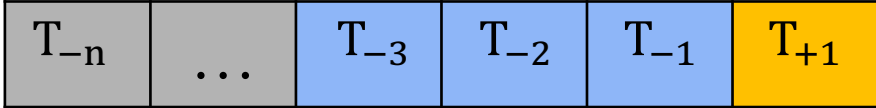


Size of Training data

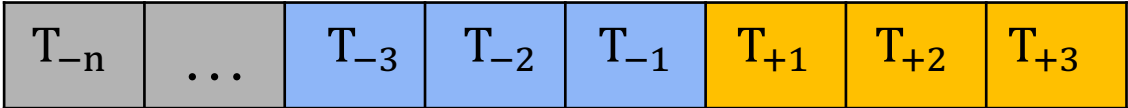
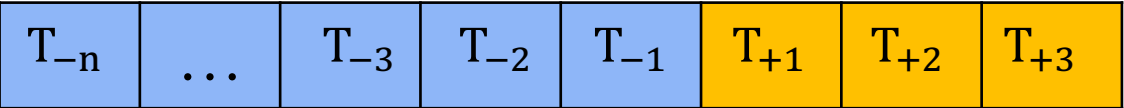
Full Size

Window Size

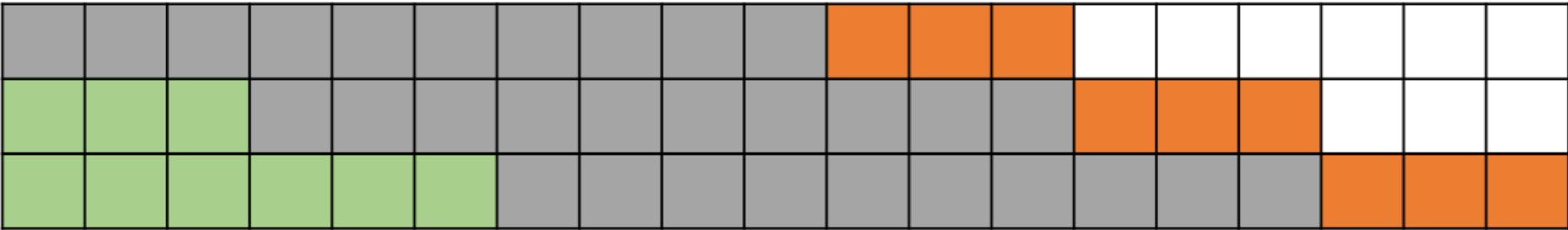
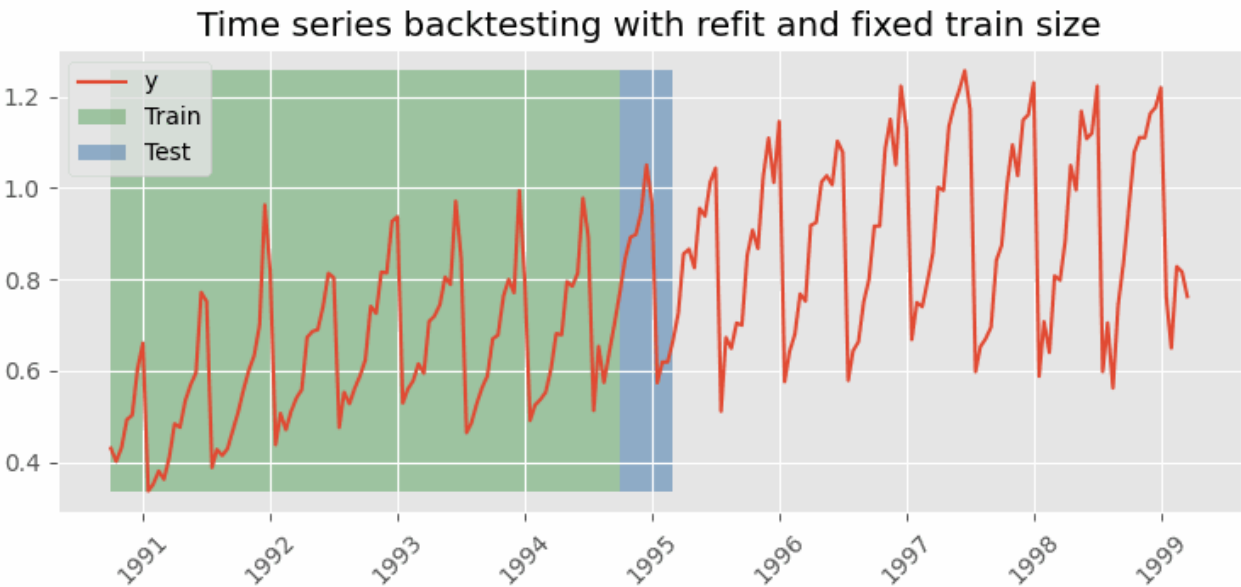
Single-step



Multi-step

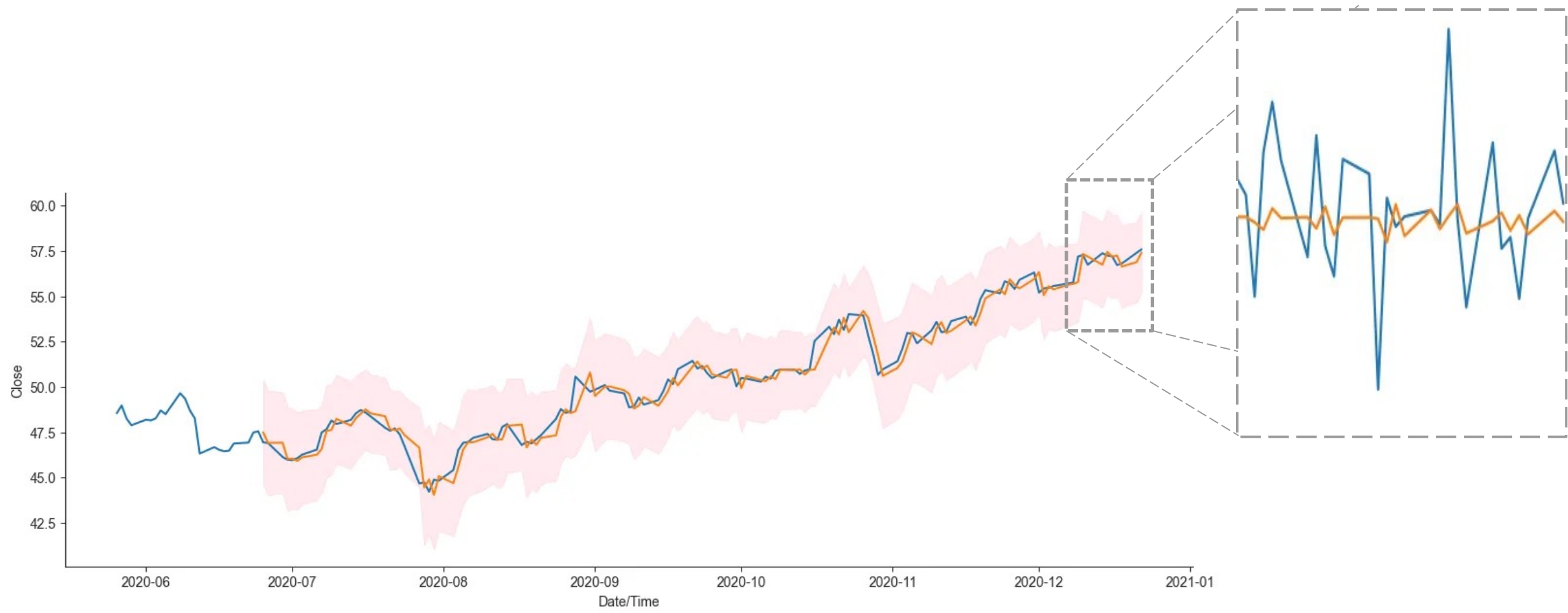


Rolling window (Cross-validation)

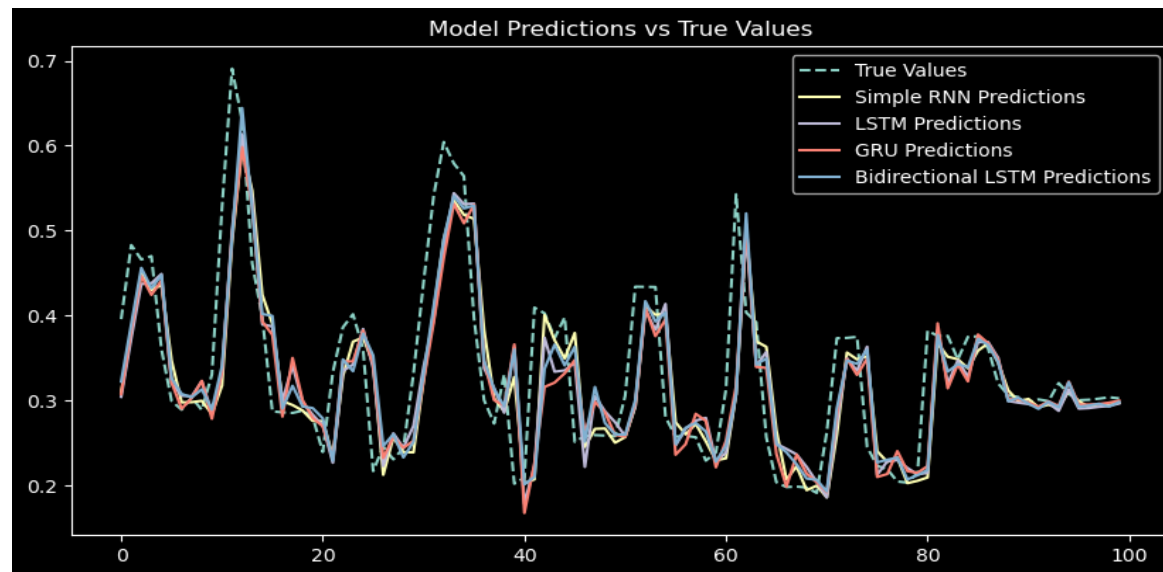


Training set
Test set
Unseen data
Seen data but not used for training

Single-step Forecasting



Single-step Forecasting – Rolling Window



Advantages

1. Simplicity
2. High Accuracy
3. Easy to Interpret
4. Effective for Short-Term

Drawbacks

1. Ignores Long-Term Patterns (trends, periodicity,...)
2. Difficulty in Choosing the Window Size

Outline

- **Review: Recurrent Neural Network (RNN)**
- **Long-Short Term Memory (LSTM)**
- **Gated Recurrent Unit (GRU)**
- **Time Series – Basic Knowledge**
- **LSTM, GRU for Time Series Data**
- **LSTM, GRU for Text Data**



Household Electric Power Consumption

Data Time	Sub Meter 1	Sub Meter 2	Sub Meter 3	Global Power
2006-12-17 10:23:00	1	7	17	4.216
2006-12-17 10:24:00	2	35	17	5.36
2006-12-17 10:25:00	1	26	17	5.374

...

Meter located in kitchen, laundry room and heater systems. (Wat)

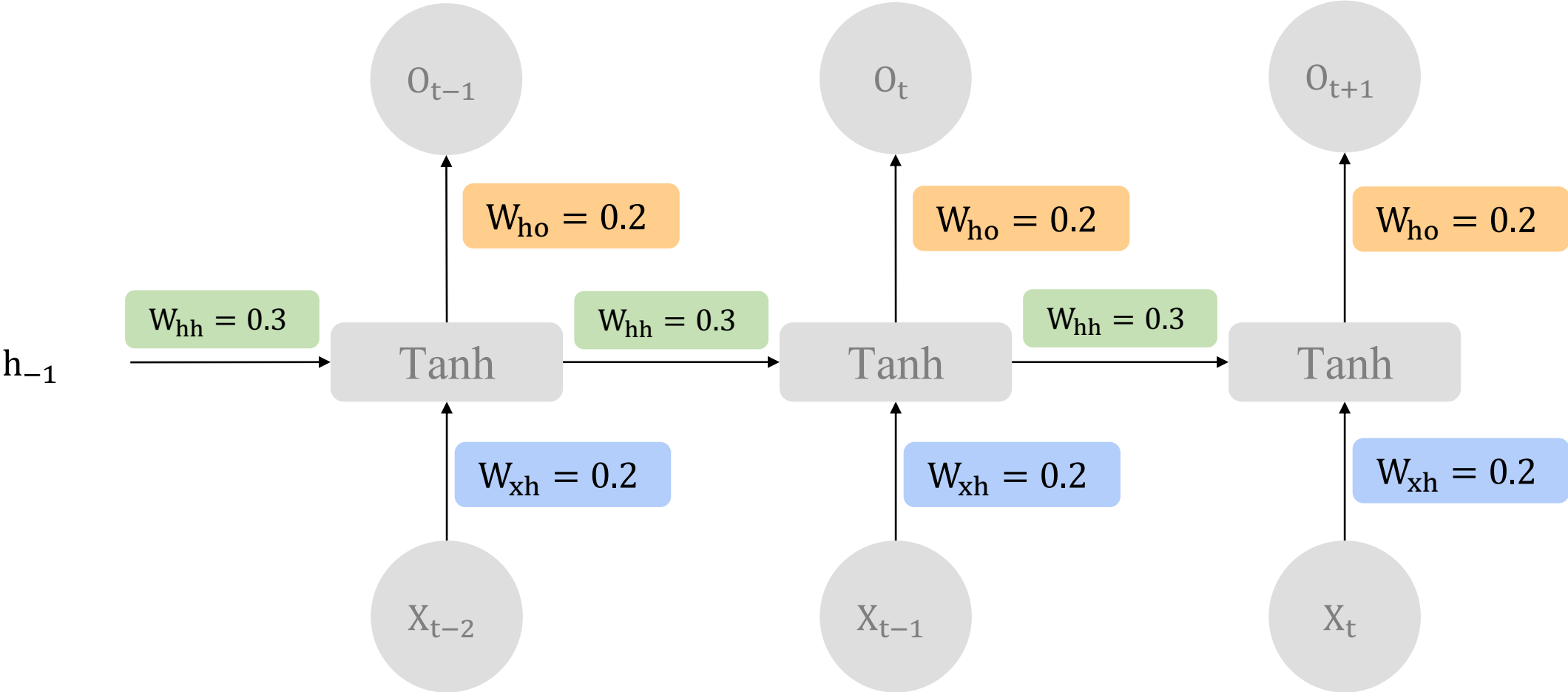
Average household energy usage. (Kilowat)

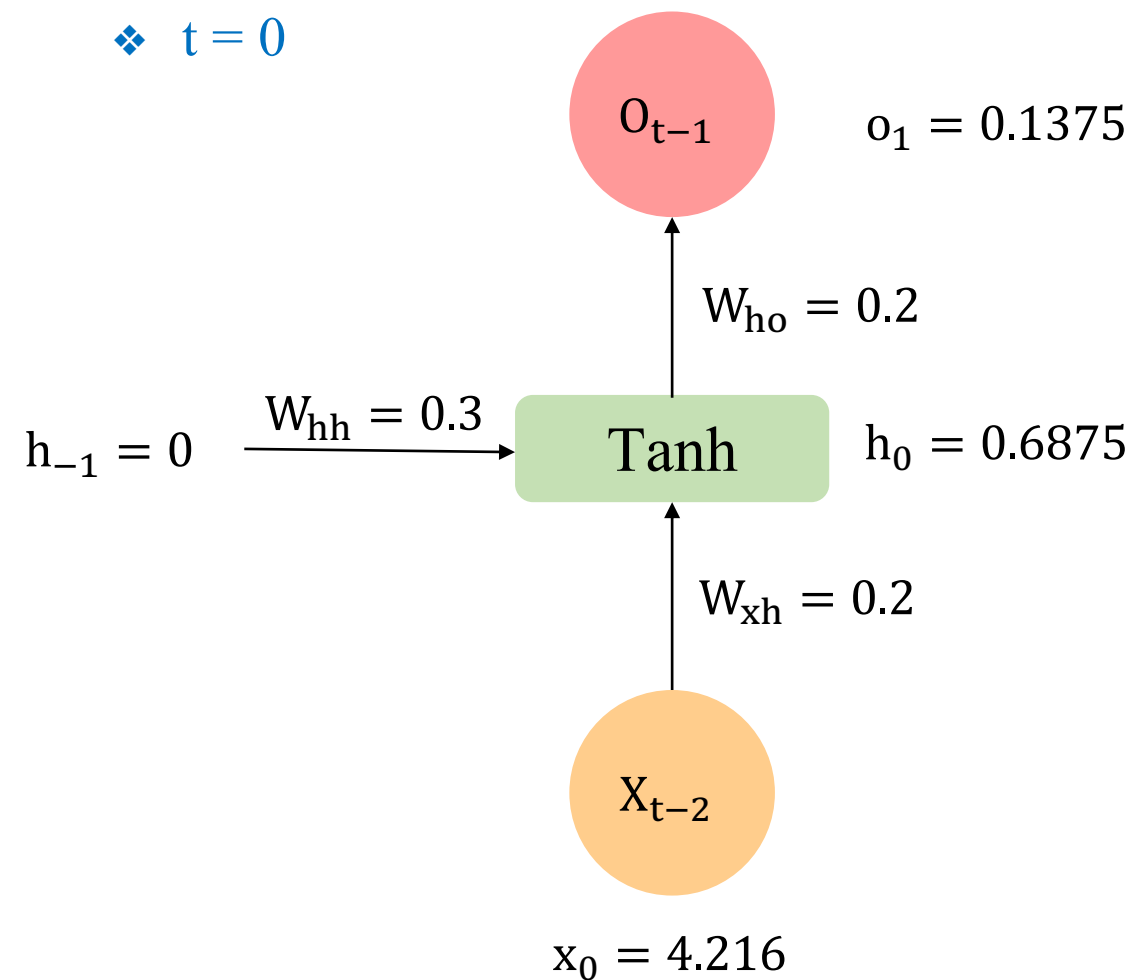
Data Time	Sub Meter 1	Sub Meter 2	Sub Meter 3	Global Power
2006-12-17 10:23:00	1	7	17	4.216
2006-12-17 10:24:00	2	35	17	5.36
2006-12-17 10:25:00	1	26	17	5.374

- Univariate
- Single-step
- One to one prediction

```
import torch

W_xh, W_hh, W_ho = 0.2, 0.3, 0.2
```



❖ $t = 0$ 

```
x = torch.tensor(4.216)
h = torch.tensor(0)
```

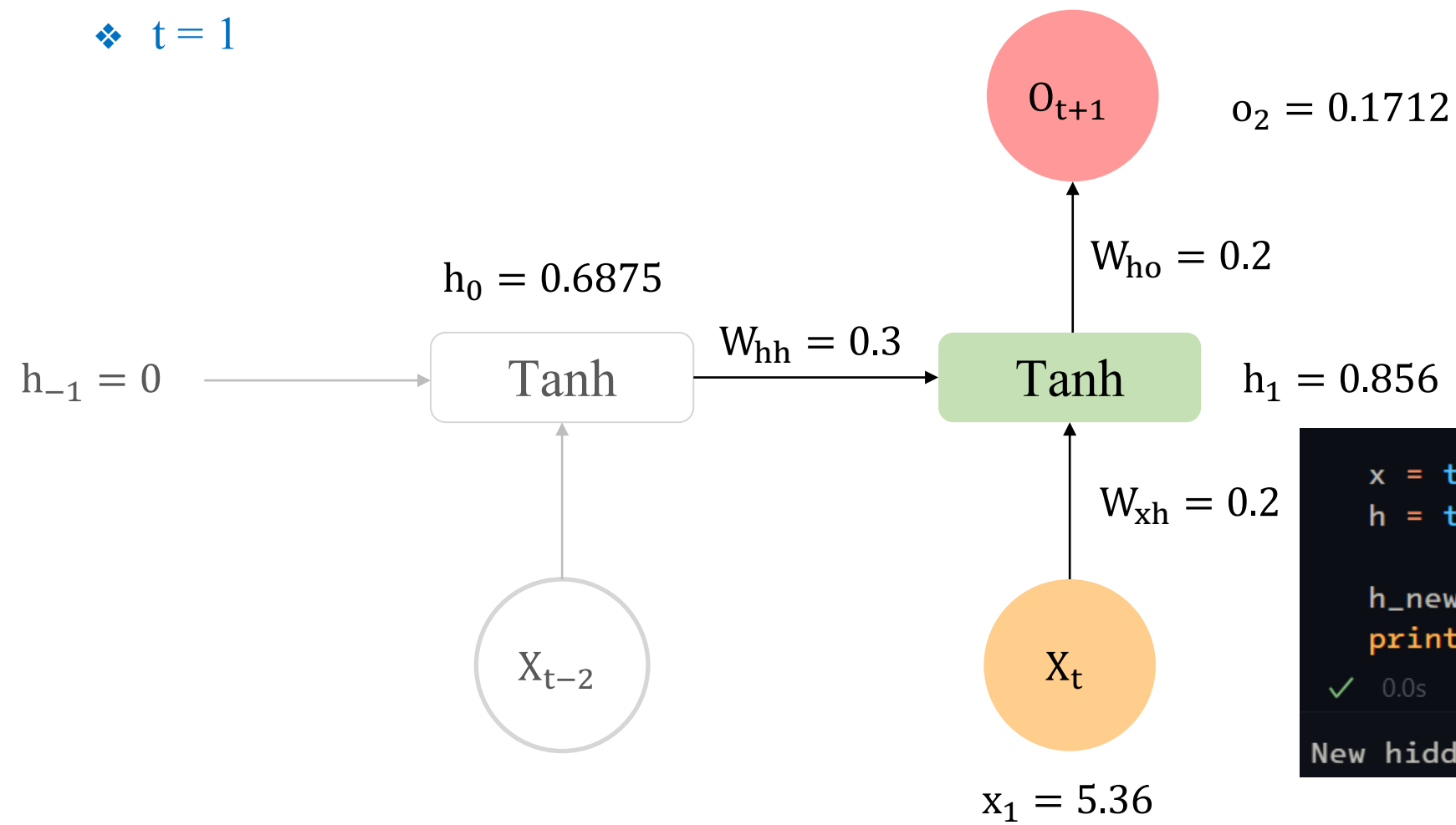
```
h_new = torch.tanh(W_xh * x + W_hh * h)
print("New hidden state:", h_new)
```

```
o = W_ho * h_new
print("Output:", o)
```

✓ 0.0s

```
New hidden state: tensor(0.6875)
Output: tensor(0.1375)
```

❖ $t = 1$



```
o = W_ho * h_new
print("Output:", o)
✓ 0.0s
Output: tensor(0.1712)
```

```
x = torch.tensor(5.36)
h = torch.tensor(0.6875)

h_new = torch.tanh(W_xh * x + W_hh * h)
print("New hidden state:", h_new)
✓ 0.0s
New hidden state: tensor(0.8560)
```

❖ $t = 2$

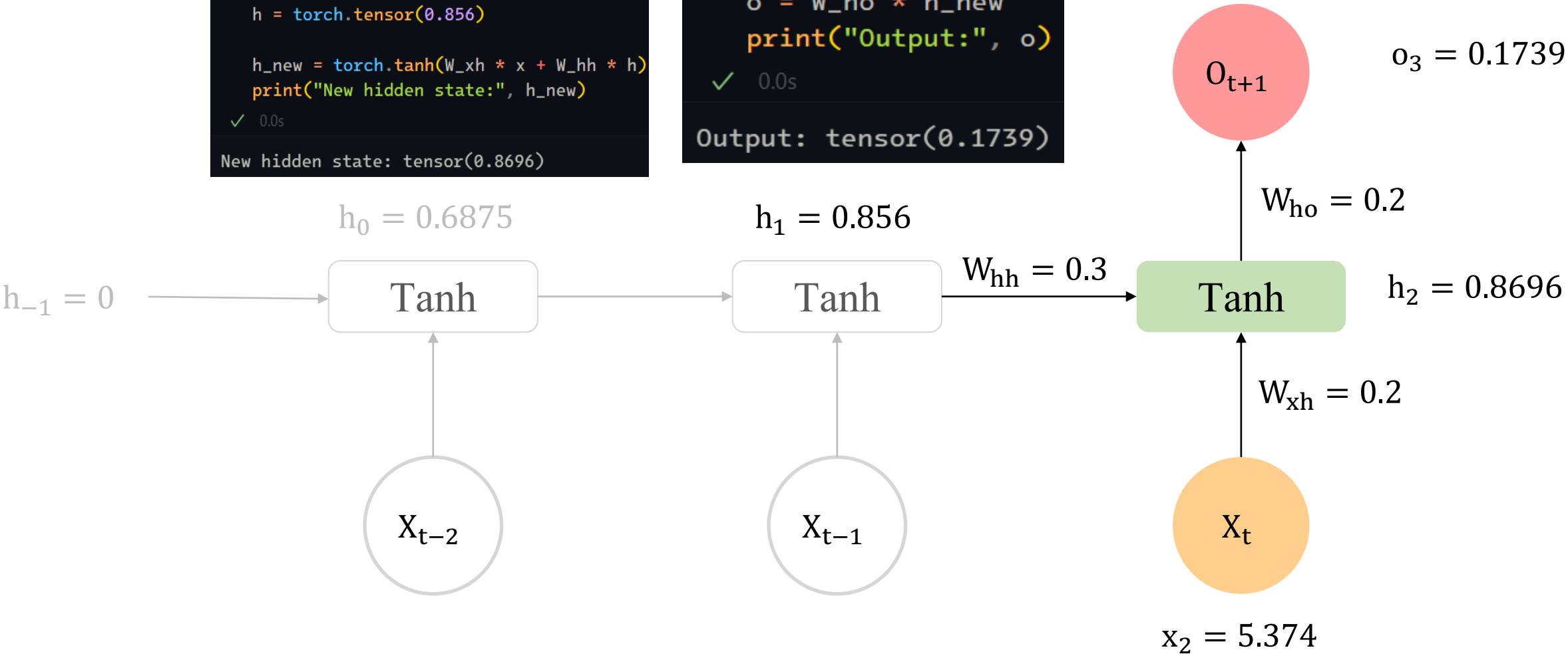
```
x = torch.tensor(5.374)
h = torch.tensor(0.856)

h_new = torch.tanh(W_xh * x + W_hh * h)
print("New hidden state:", h_new)

✓ 0.0s
New hidden state: tensor(0.8696)
```

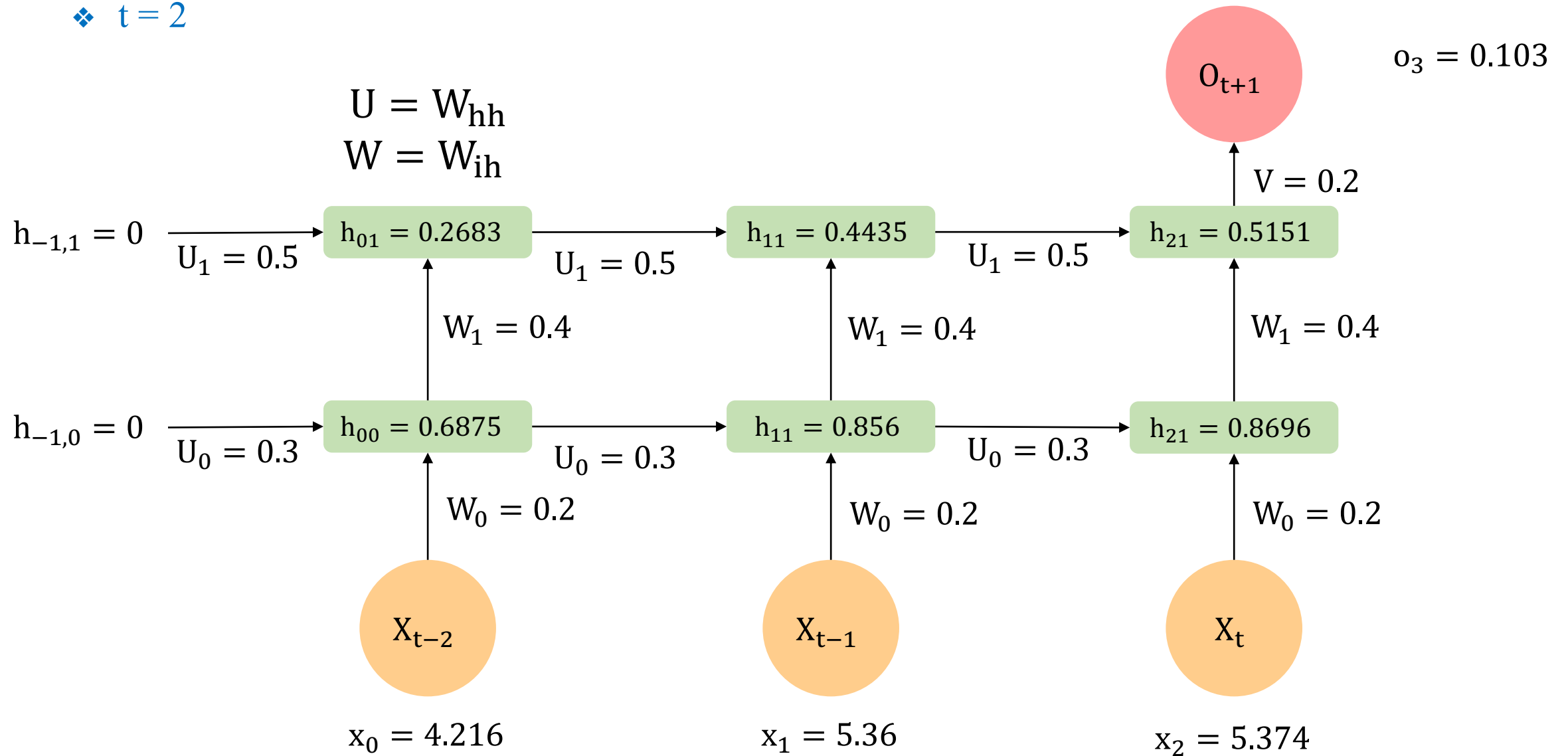
```
o = W_ho * h_new
print("Output:", o)

✓ 0.0s
Output: tensor(0.1739)
```



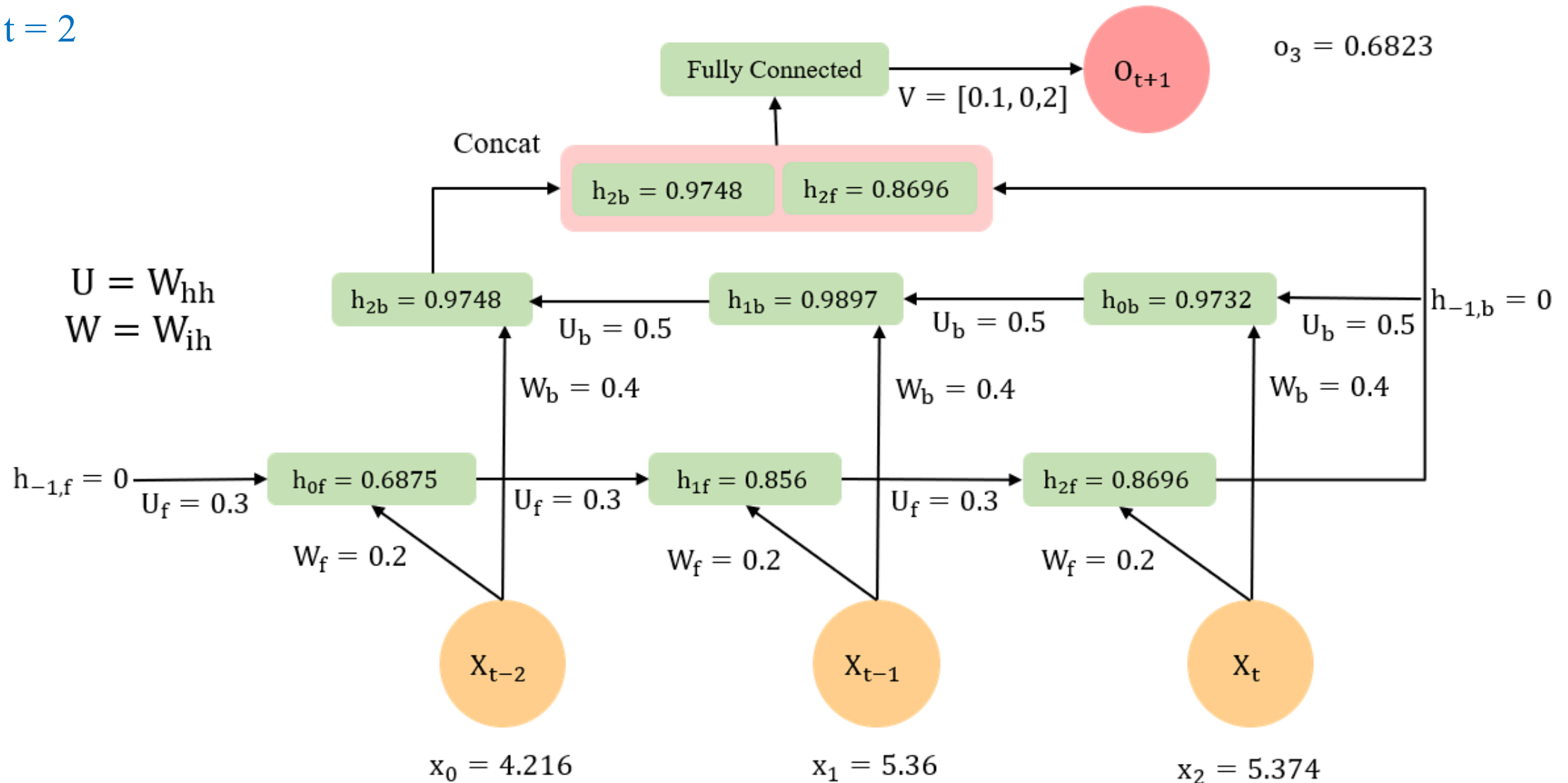
Stacked RNN

❖ $t = 2$



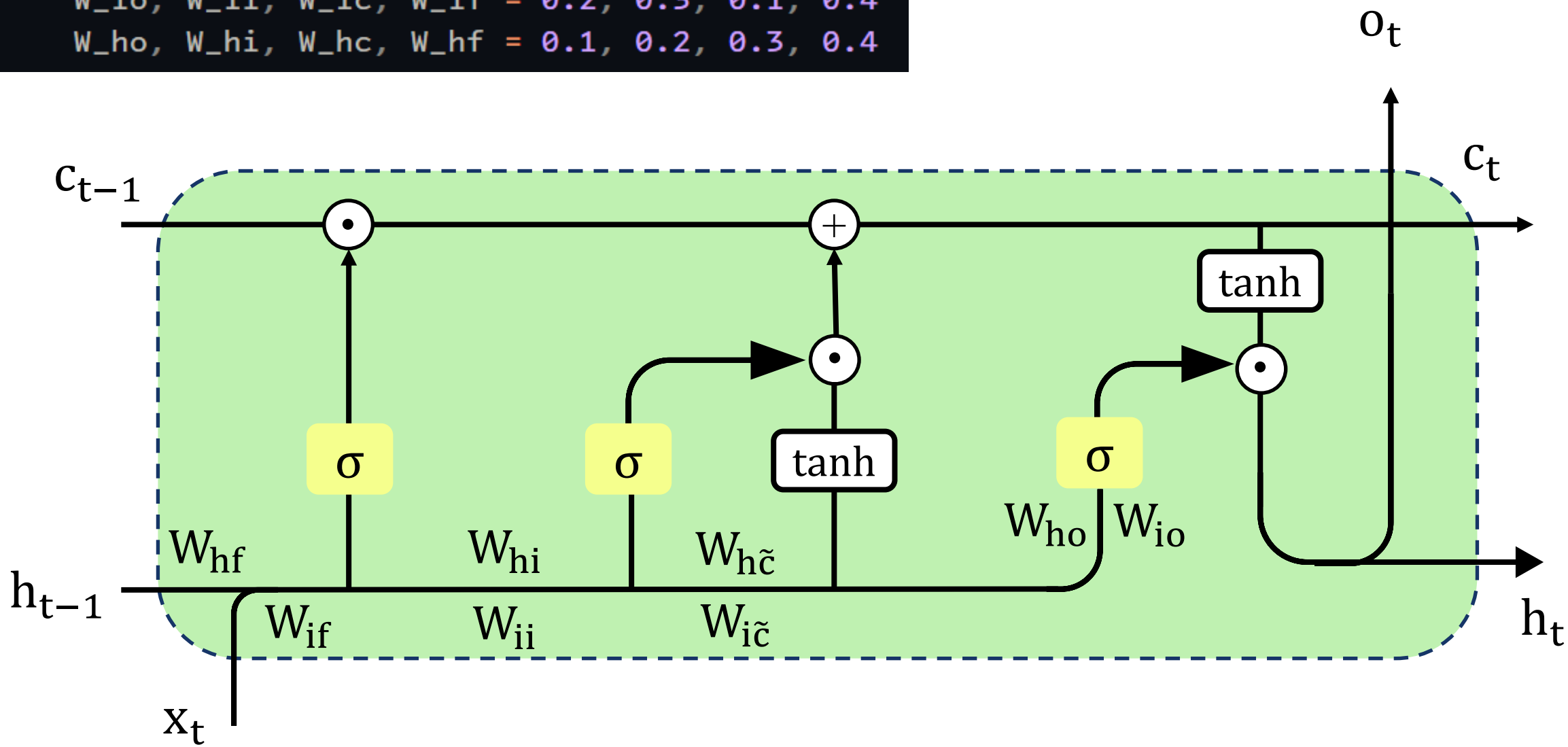
Bidirectional RNN

❖ $t = 2$

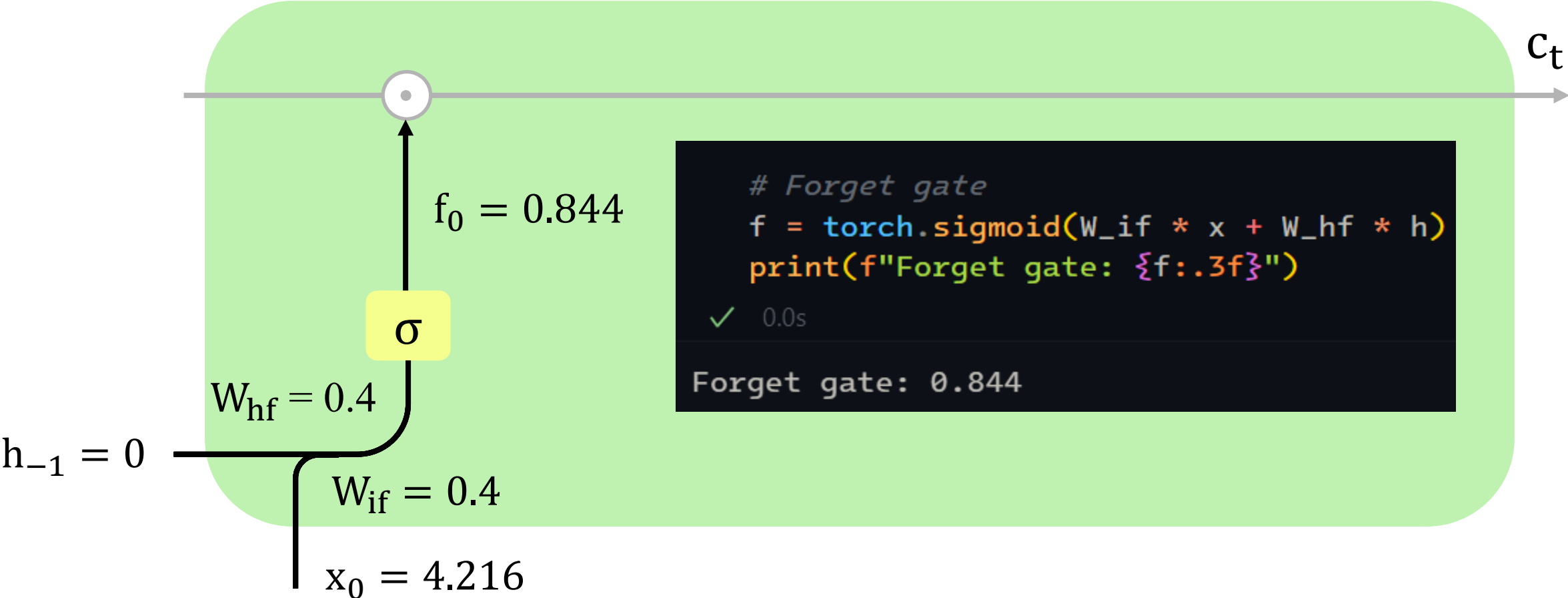


LSTM

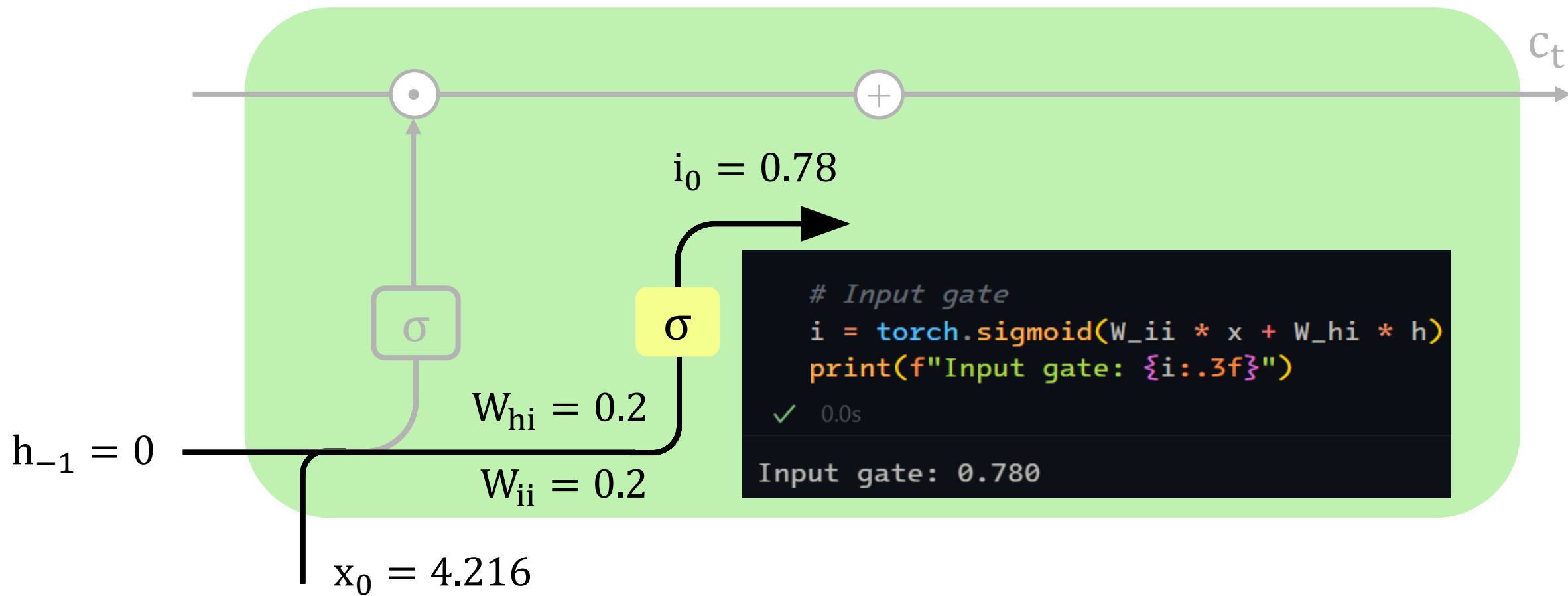
```
W_io, W_ii, W_ic, W_if = 0.2, 0.3, 0.1, 0.4  
W_ho, W_hi, W_hc, W_hf = 0.1, 0.2, 0.3, 0.4
```



❖ $t = 0$



```
# Forget gate
f = torch.sigmoid(W_if * x + W_hf * h)
print(f"Forget gate: {f:.3f}")
✓ 0.0s
Forget gate: 0.844
```

❖ $t = 0$ 

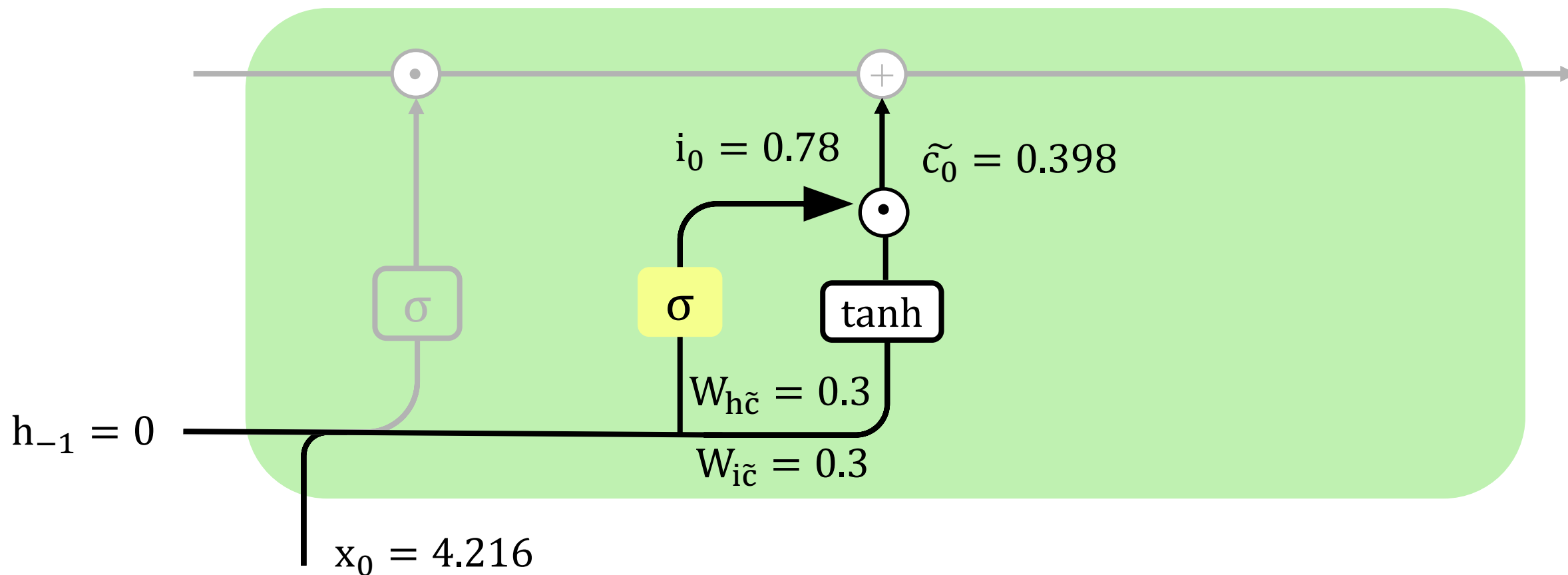
LSTM

❖ $t = 0$

```
c_dash = torch.tanh(W_ic * x + W_hc * h)
print(f"Cell gate: {c_dash:.3f}")
```

✓ 0.0s

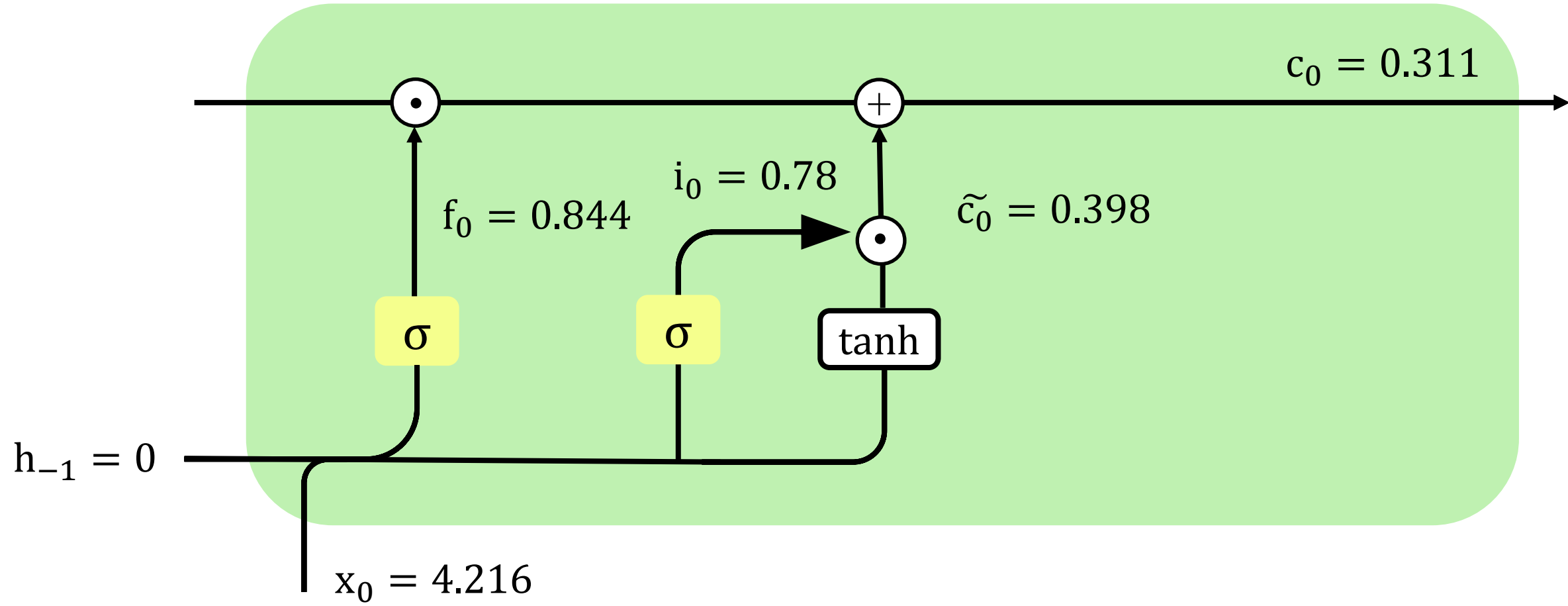
Cell gate: 0.398



LSTM

❖ $t = 0$

```
# Cell state
cell_state = f * c + i * c_dash
print(f"Cell state: {cell_state:.3f}")
✓ 0.0s
Cell state: 0.311
```



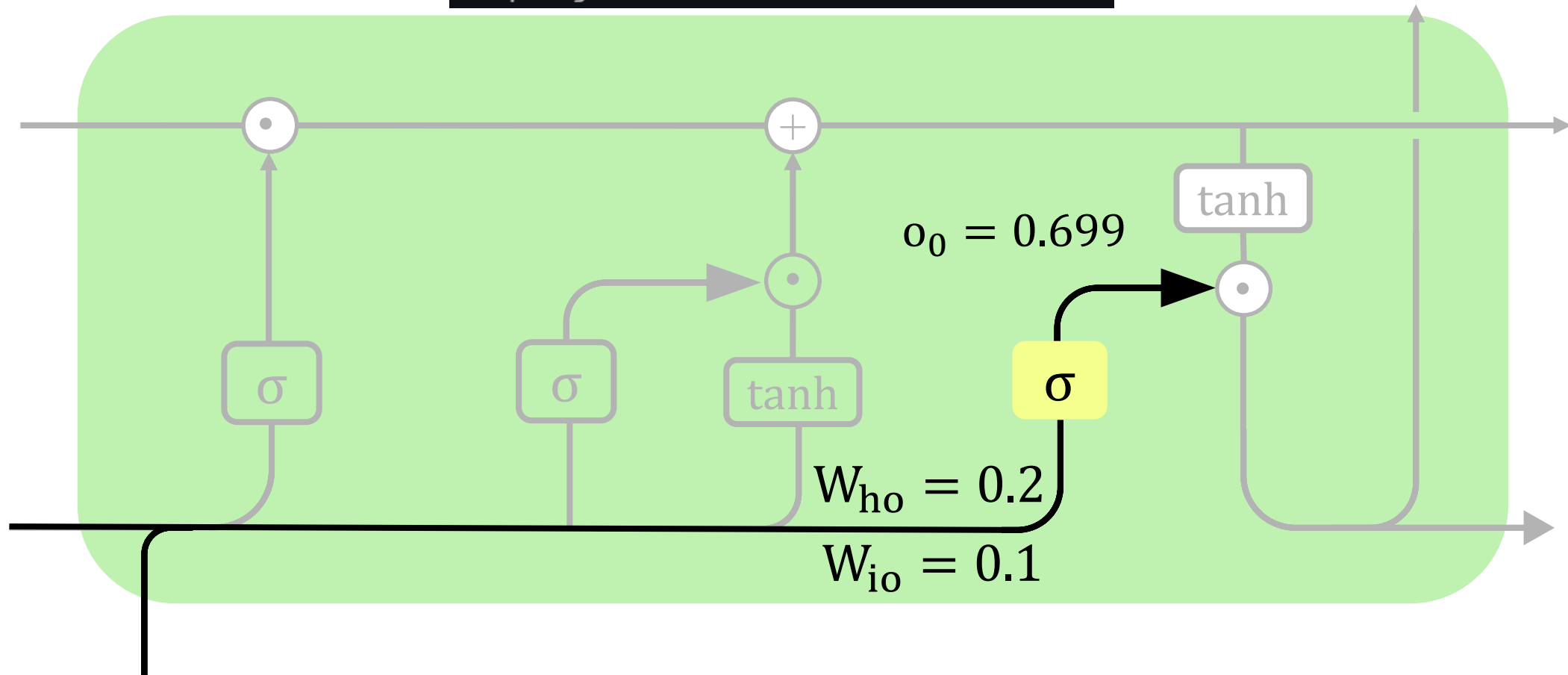
LSTM

❖ $t = 0$

```
# Output gate
o = torch.sigmoid(W_io * x + W_ho * h)
print(f"Output gate: {o:.3f}")

✓ 0.0s

Output gate: 0.699
```



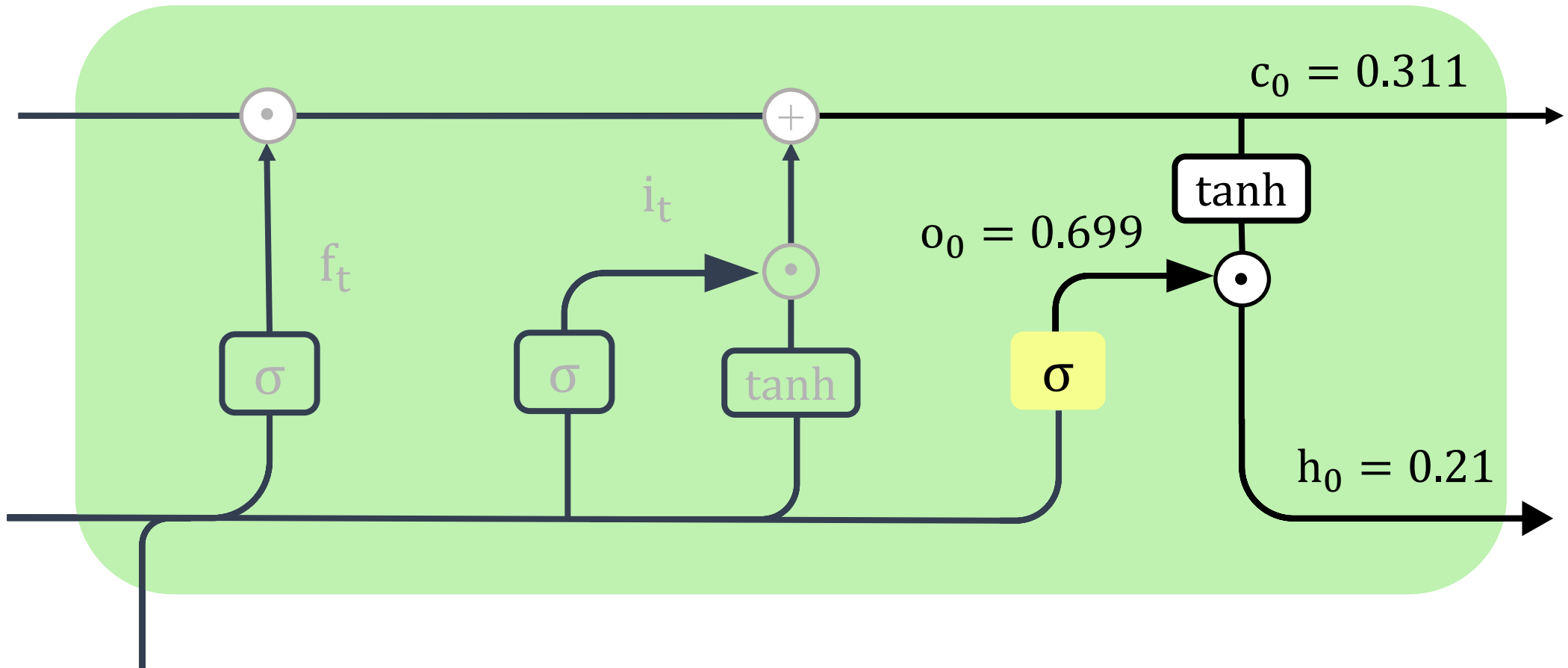
LSTM

❖ $t = 0$

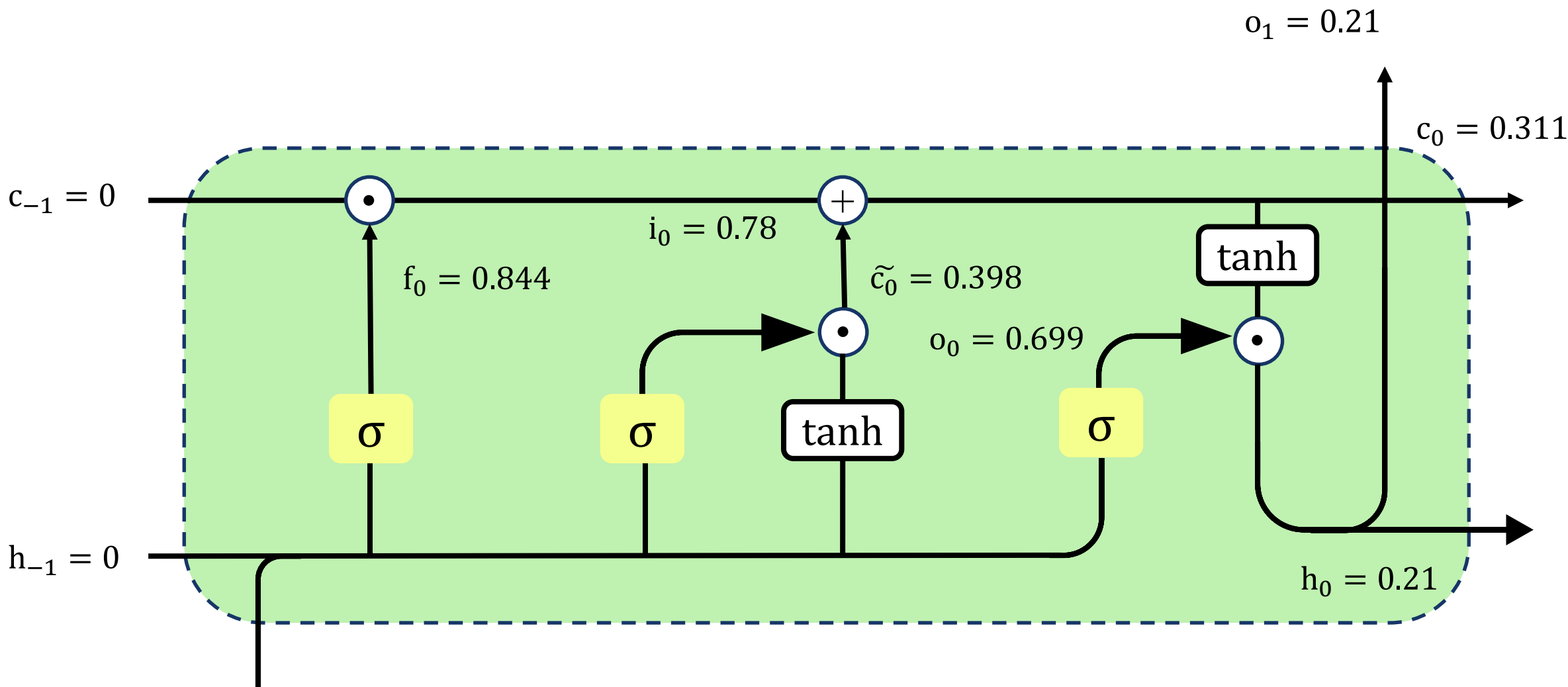
```
# Hidden state
hidden_state = o * torch.tanh(cell_state)
print(f"Hidden state: {hidden_state:.3f}")

✓ 0.0s

Hidden state: 0.210
```

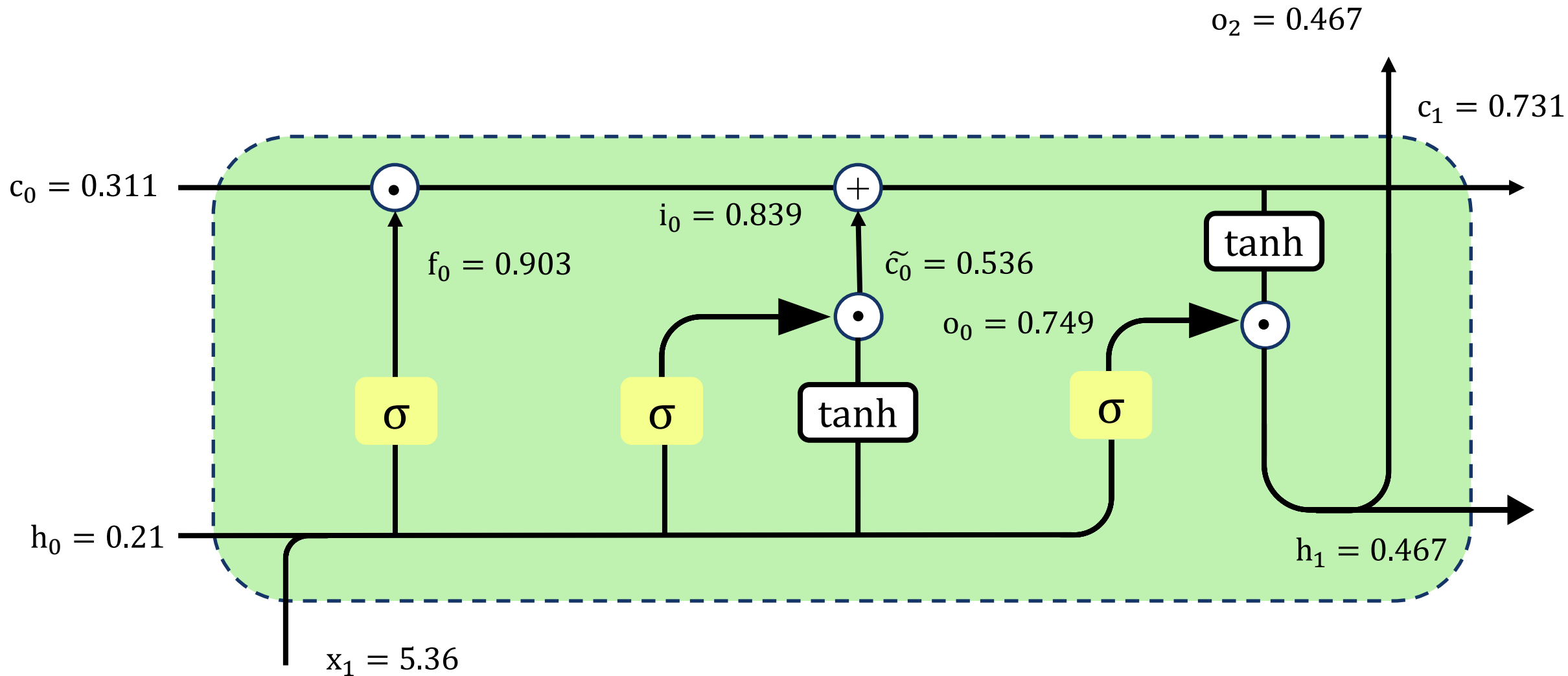


❖ $t = 0$

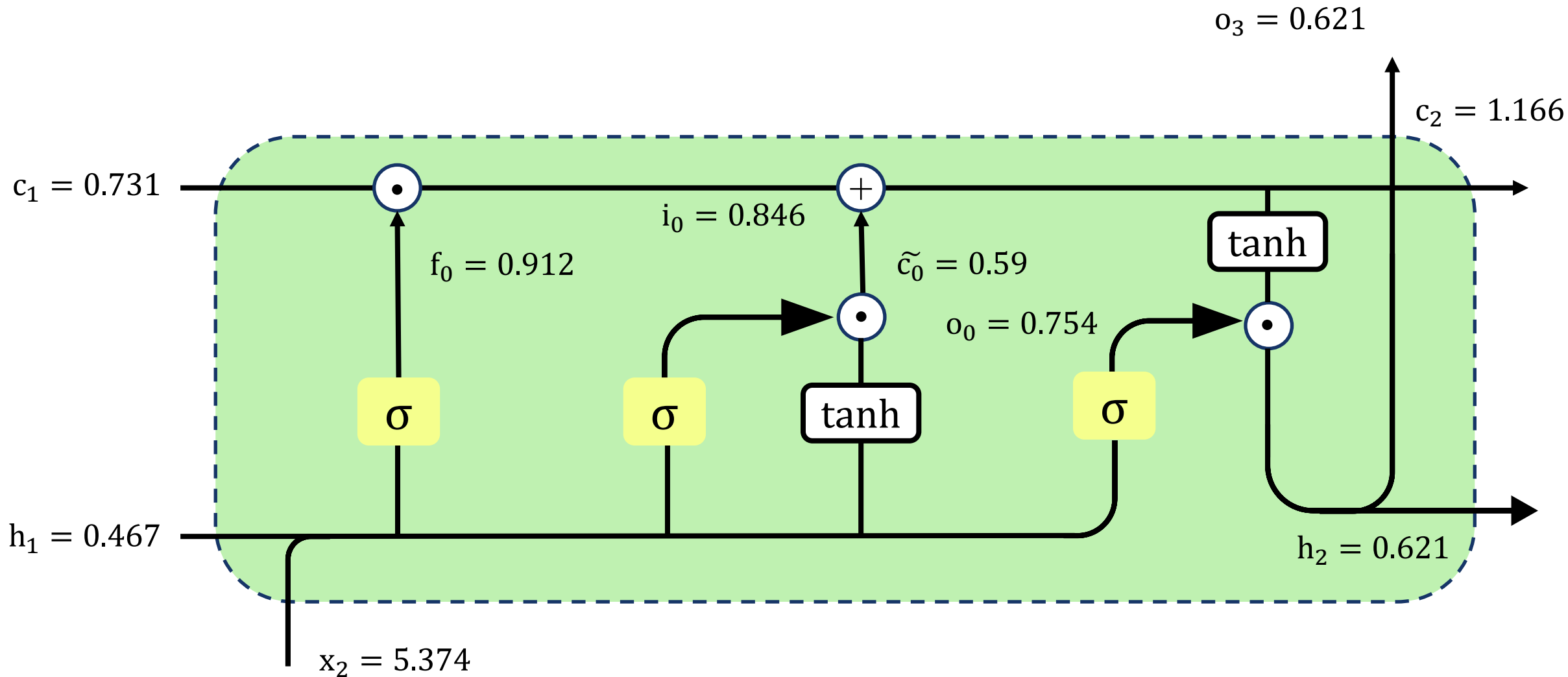


LSTM

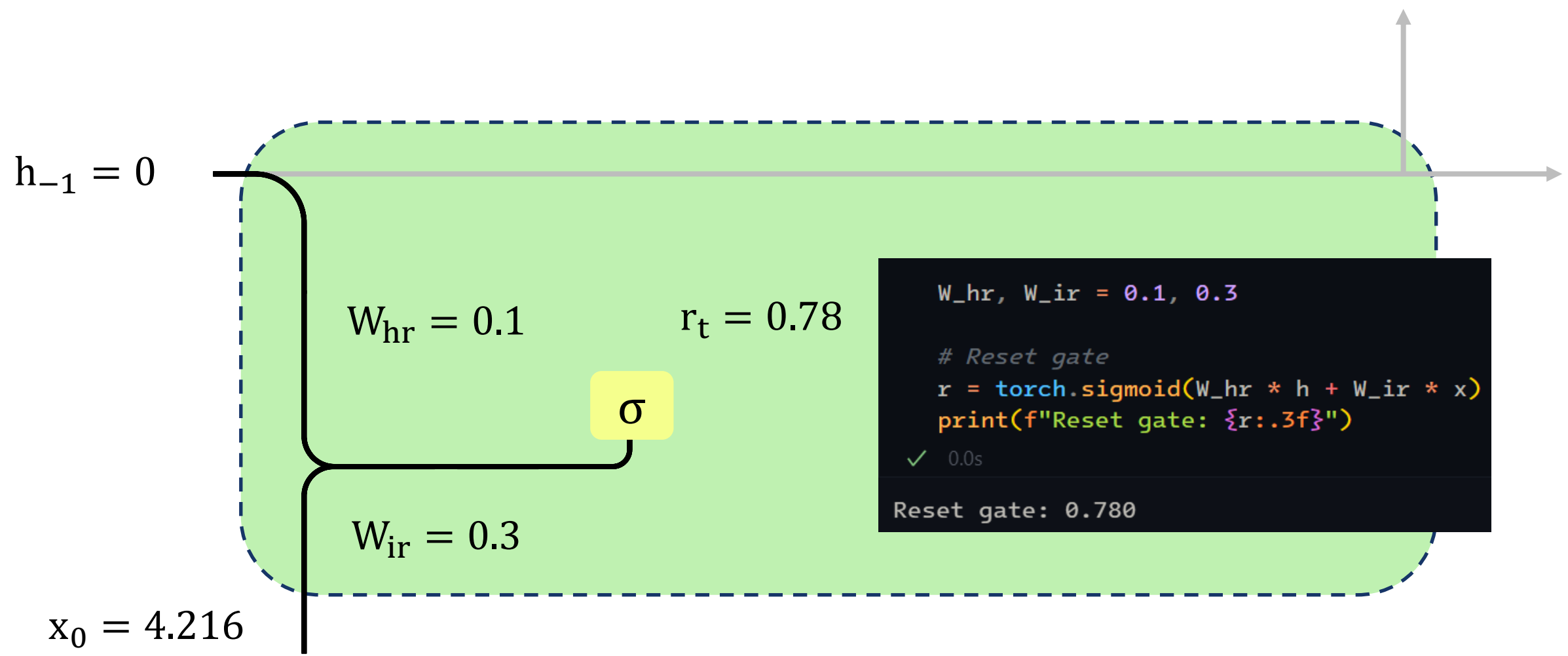
❖ $t = 1$



❖ $t = 2$



❖ $t = 0$

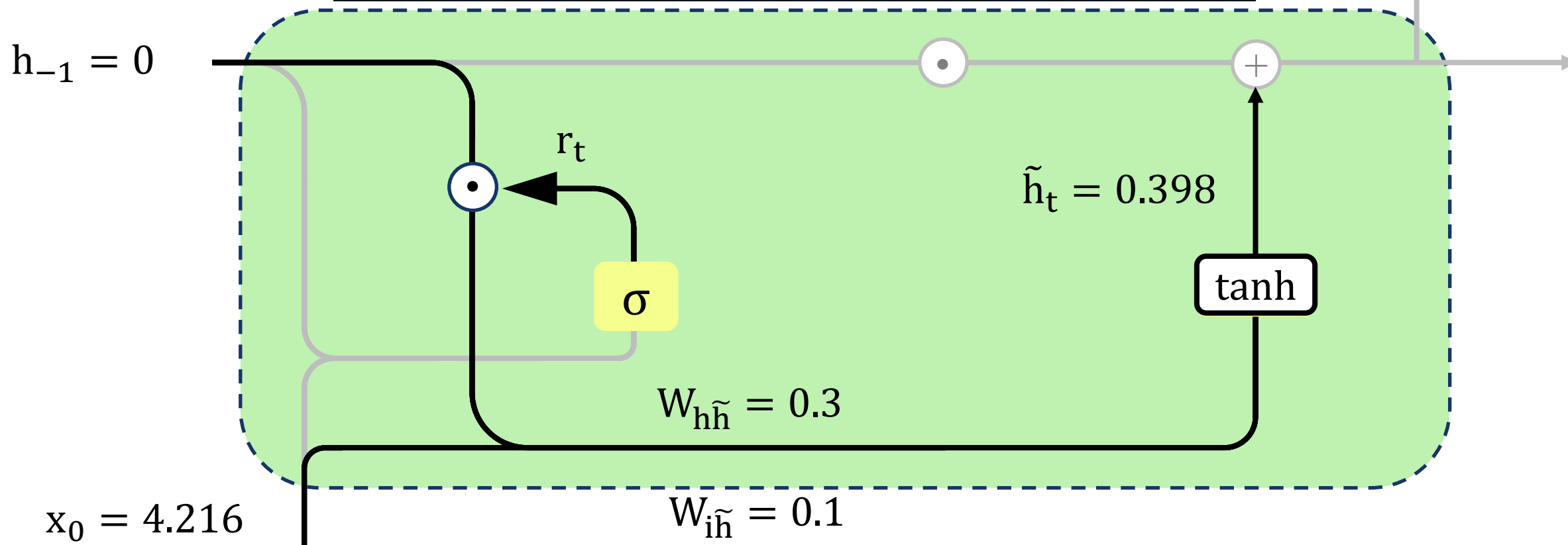


❖ $t = 0$

```
h_dash = torch.tanh(W_hh * (r * h) + W_ih * x)
print(f"Hidden gate: {h_dash:.3f}")
```

✓ 0.0s

Hidden gate: 0.398

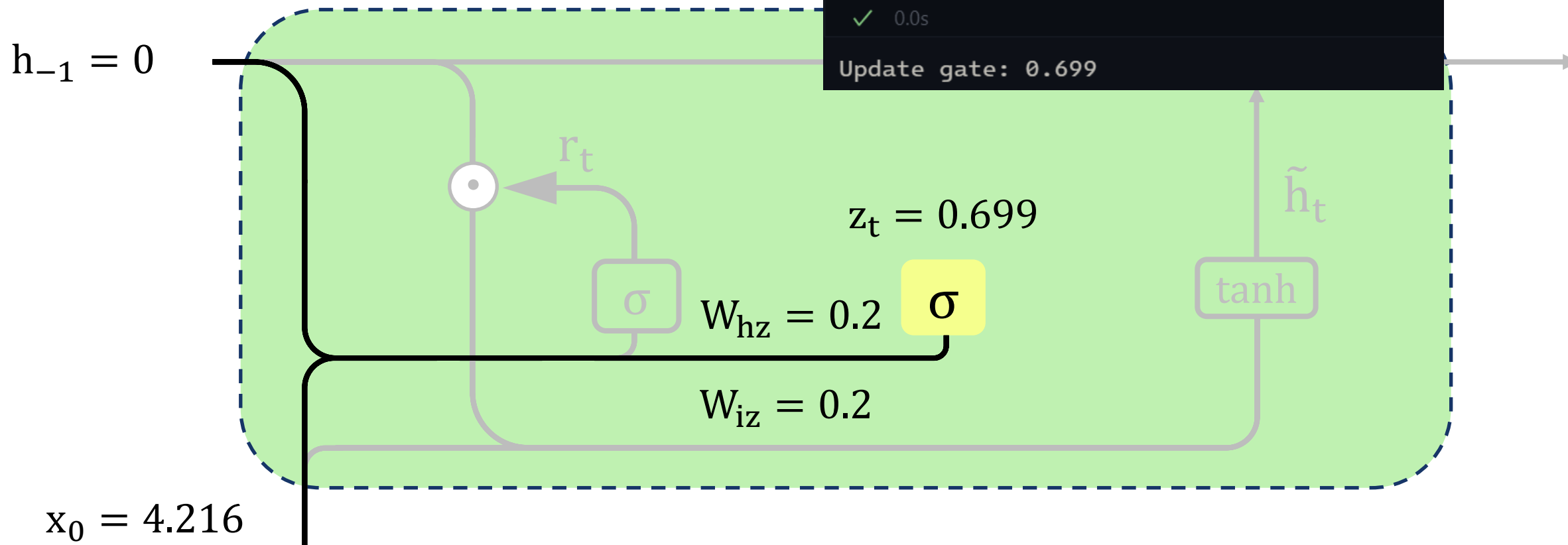


❖ $t = 0$

```
W_hz, W_iz = 0.2, 0.2

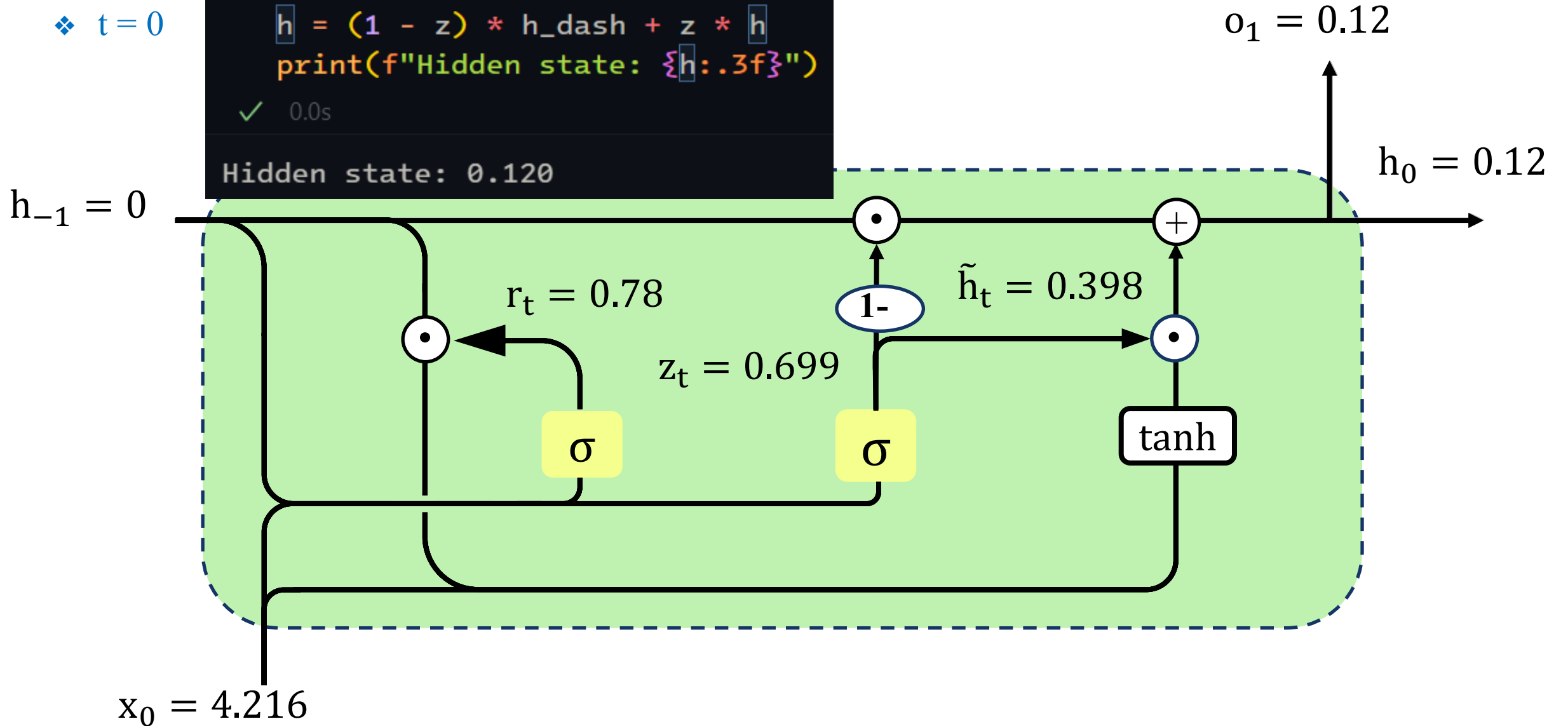
# Update gate
z = torch.sigmoid(W_hz * h + W_iz * x)
print(f"Update gate: {z:.3f}")

✓ 0.0s
Update gate: 0.699
```

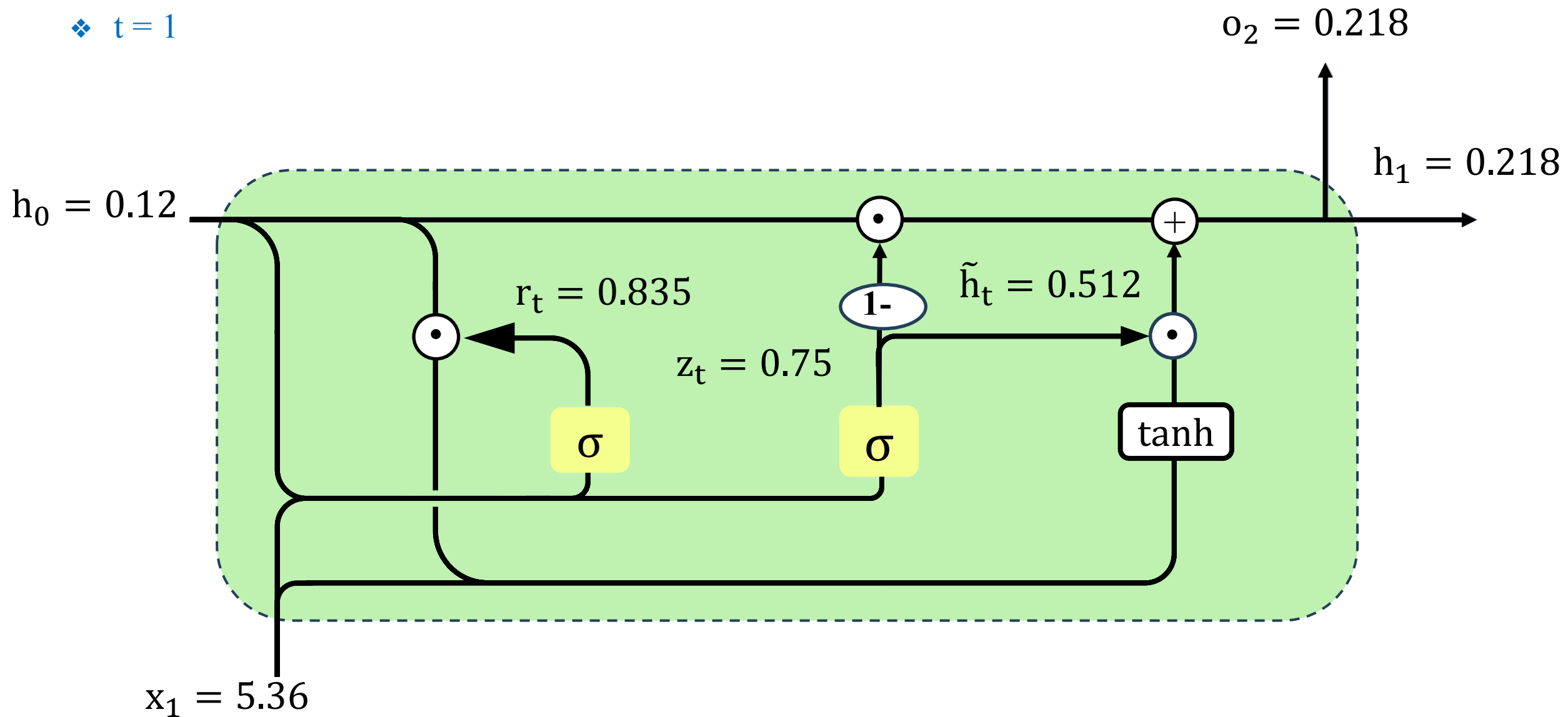


❖ $t = 0$

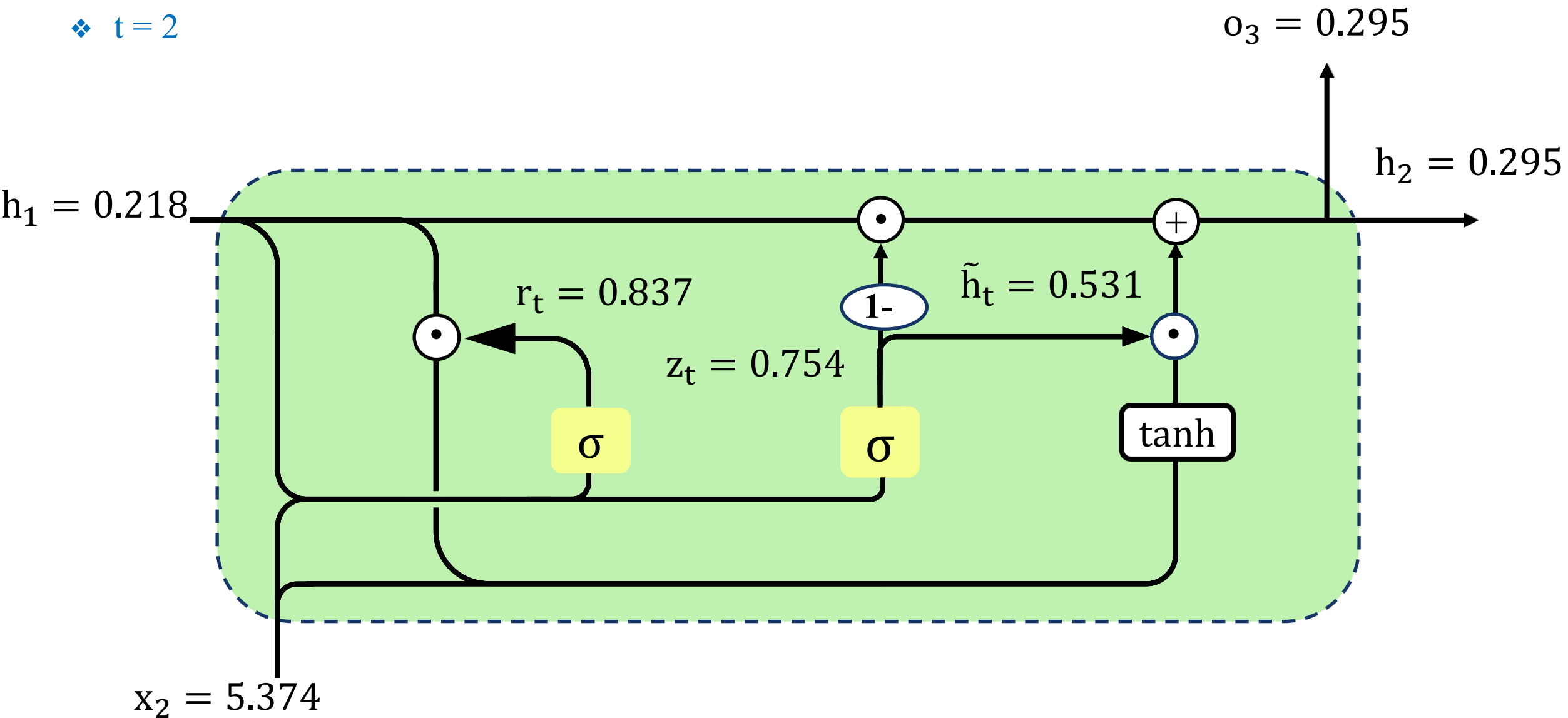
```
h = (1 - z) * h_dash + z * h
print(f"Hidden state: {h:.3f}")
✓ 0.0s
Hidden state: 0.120
```



❖ $t = 1$



❖ $t = 2$



Outline

- **Review: Recurrent Neural Network (RNN)**
- **Long-Short Term Memory (LSTM)**
- **Gated Recurrent Unit (GRU)**
- **Time Series – Basic Knowledge**
- **LSTM, GRU for Time Series Data**
- **LSTM, GRU for Text Data**



Dataset

Sentiment Analysis

Text	Sentiment
I love programming	1
This is a boring task	0
This is not interesting	0

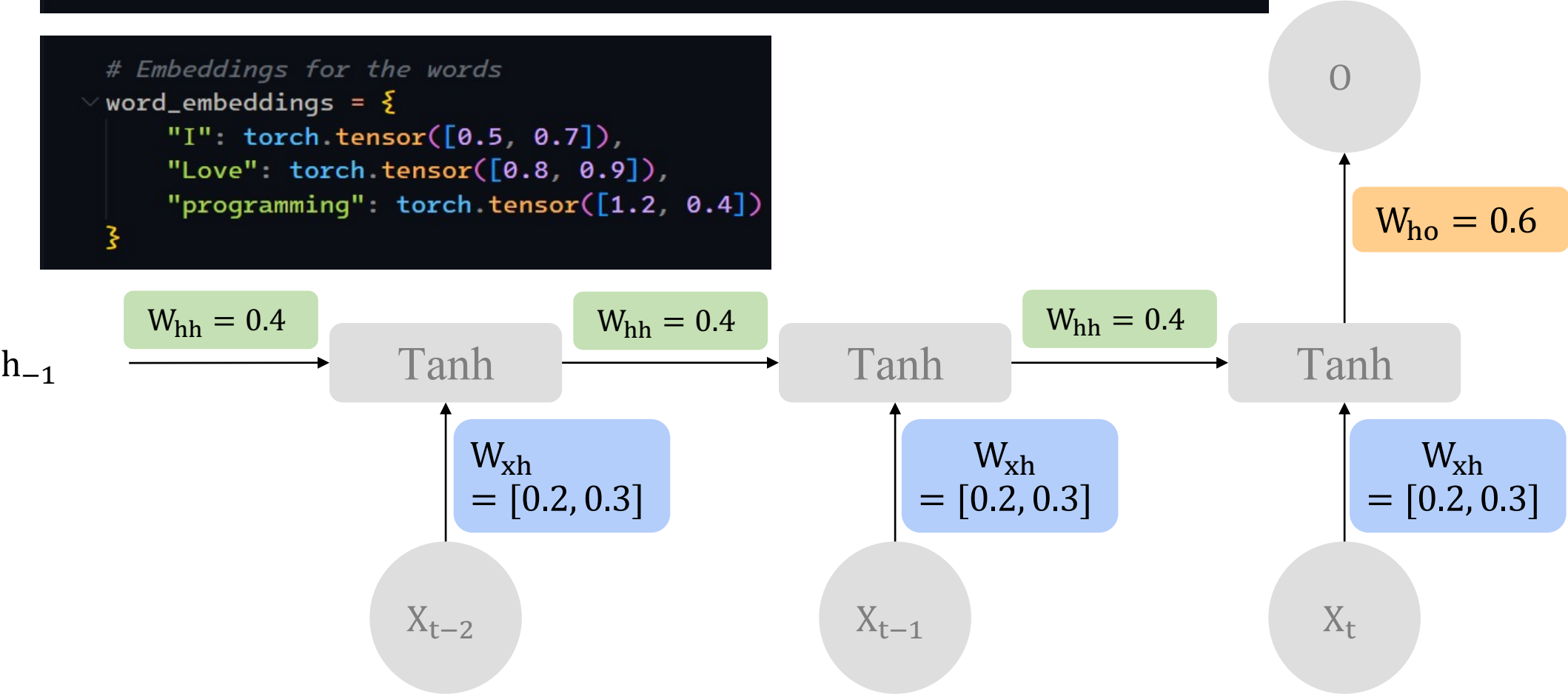
...

Word	Embedding		Sentiment
I	0.5	0.7	1
Love	0.8	0.9	
programming	1.2	0.4	

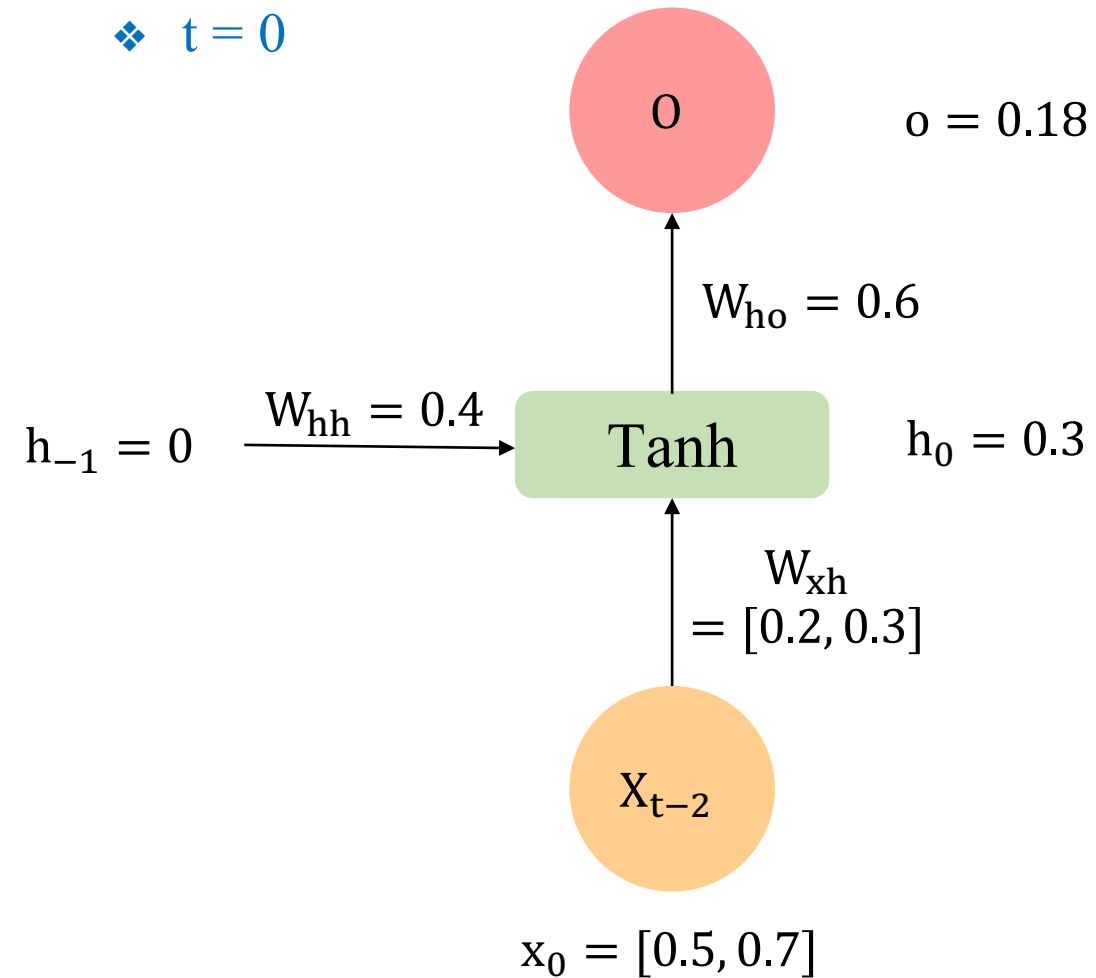
...

```
# Define RNN weights
W_xh = torch.tensor([[0.2, 0.3]]) # Shape (hidden_size, embedding_size)
W_hh = torch.tensor([[0.4]]) # Shape (hidden_size, hidden_size)
W_ho = torch.tensor([[0.6]]) # Shape (output_size, hidden_size)
```

```
# Embeddings for the words
word_embeddings = {
    "I": torch.tensor([0.5, 0.7]),
    "Love": torch.tensor([0.8, 0.9]),
    "programming": torch.tensor([1.2, 0.4])
}
```



❖ $t = 0$



```
print(f"Processing word: {word}")
print("Embedding:", embedding)

# Compute new hidden state
h_new = torch.tanh(torch.matmul(W_xh, embedding) + torch.matmul(W_hh, h))
print("New hidden state:", h_new)

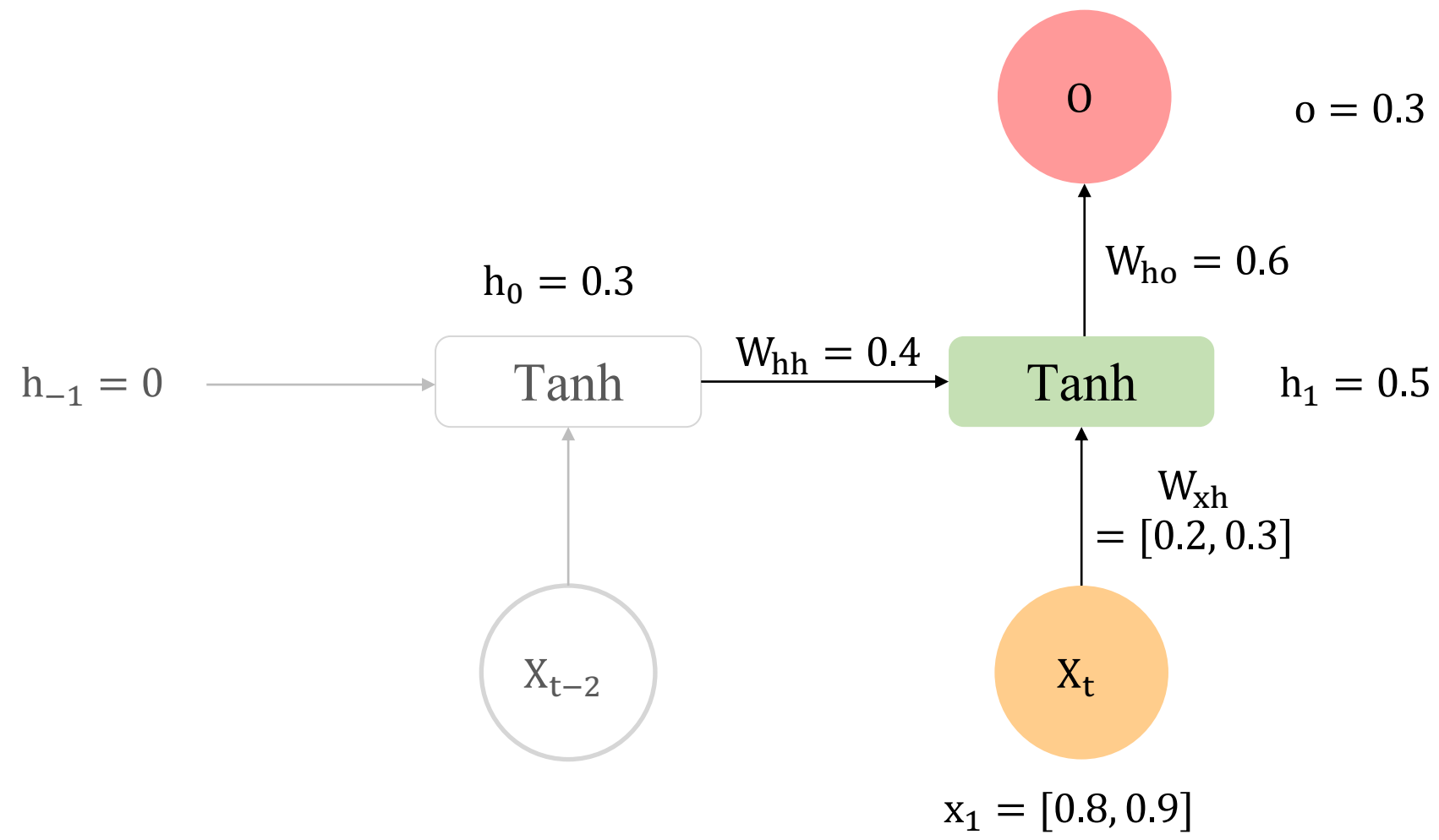
# Compute output
o = torch.matmul(W_ho, h_new)
print("Output:", o)

# Update hidden state
h = h_new
print("-" * 30)
```

✓ 0.0s

```
Processing word: I
Embedding: tensor([0.5000, 0.7000])
New hidden state: tensor([0.3004])
Output: tensor([0.1803])
-----
```

❖ $t = 1$



```
word = 'Love'
embedding = word_embeddings[word]

print(f"Processing word: {word}")
print("Embedding:", embedding)

# Compute new hidden state
h_new = torch.tanh(
    torch.matmul(W_xh, embedding) +
    torch.matmul(W_hh, h)
)
print("New hidden state:", h_new)

# Compute output
o = torch.matmul(W_ho, h_new)
print("Output:", o)

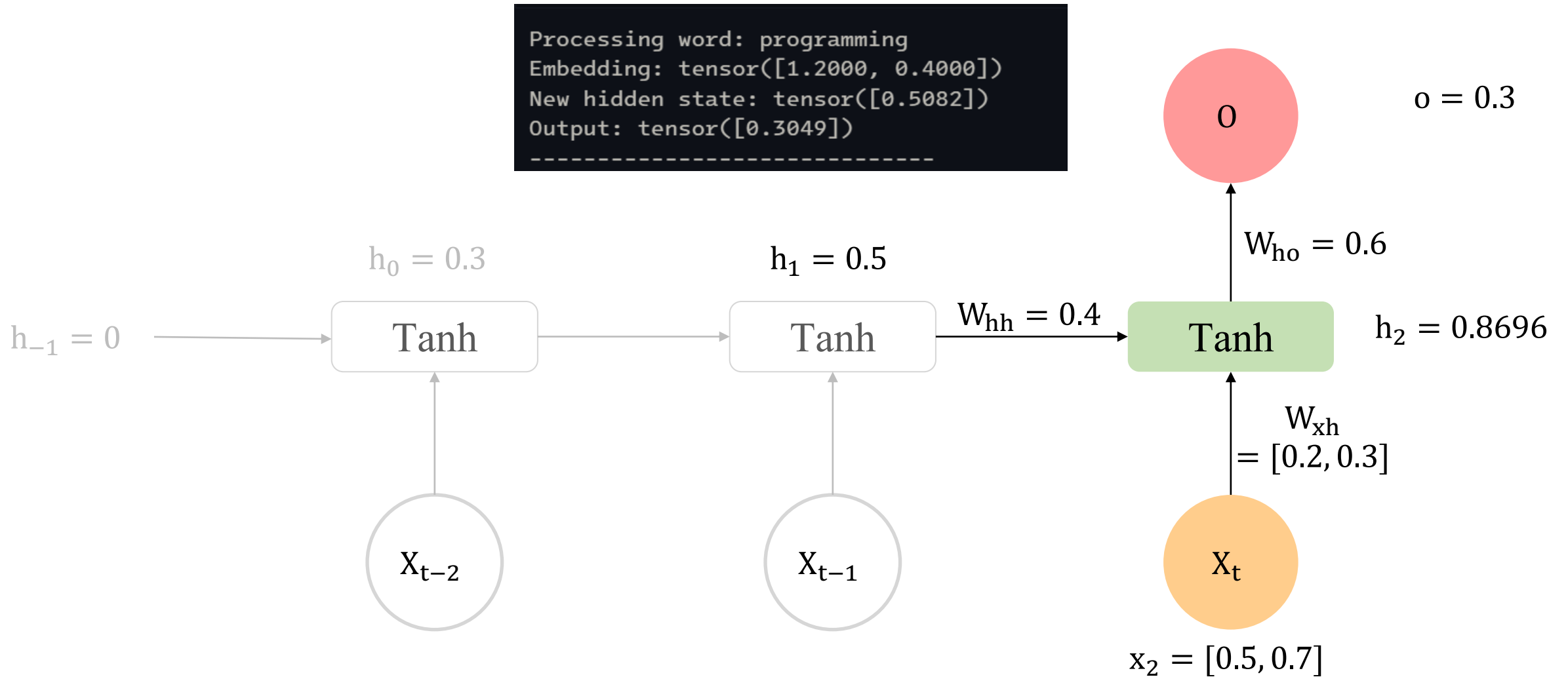
# Update hidden state
h = h_new
print("-" * 30)

✓ 0.0s
```

```
Processing word: Love
Embedding: tensor([0.8000, 0.9000])
New hidden state: tensor([0.5007])
Output: tensor([0.3004])
-----
```

RNN

❖ $t = 2$



```
# Define LSTM weights
```

```
W_if = torch.tensor([[0.4, 0.3]])
```

```
W_ii = torch.tensor([[0.5, 0.6]])
```

```
W_ic = torch.tensor([[0.2, 0.4]])
```

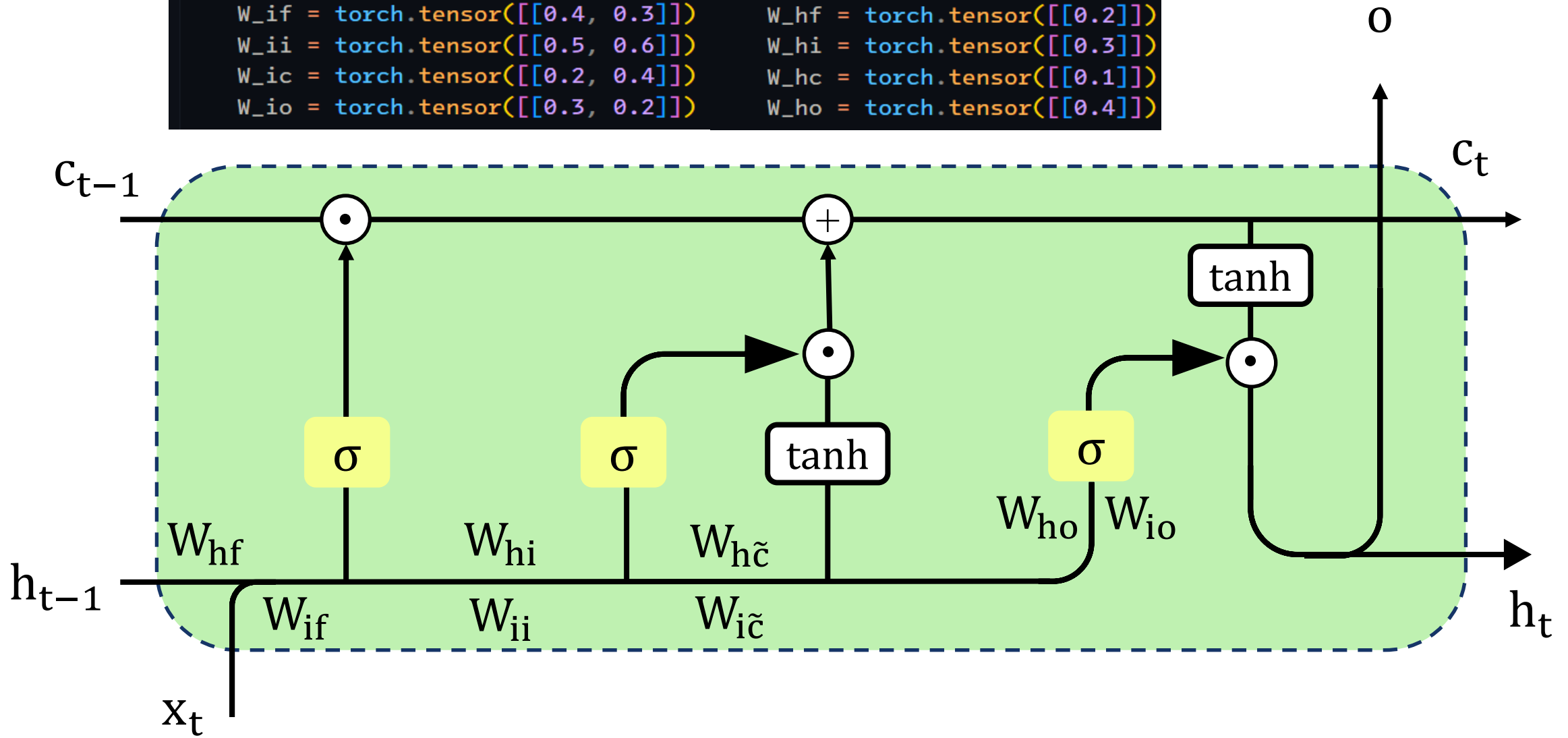
```
W_io = torch.tensor([[0.3, 0.2]])
```

```
W_hf = torch.tensor([[0.2]])
```

```
W_hi = torch.tensor([[0.3]])
```

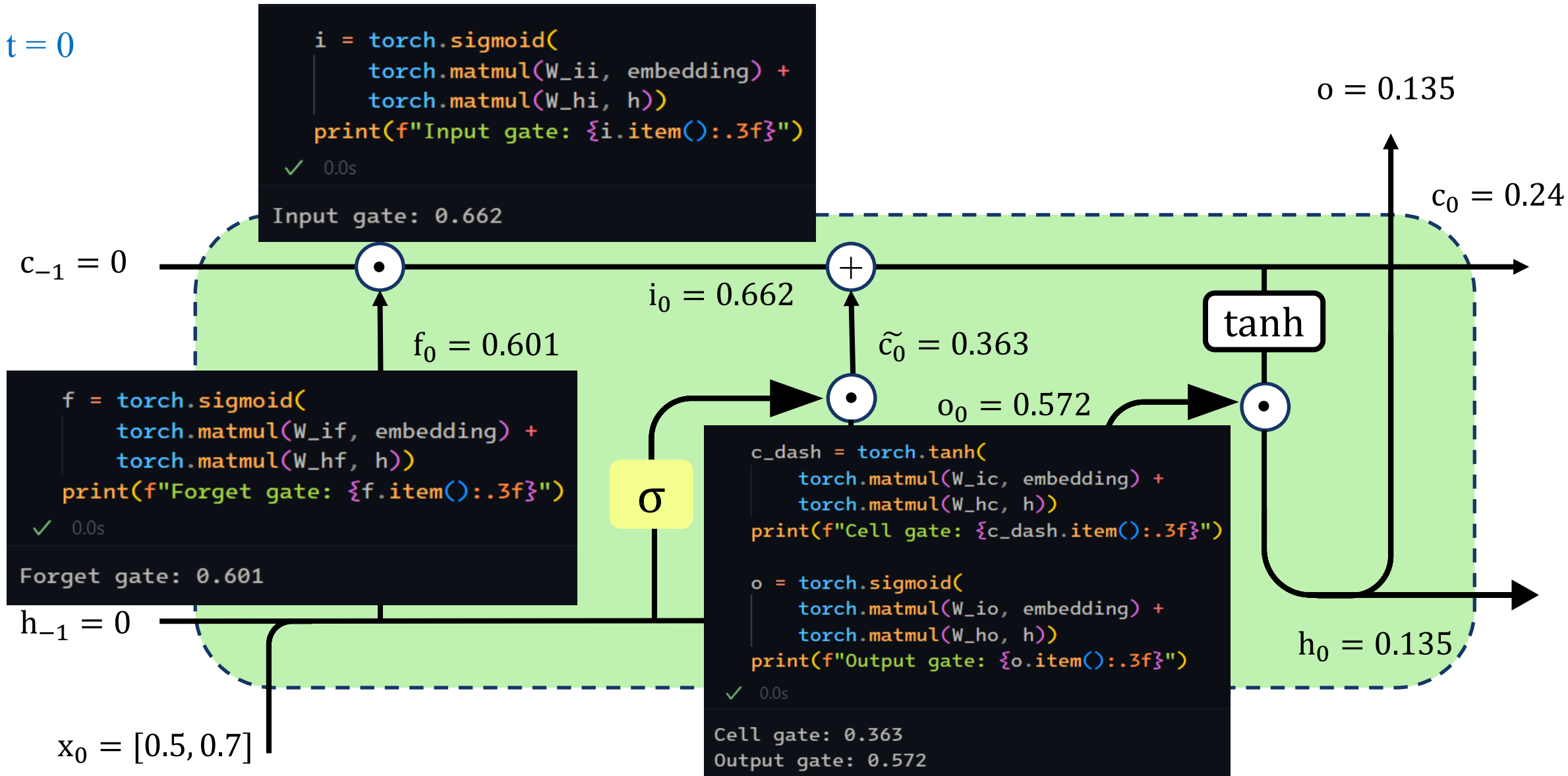
```
W_hc = torch.tensor([[0.1]])
```

```
W_ho = torch.tensor([[0.4]])
```

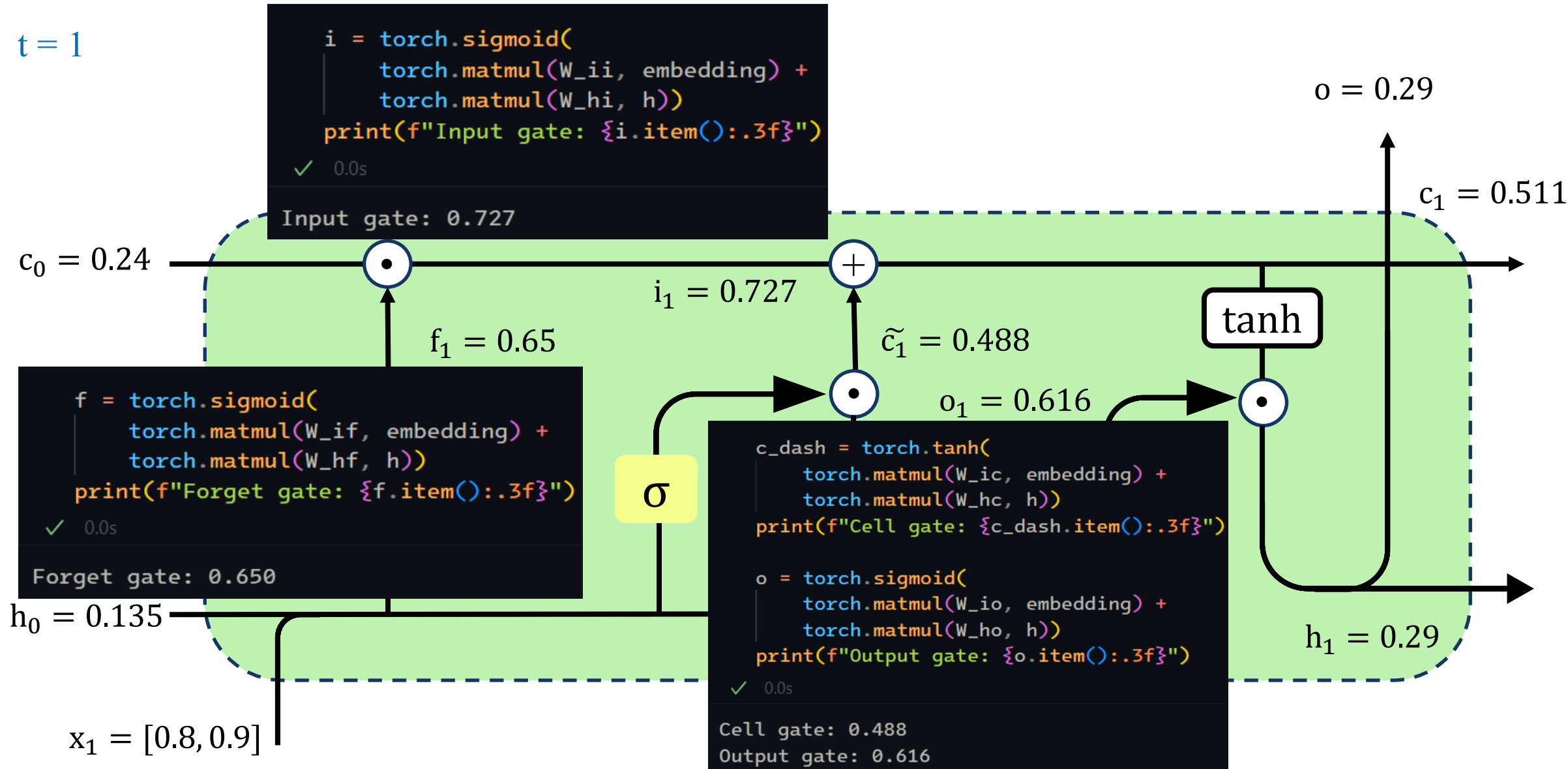


LSTM

❖ $t = 0$

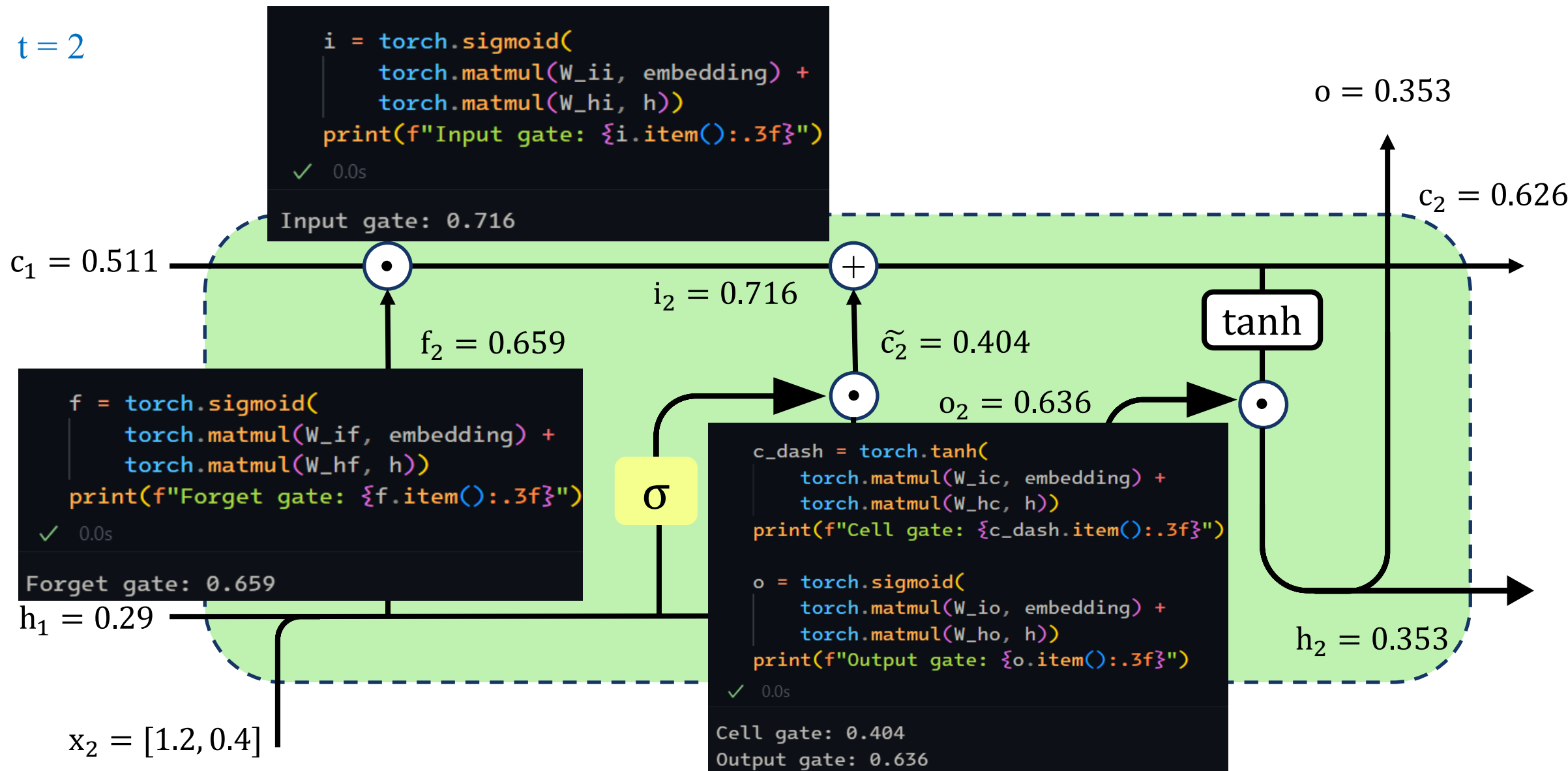


❖ $t = 1$

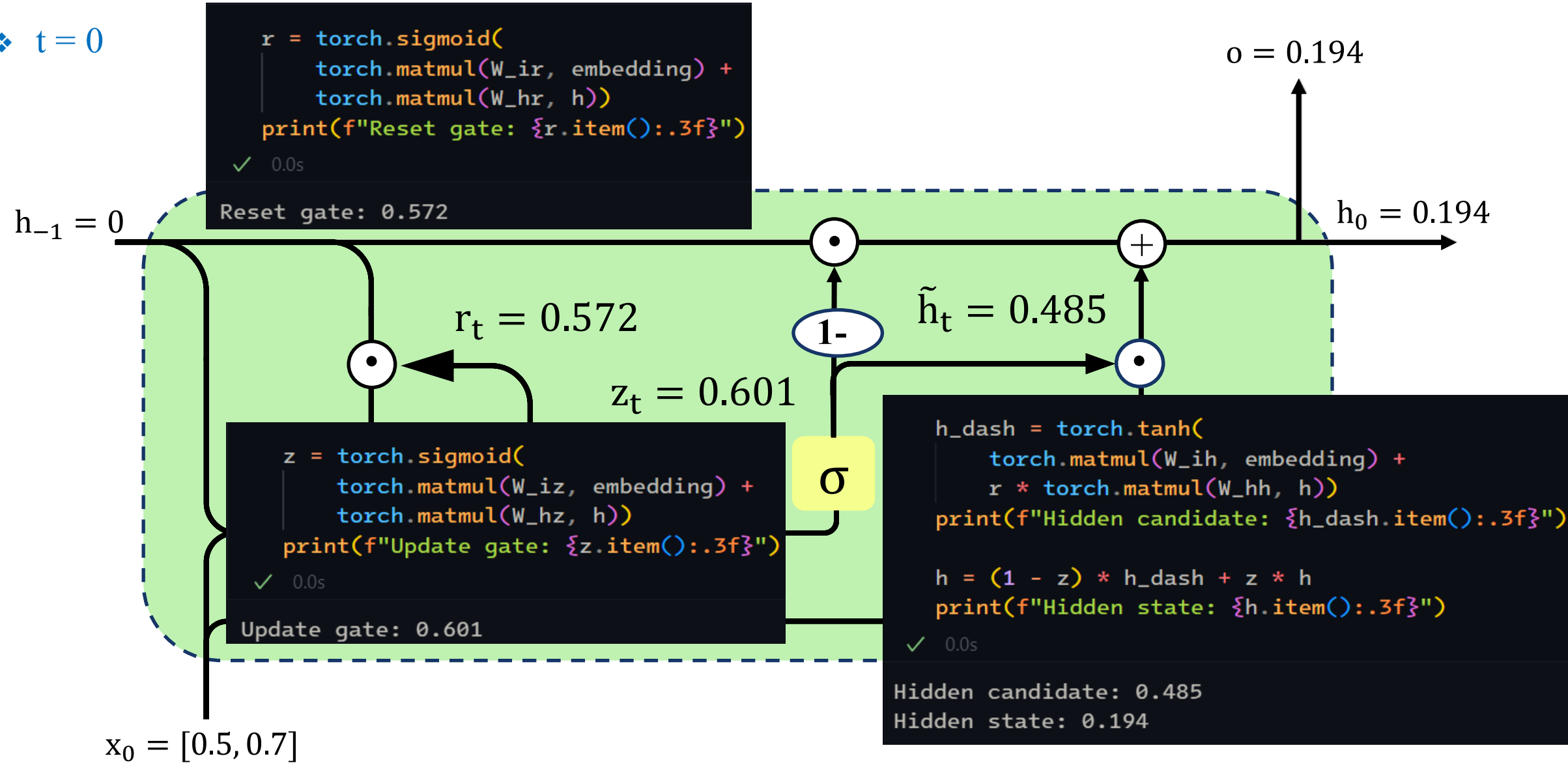


LSTM

❖ $t = 2$



❖ $t = 0$



❖ $t = 1$

```
r = torch.sigmoid(
    torch.matmul(W_ir, embedding) +
    torch.matmul(W_hr, h))
print(f"Reset gate: {r.item():.3f}")
```

✓ 0.0s

Reset gate: 0.608

$h_0 = 0.194$

$r_t = 0.608$

$z_t = 0.652$

```
z = torch.sigmoid(
    torch.matmul(W_iz, embedding) +
    torch.matmul(W_hz, h))
print(f"Update gate: {z.item():.3f}")
```

✓ 0.0s

Update gate: 0.652

σ

```
h_dash = torch.tanh(
    torch.matmul(W_ih, embedding) +
    r * torch.matmul(W_hh, h))
print(f"Hidden candidate: {h_dash.item():.3f}")
```

```
h = (1 - z) * h_dash + z * h
print(f"Hidden state: {h.item():.3f}")
```

✓ 0.0s

Hidden candidate: 0.661

Hidden state: 0.356

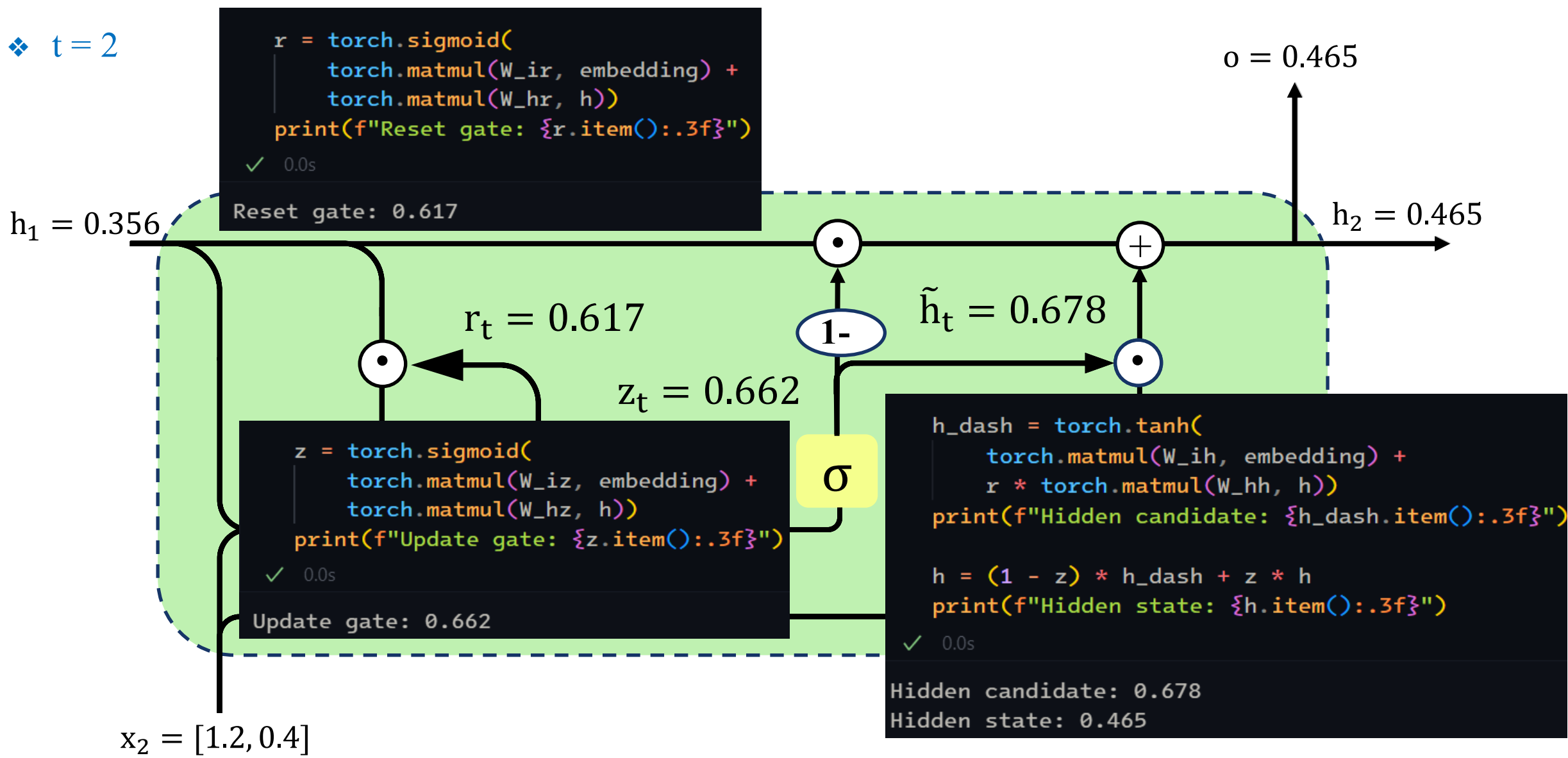
$o = 0.356$

$h_1 = 0.356$

$x_1 = [0.8 \ 0.9]$

GRU

❖ $t = 2$



Summary

Architecture	RNN	LSTM/GRU	Transformer
Advantages	Simple, easy to run	Reduce vanishing/exploding gradient Know what to forget, input and output	Non-sequential models process data in parallel
Disadvantages	Gradient Vanishing/Exploding Unable to choose what to remember	Complexity	Black-box model
Training Process	Sequence to Sequence	Sequence to Sequence	Parallel
Long-range dependencies	Short-term	Long-term & Short-term	Super Long-term & Short-term
Time	Slow	Slower	Depends on the architecture

